Advanced technical concepts for Implicit Neural Representations:

Meta-Learning, Compression and Functa

Jonathan Richard Schwarz - schwarzjn@gmail.com @schwarzjn_

MICCAI 2024 Tutorial on
Implicit Neural Representations for Medical Imaging

06/10/2024

Joint work with:

Matthias Bauer

Emilien Dupont

Jihoon Tack

Hyunjik Kim

Yee Whye Teh

& many others

Outline

1 – Introduction: Data as Functions

- From data to functa: Your data point is a function and you can treat it like one [ICML 2022]
- COIN: COmpression with Implicit Neural representations [2021]

2 – Algorithms and Architectures for Neural Fields

- Meta-Learning INRs
  - Efficient Meta-Learning via Error-based Context Pruning for Implicit Neural Representations [NeurIPS 2023]
- Better parameterizations & Advanced Compression
  - Meta-Learning Sparse Compression Networks [TMLR 2022]
  - Modality-Agnostic Variational Compression of Implicit Neural Representations [ICML 2023]
  - C3: High-performance and low-complexity neural compression from a single image or video [CVPR 2024]

Outline

1 – Introduction: Data as Functions

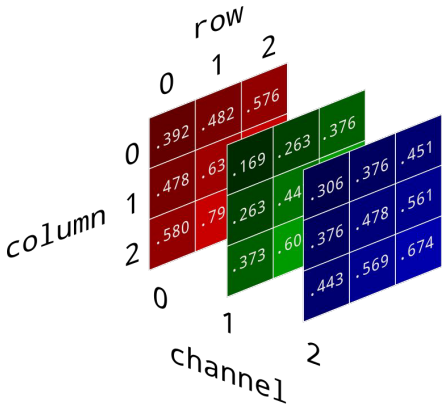
- From data to functa: Your data point is a function and you can treat it like one [ICML 2022]
- COIN: COmpression with Implicit Neural representations [2021]

2 – Algorithms and Architectures for Neural Fields

- Meta-Learning INRs
  - Efficient Meta-Learning via Error-based Context Pruning for Implicit Neural Representations [NeurIPS 2023]
- Better parameterizations & Advanced Compression
  - Meta-Learning Sparse Compression Networks [TMLR 2022]
  - Modality-Agnostic Variational Compression of Implicit Neural Representations [ICML 2023]
  - C3: High-performance and low-complexity neural compression from a single image or video [CVPR 2024]

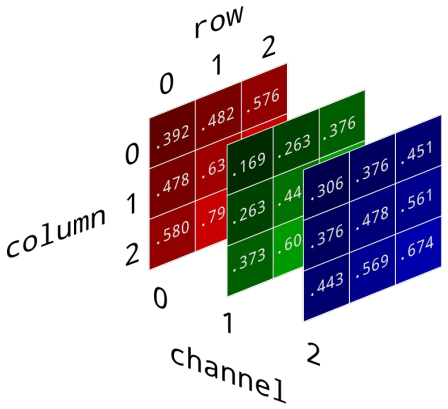
Representing data points as functions

In DL, each datapoint is usually treated as an array e.g. images are HxWxC arrays:

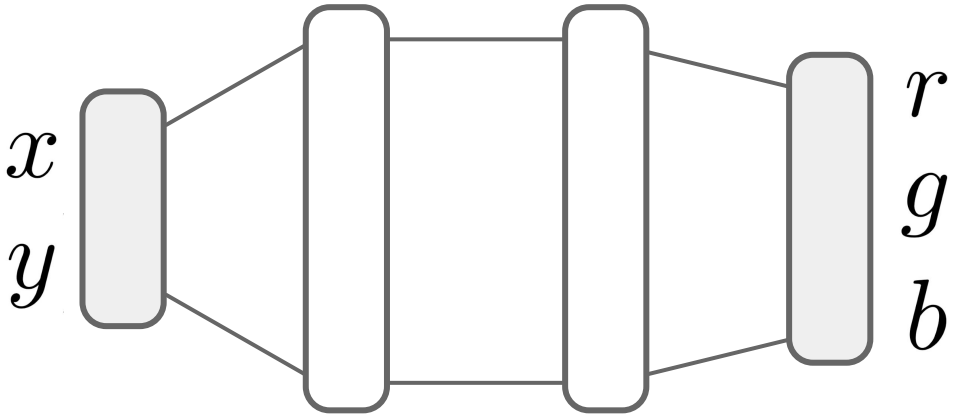


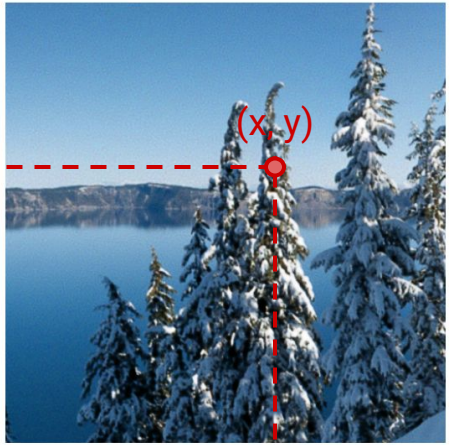
Representing data points as functions

In DL, each datapoint is usually treated as an array e.g. images are HxWxC arrays:



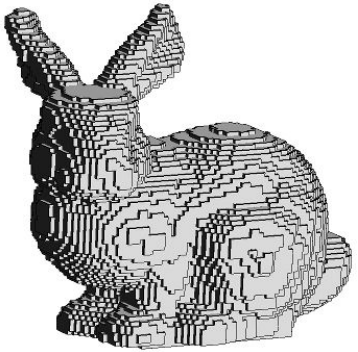
Instead, we can think of each **individual datapoint** as a **function**:

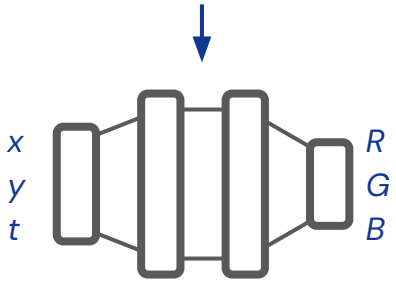


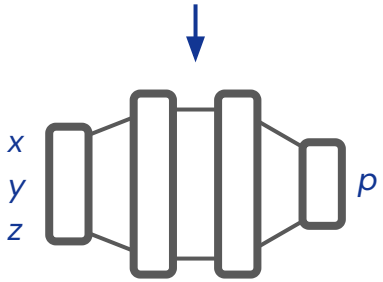


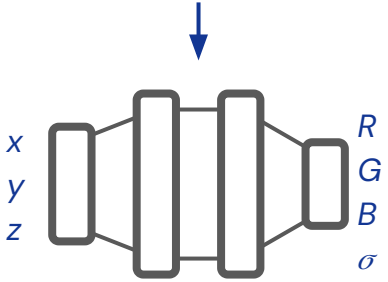
Representing data points as functions

Representing each data point with a continuous function can be done with various data modalities:







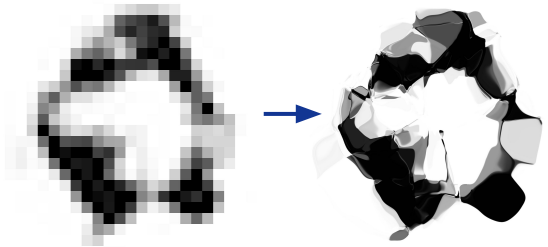


These functions f_θ can be parameterised by an MLP and fit to data point by minimising reconstruction error:
e.g. for any specific image:

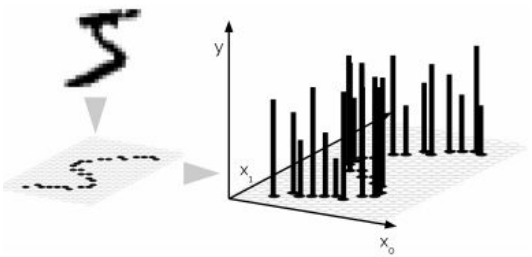
$$\min_\theta \sum_{x,y} \| f_\theta(x,y) - I[x,y] \|_2^2$$

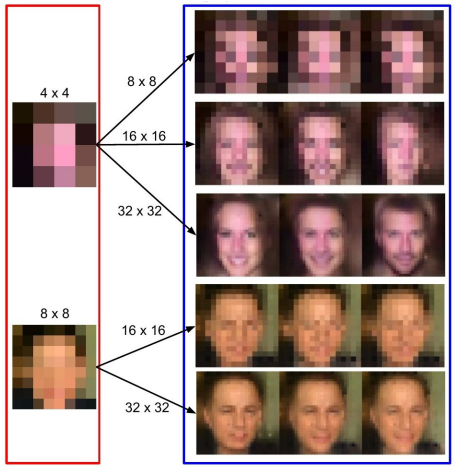
A brief history of representing data points as functions

[1] Represented MNIST as such 2D functions functions and built generative models of MNIST digits as functions



This idea was also explored in Neural Processes [2, 3, 4] that also learn distributions over functions

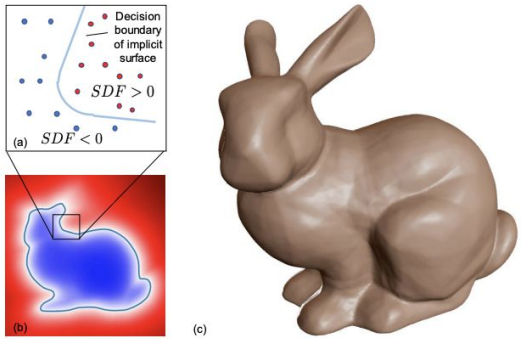




[1] *Generating large images from latent vectors*, David Ha, 2016. blog.otoro.net
[2] *Neural Processes,* Garnelo et al., 2018.
[3] *Attentive Neural Processes*, Kim et. al., 2019
[4] *Convolutional Conditional Neural Processes, Gordon et. al., 2020*

A brief history of representing data points as functions

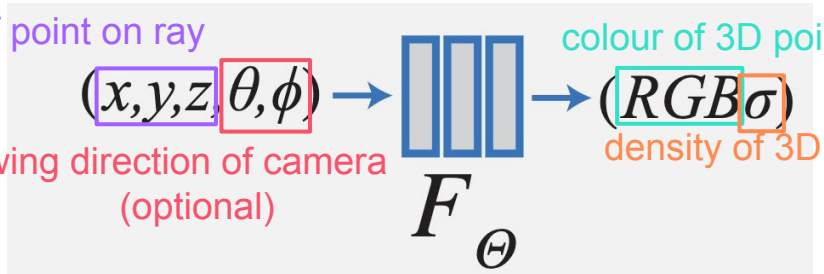
More recently, there were various (concurrent) works using such functional ("implicit") representations for 3D geometry [1,2,3].

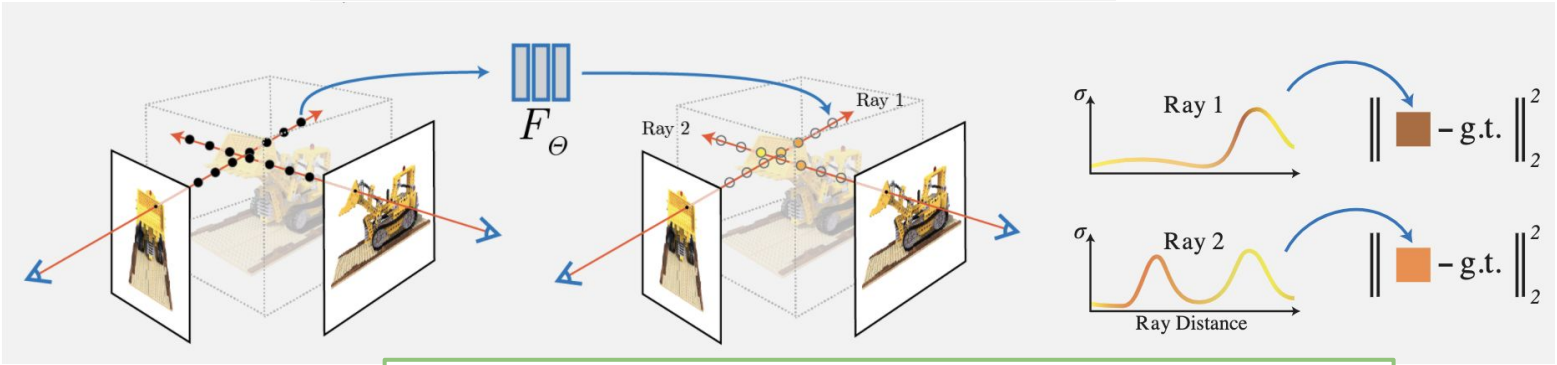


[1] *DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation*, Park et. al., 2019
[2] *Occupancy Networks: Learning 3D Reconstruction in Function Space*, Mescheder et. al., 2019
[3] *IM-Net: Learning Implicit Fields for Generative Shape Modeling*, Chen et. al., 2019

A brief history of representing data points as functions

Breakthrough: **Neural Radiance Fields** (NeRF) [1]





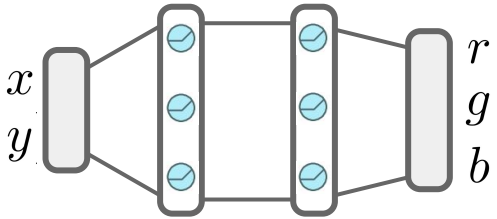
positional encodings

$$\gamma(\mathbf{x}) = \left[\sin(\mathbf{x}), \cos(\mathbf{x}), \ldots, \sin\left(2^{L-1}\mathbf{x}\right), \cos\left(2^{L-1}\mathbf{x}\right)\right]^{\mathrm{T}}.$$

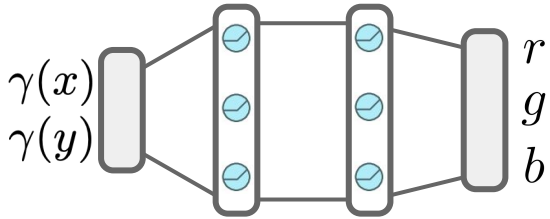
[1] Representing Scenes as *Neural Radiance Fields for View Synthesis*, Mildenhall et. al., 2020

A brief history of representing data points as functions

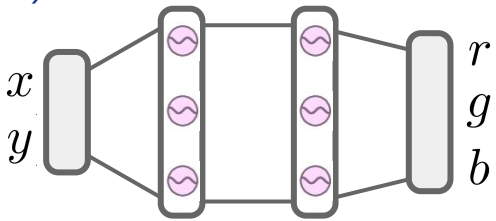
ReLU MLP

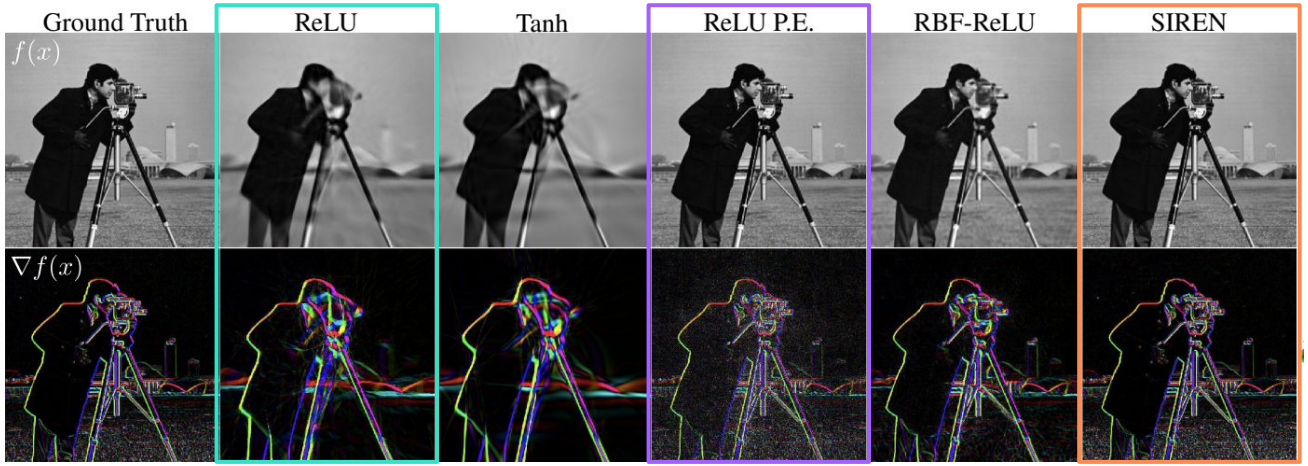


Sinusoidal Positional Encodings



Sinusoidal Representation Networks (**SIREN**) - MLP with sinusoidal nonlinearities

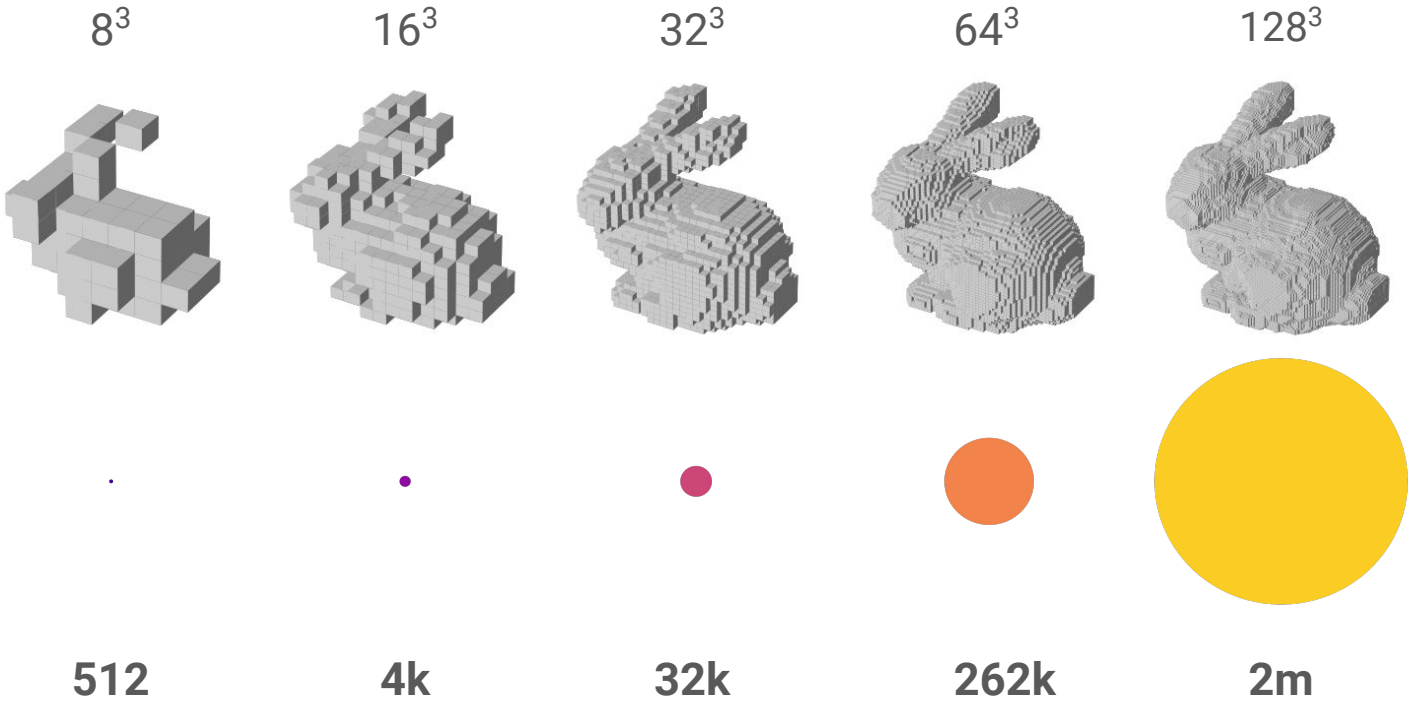




[1] Implicit Neural Representations with Periodic Activation Functions, Sitzmann et. al., 2020

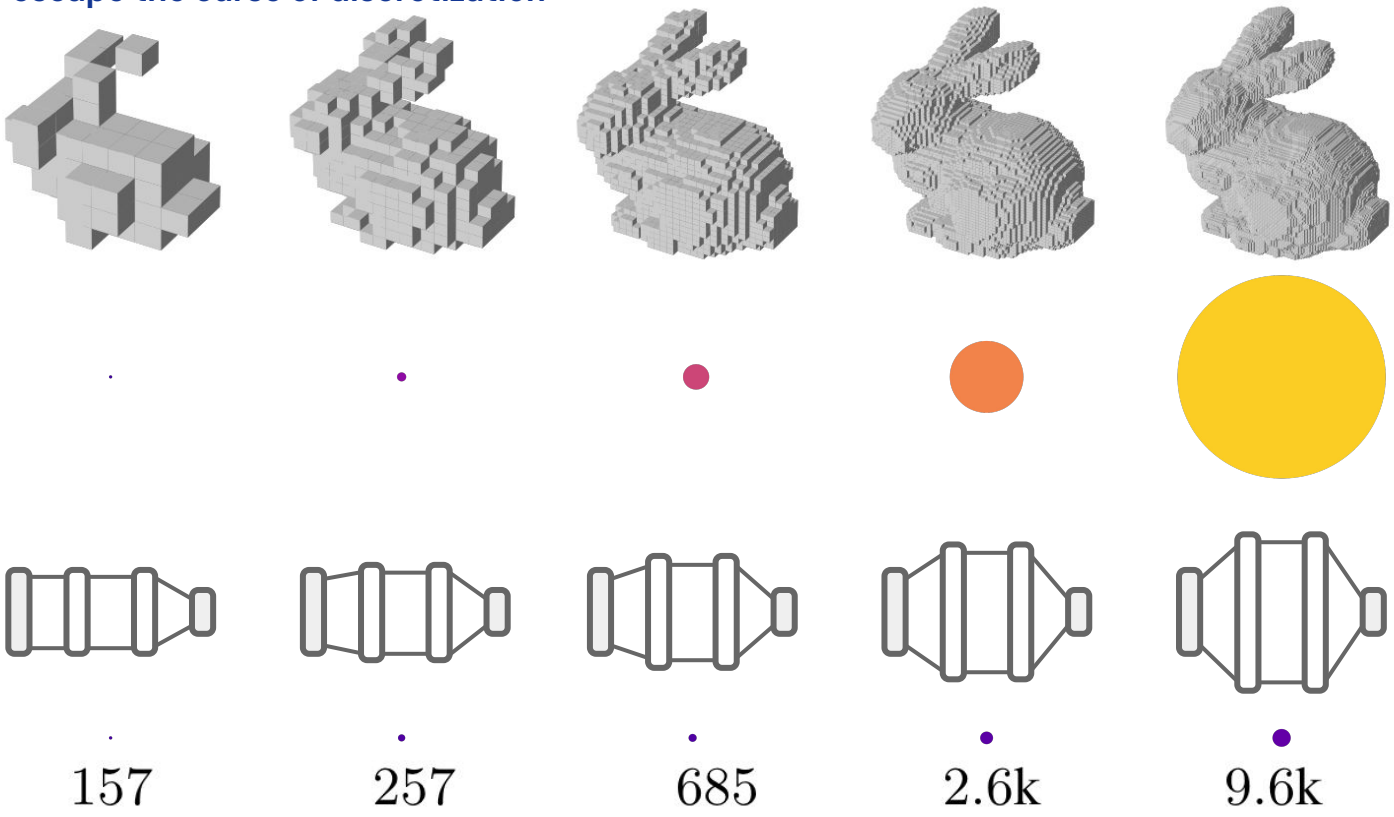
What's the point of representing data points as functions?

Scaling – escape the curse of discretization



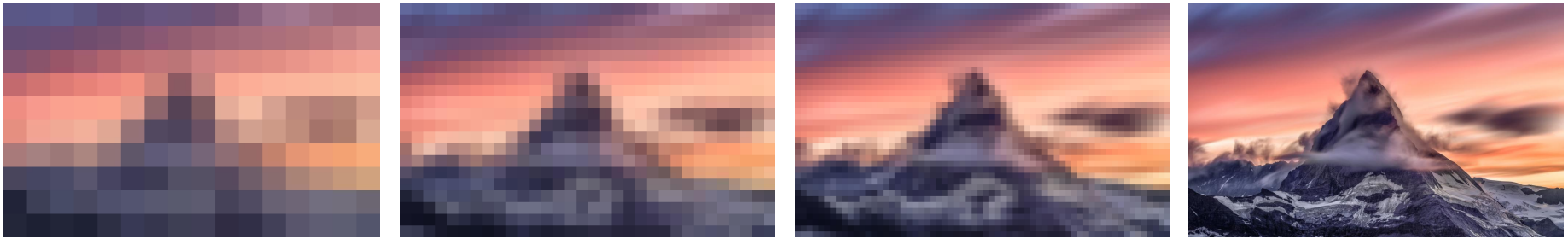
What's the point of representing data points as functions?

Scaling – escape the curse of discretization



What's the point of representing data points as functions?

Moving away from fixed resolution



These images have different resolutions, but all represent the **same underlying signal**.

Pixels are artifacts of discretisation - the signal that it represents is inherently continuous.

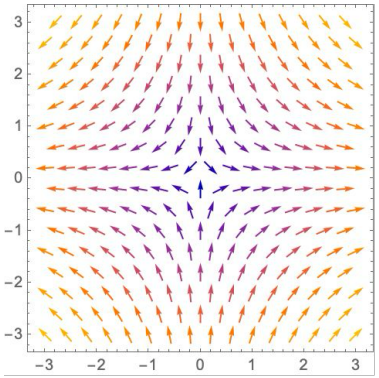
Functional representations allow us to model this signal instead of a fixed resolution.

What's the point of representing data points as functions?

Some data are **inherently continuous**, so difficult to discretise.

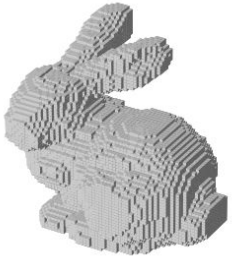
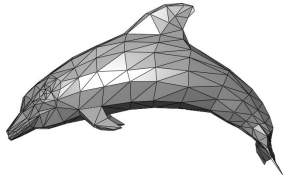
Scenes

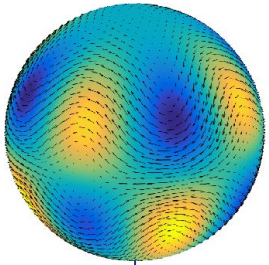
Fields



What's the point of representing data points as functions?

Unified framework/architecture for multiple data modalities



2D Conv | 3D Conv | Graph NN | Autoregressive/RNN | Spherical Conv

representation | representation | representation | representation | representation

Optimisation | Optimisation | Optimisation | Optimisation | Optimisation

functional representation | functional representation | functional representation | functional representation | functional representation

"Encoding is hard, optimisation is easy"

Deep Learning on Functa

Given these advantages of functional reps, we propose a new framework for doing DL:

1. **Fit functions** (MLP) to each data point in an efficient way (i.e. via optimization).

   We call them *functa* - a short term for such *MLPs that are to be thought of as data*

Deep Learning on Functa

Given these advantages of functional reps, we propose a new framework for doing DL:

1. **Fit functions** (MLP) to each data point in an efficient way (i.e. via optimization).

   We call them *functa* – a short term for such *MLPs that are to be thought of as data*

2. Express these functa using a **compact rep** (later introduced as *modulations*)

Deep Learning on Functa

Given these advantages of functional reps, we propose a new framework for doing DL:

1. **Fit functions** (MLP) to each data point in an efficient way (i.e. via optimization).

   We call them *functa* – a short term for such *MLPs that are to be thought of as data*

2. Express these functa using a **compact rep** (later introduced as *modulations*)
3. **Do Deep Learning on these compact reps** instead of the original array rep of data.

Deep Learning on Functa

Given these advantages of functional reps, we propose a new framework for doing DL:

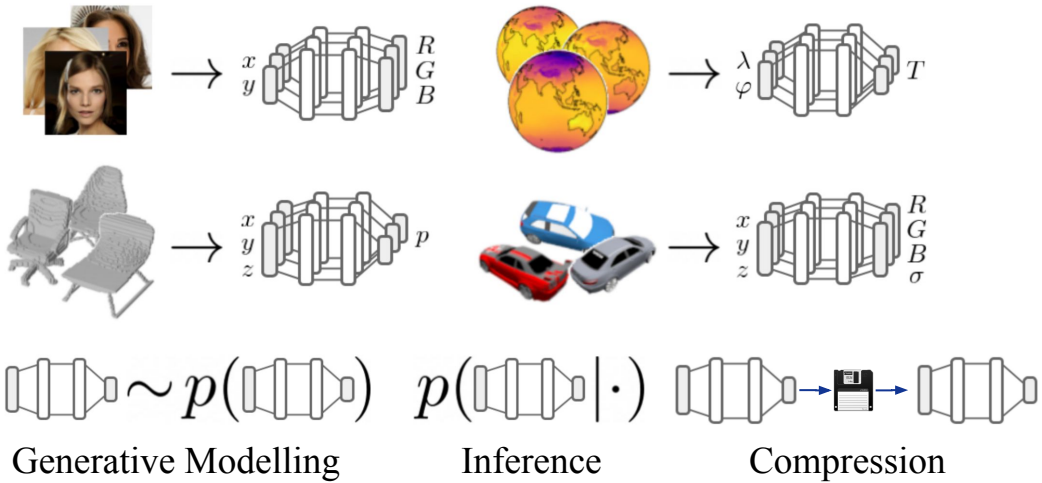
1. **Fit functions** (MLP) to each data point in an efficient way (i.e. via optimisation).

   We call them *functa* - a short term for such *MLPs that are to be thought of as data*

2. Express these functa using a **compact rep** (later introduced as *modulations*)
3. **Do Deep Learning on these compact reps** instead of the original array rep of data.

tl;dr: "Represent data as functa, do deep learning on functa"

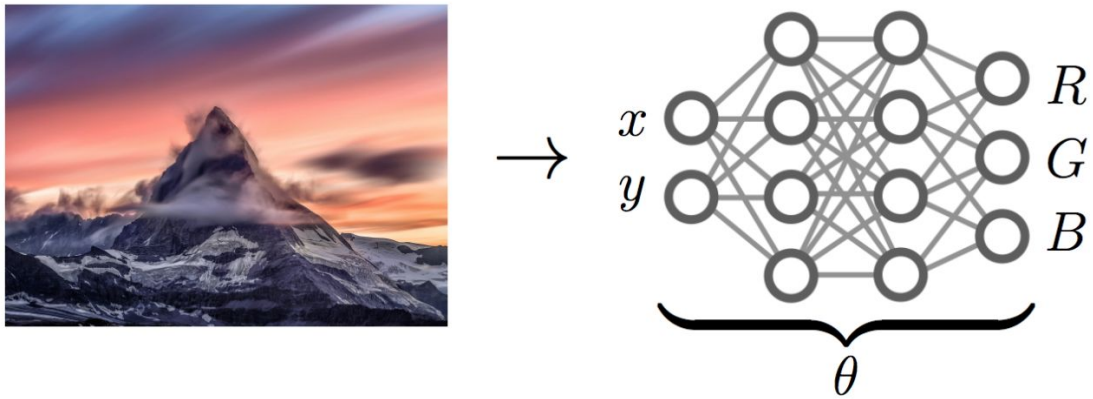
Deep Learning on Functa



What does this have to do with compression?

What does this have to do with compression?

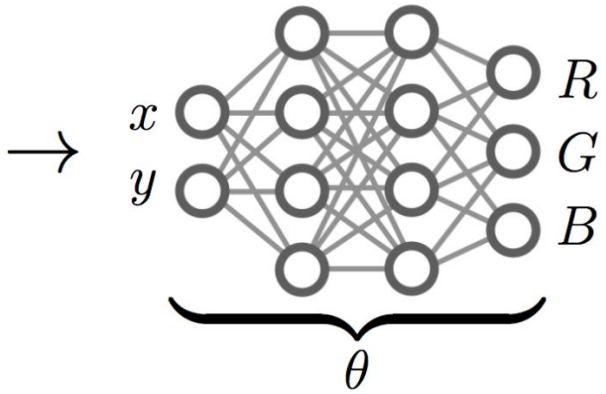
Basic idea introduced by **COIN: COmpression with Implicit Neural representations**



Dupont & Golinski et al. “COIN: COmpression with Implicit Neural representations", 2021

What does this have to do with compression?

Basic idea introduced by **COIN: COmpression with Implicit Neural representations**



$$I[x, y]$$

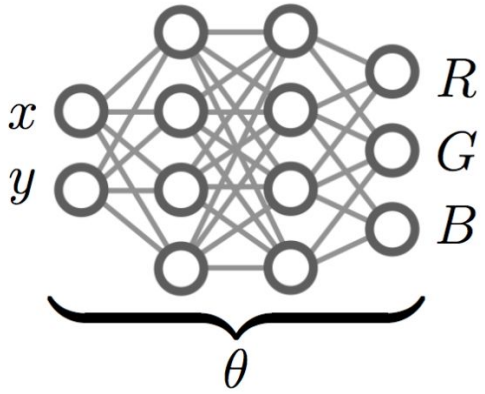
$$f_\theta(x, y)$$

Dupont & Golinski et al. “COIN: COmpression with Implicit Neural representations", 2021

What does this have to do with compression?

Basic idea introduced by **COIN: COmpression with Implicit Neural representations**

$\rightarrow$



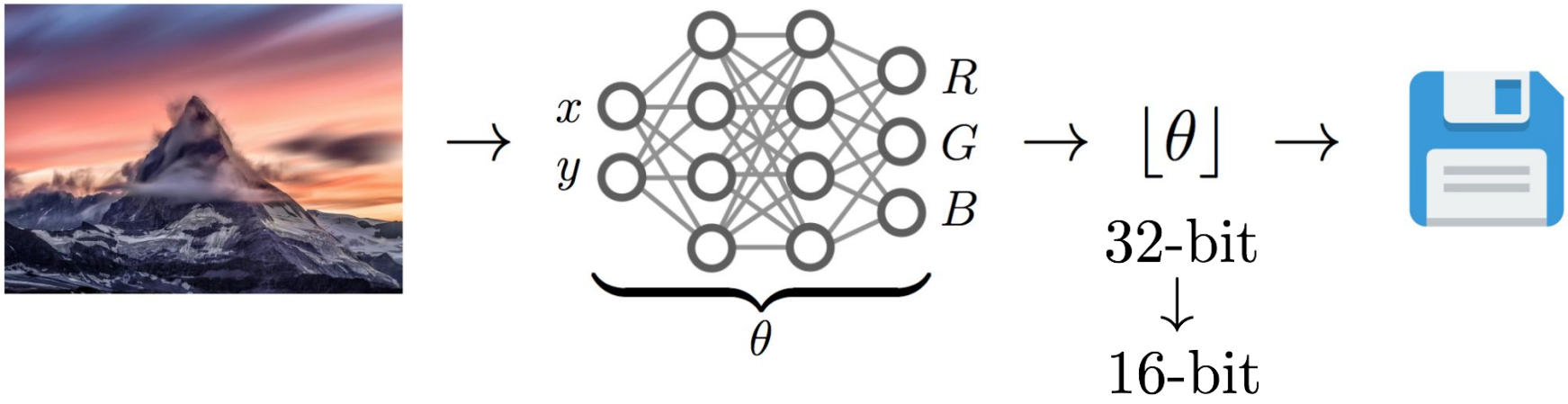
$$I[x, y] \qquad f_\theta(x, y)$$

$$\min_\theta \sum_{x,y} \| f_\theta(x, y) - I[x, y] \|_2^2$$

Dupont & Golinski et al. “COIN: COmpression with Implicit Neural representations", 2021

What does this have to do with compression?

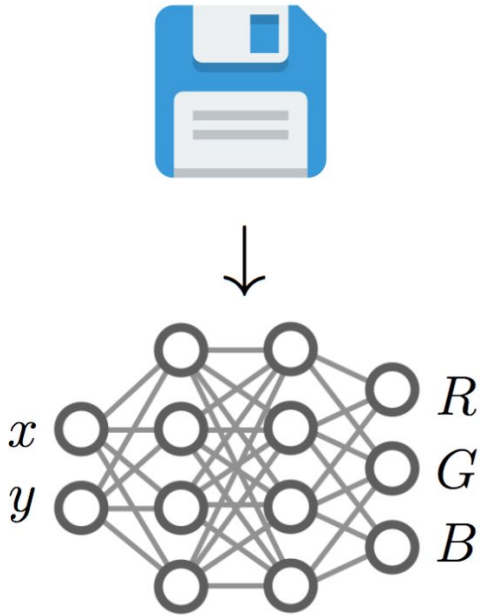
Basic idea introduced by **COIN: COmpression with Implicit Neural representations**



Dupont & Golinski et al. “COIN: COmpression with Implicit Neural representations", 2021

What does this have to do with compression?

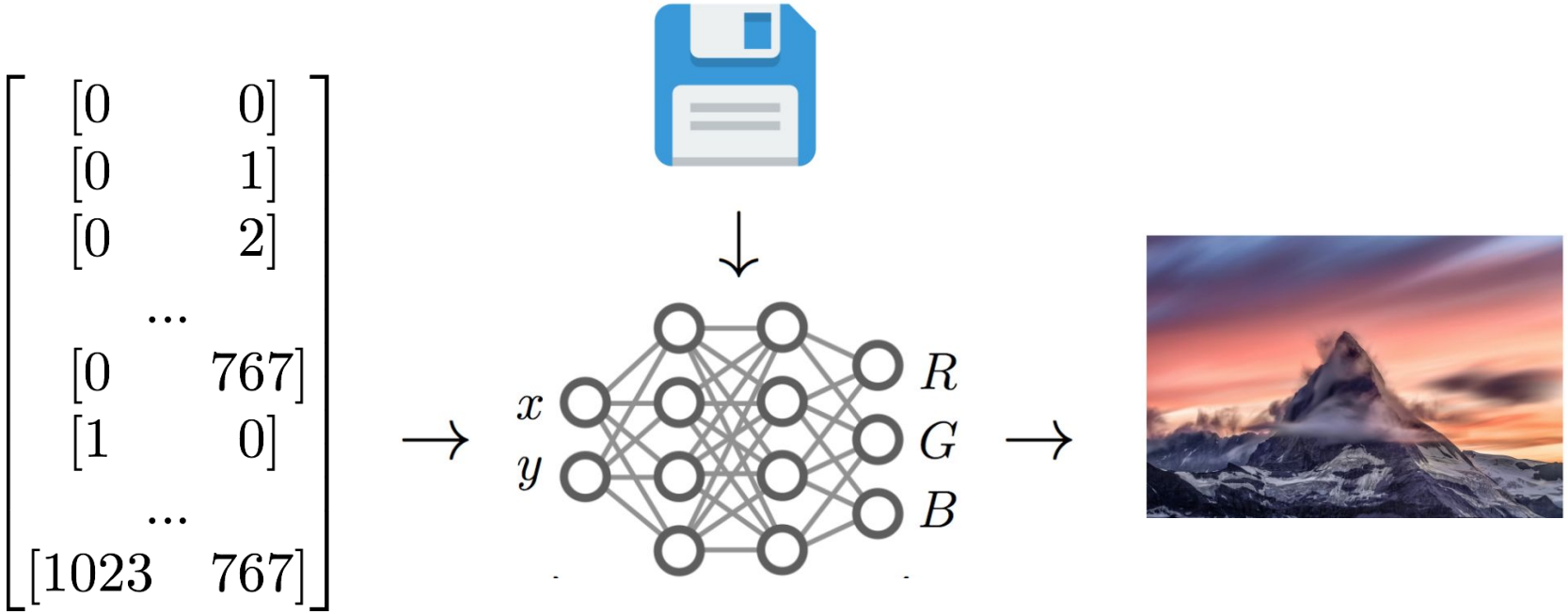
Basic idea introduced by **COIN: COmpression with Implicit Neural representations**



Dupont & Golinski et al. “COIN: COmpression with Implicit Neural representations", 2021

What does this have to do with compression?

Basic idea introduced by **COIN: COmpression with Implicit Neural representations**



Dupont & Golinski et al. “COIN: COmpression with Implicit Neural representations", 2021

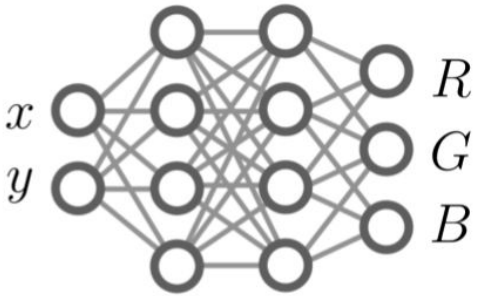
What does this have to do with compression?

JPEG

12.8kB

Overfit
— neural net →
to image

@25dB

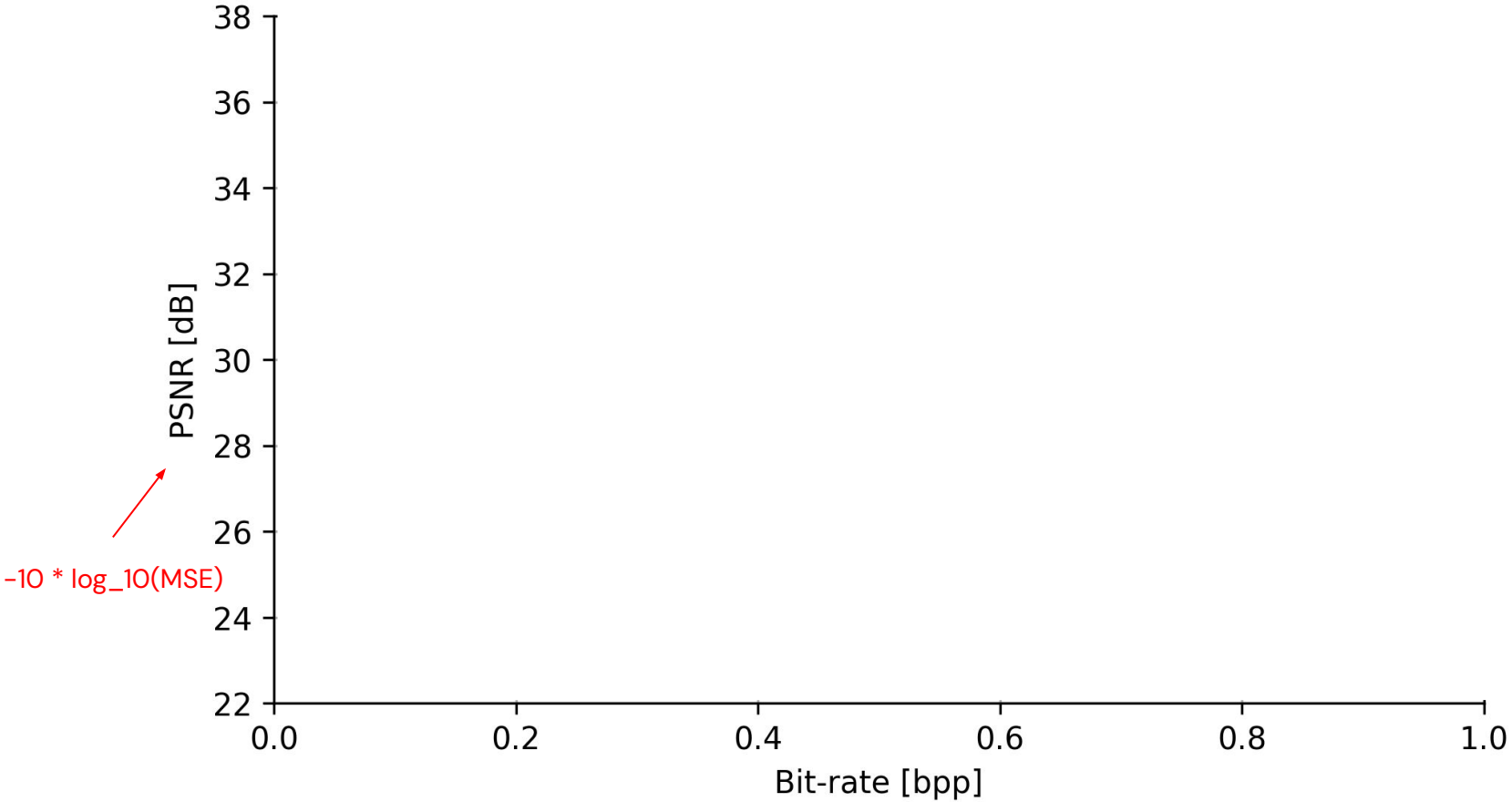


Quantize and store
weights

9.4kB

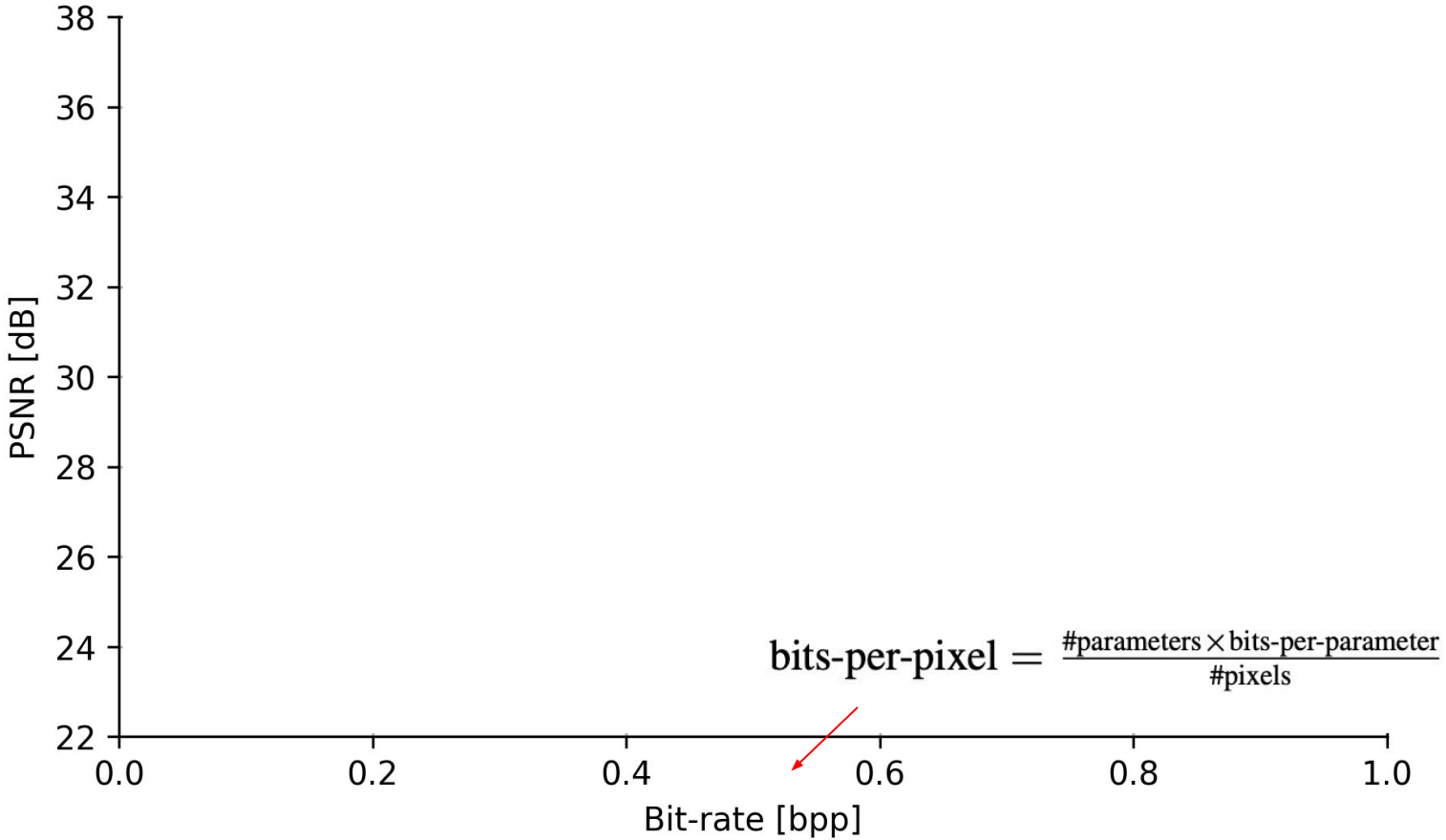
Dupont & Golinski et al. “COIN: COmpression with Implicit Neural representations", 2021

Compression results for COIN (KODAK)



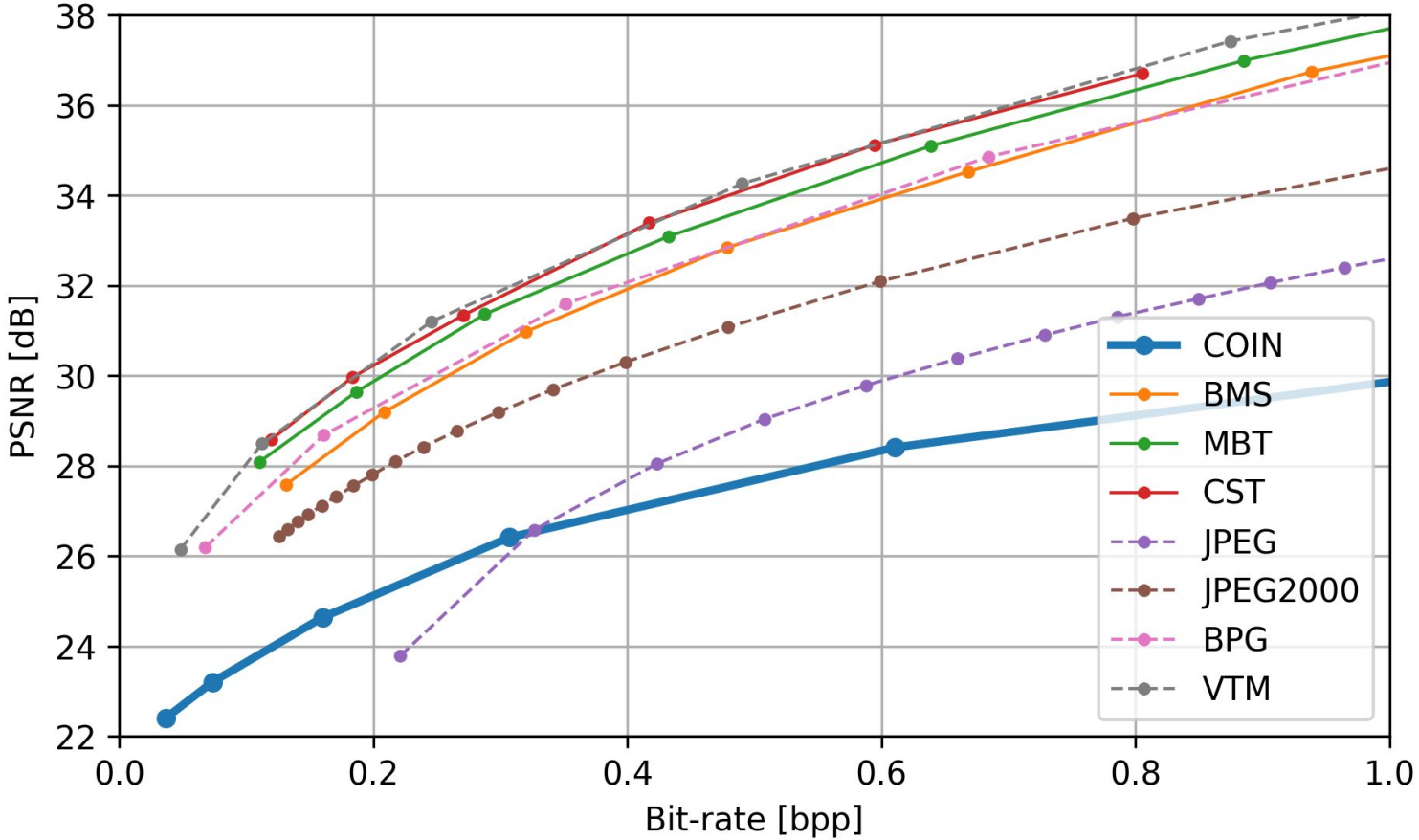
Dupont & Golinski et al. "COIN: COmpression with Implicit Neural representations", 2021

Compression results for COIN (KODAK)



Dupont & Golinski et al. “COIN: COmpression with Implicit Neural representations", 2021

Compression results for COIN (KODAK)



Dupont & Golinski et al. "COIN: COmpression with Implicit Neural representations", 2021

Data Compression with COIN

Some interesting properties

- **Data** compression -> **Model** compression
- Applicable to multiple data modalities
- New approach to neural data compression

Data Compression with COIN

Some interesting properties

- **Data** compression -> **Model** compression
- Applicable to multiple data modalities
- New approach to neural data compression

Problems

- Requires architecture search for best results
- Encoding Time
- Tabula-rasa Optimisation
- R/D results uncompetitive

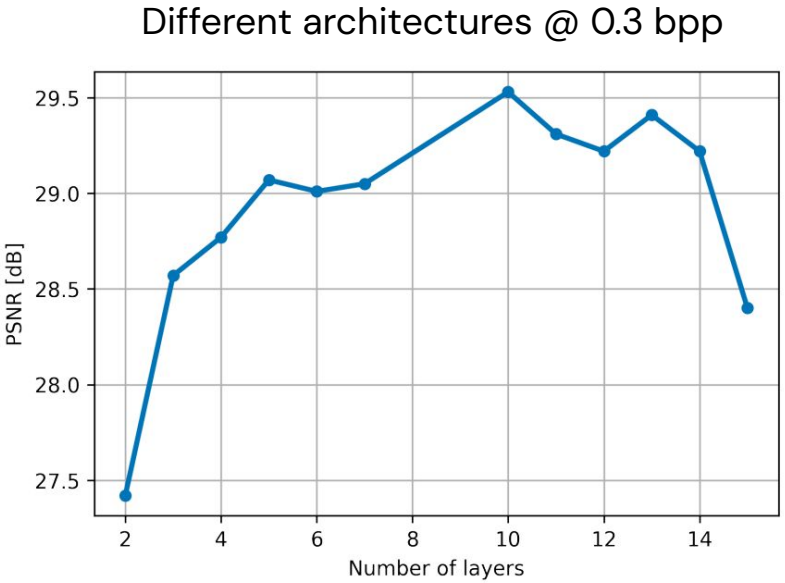
Data Compression with COIN

Some interesting properties

- **Data** compression -> **Model** compression
- Applicable to multiple data modalities
- New approach to neural data compression

Problems

- Requires architecture search for best results
- Encoding Time
- Tabula-rasa Optimisation
- R/D results uncompetitive



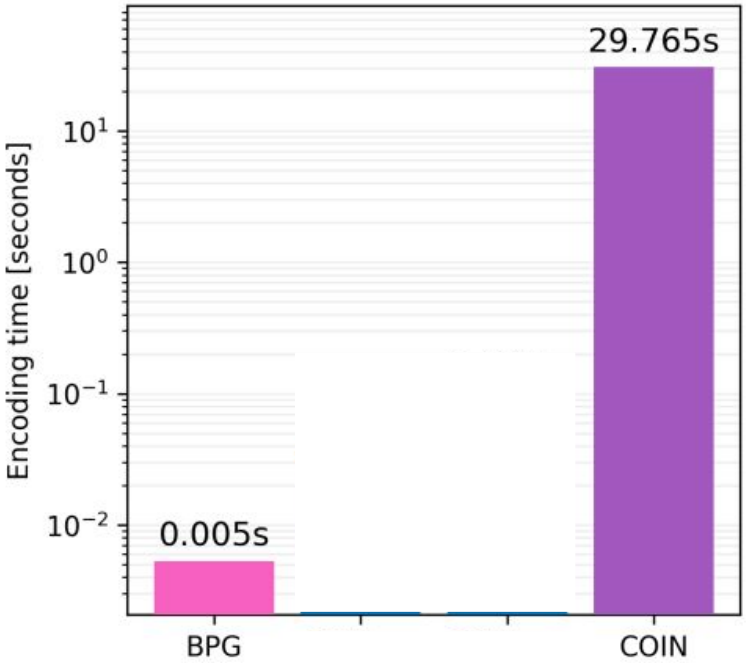
Data Compression with COIN

Some interesting properties

- **Data** compression -> **Model** compression
- Applicable to multiple data modalities
- New approach to neural data compression

Problems

- Requires architecture search for best results
- Encoding Time
- Tabula-rasa Optimisation
- R/D results uncompetitive



Outline

1 - Introduction: Data as Functions

- From data to functa: Your data point is a function and you can treat it like one [ICML 2022]
- COIN: COmpression with Implicit Neural representations [2021]

2 - Algorithms and Architectures for Neural Fields

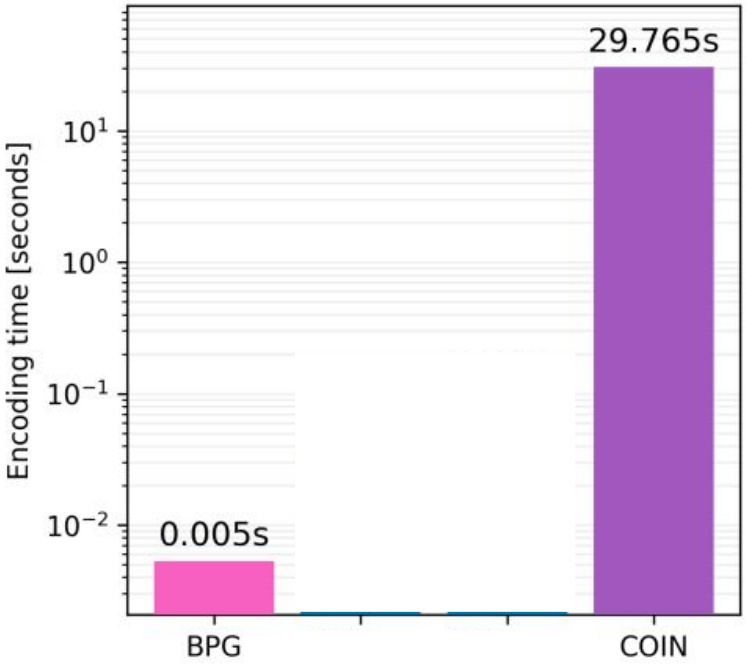
- Meta-Learning INRs
  - Efficient Meta-Learning via Error-based Context Pruning for Implicit Neural Representations [NeurIPS 2023]
- Better parameterizations & Advanced Compression
  - Meta-Learning Sparse Compression Networks [TMLR 2022]
  - Modality-Agnostic Variational Compression of Implicit Neural Representations [ICML 2023]
  - C3: High-performance and low-complexity neural compression from a single image or video [CVPR 2024]

Improving Encoding Time

How do we improve on slow encoding times?

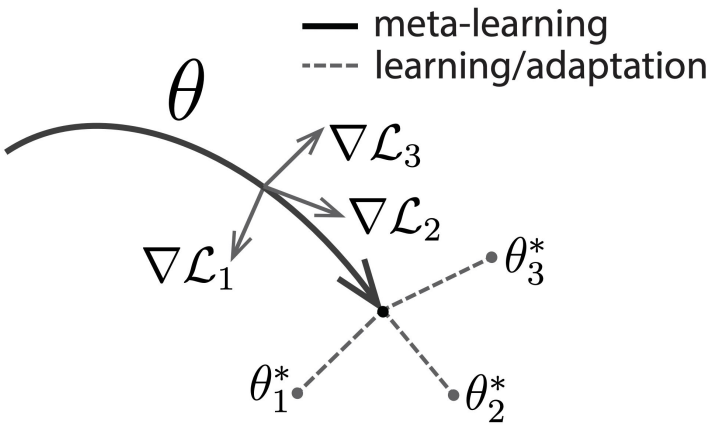
- Undesirable for compression
- Makes the creation of functasets impractical

→ *Meta-Learning*



Accelerating INRs through Meta-Learning

Runtime can be addressed through Meta-Learning:

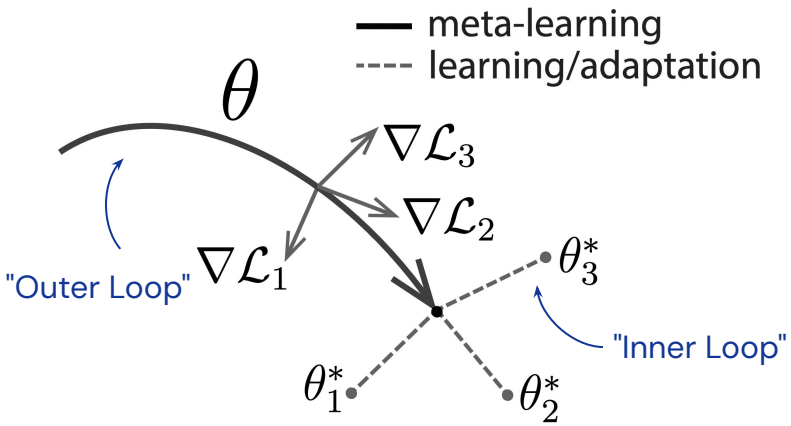


Finn et al. “Model Agnostic Meta-Learning”, ICML 2017

Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

Accelerating INRs through Meta-Learning

Runtime can be addressed through Meta-Learning:

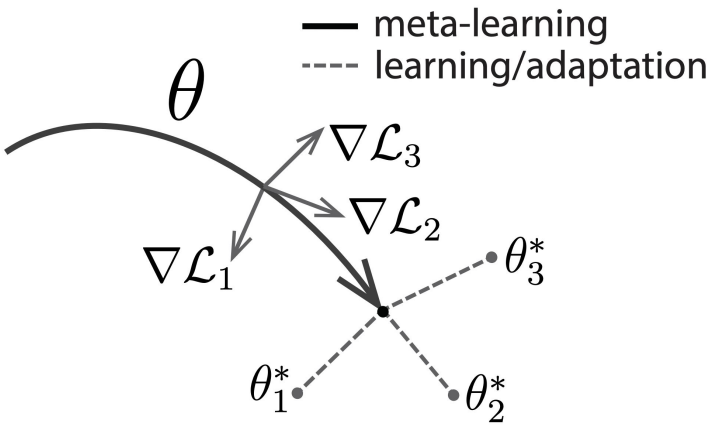


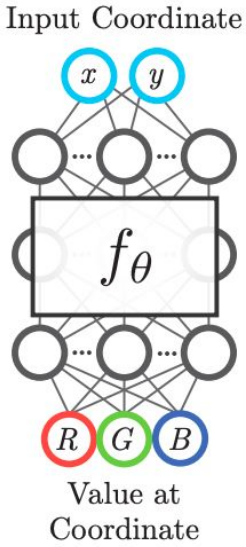
Finn et al. “Model Agnostic Meta-Learning”, ICML 2017

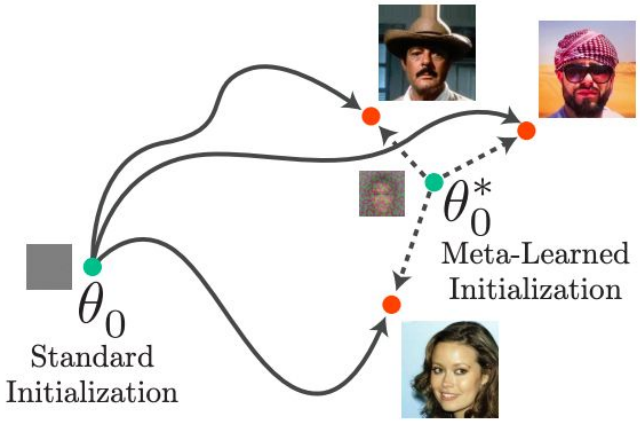
Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

Accelerating INRs through Meta-Learning

Runtime can be addressed through Meta-Learning:



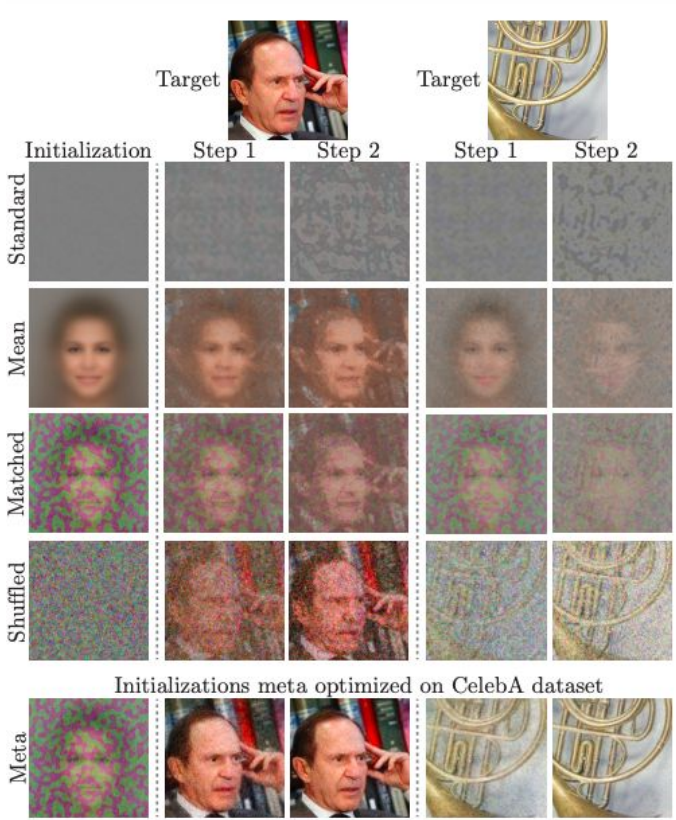




Finn et al. “Model Agnostic Meta-Learning”, ICML 2017

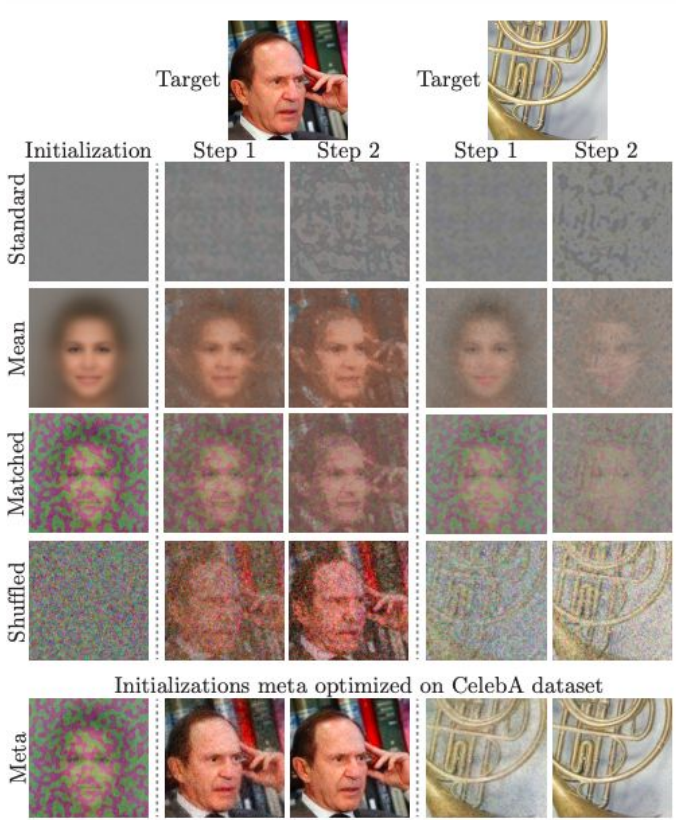
Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

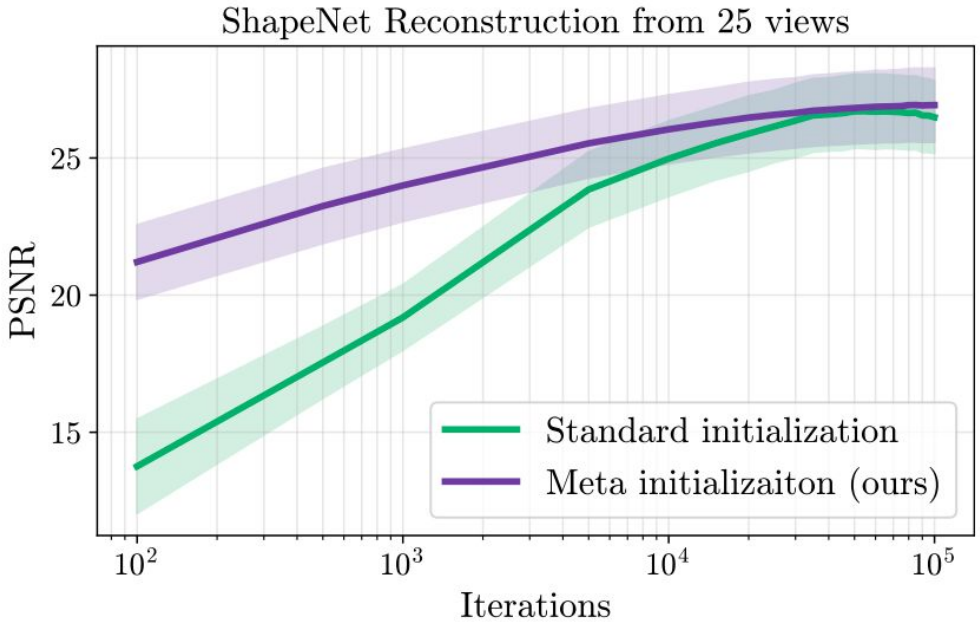
Accelerating INRs through Meta-Learning



Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

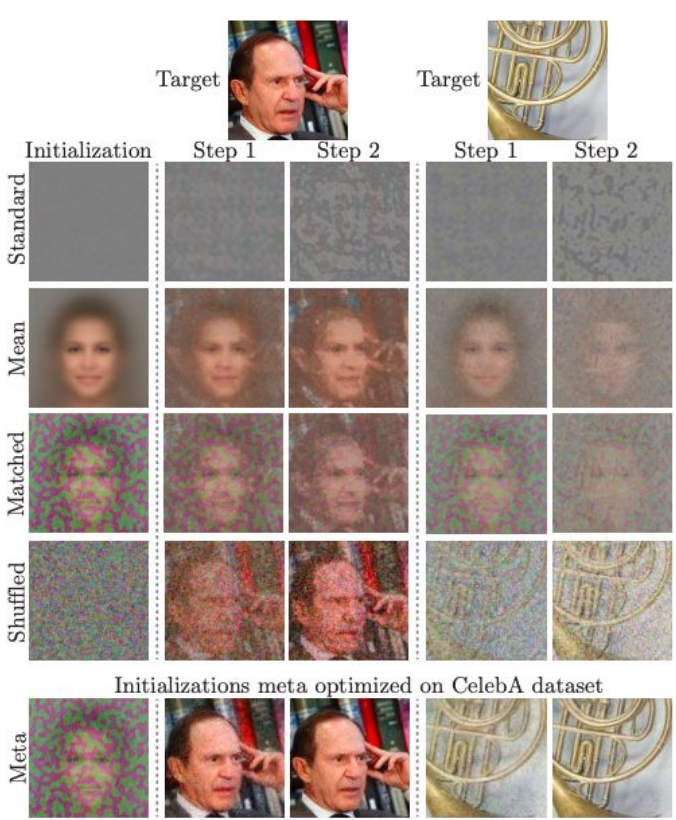
Accelerating INRs through Meta-Learning

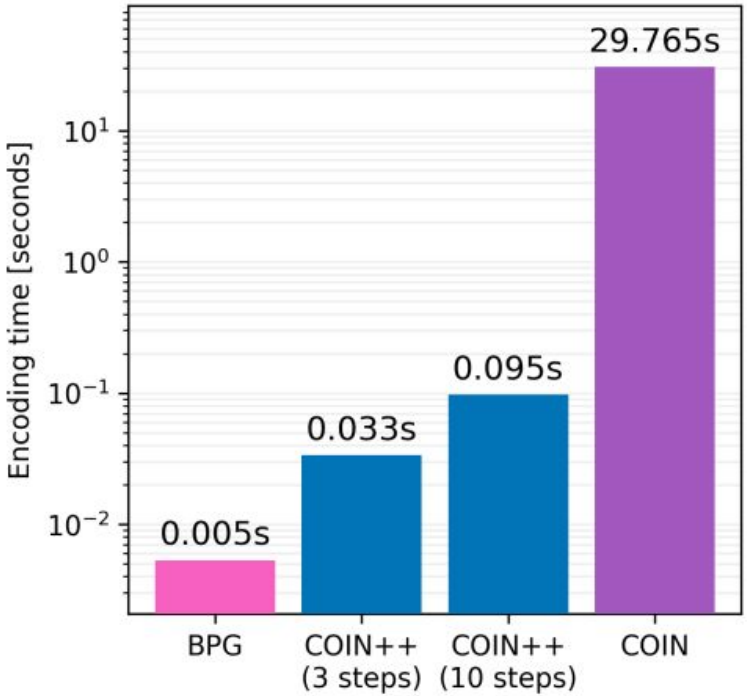




Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

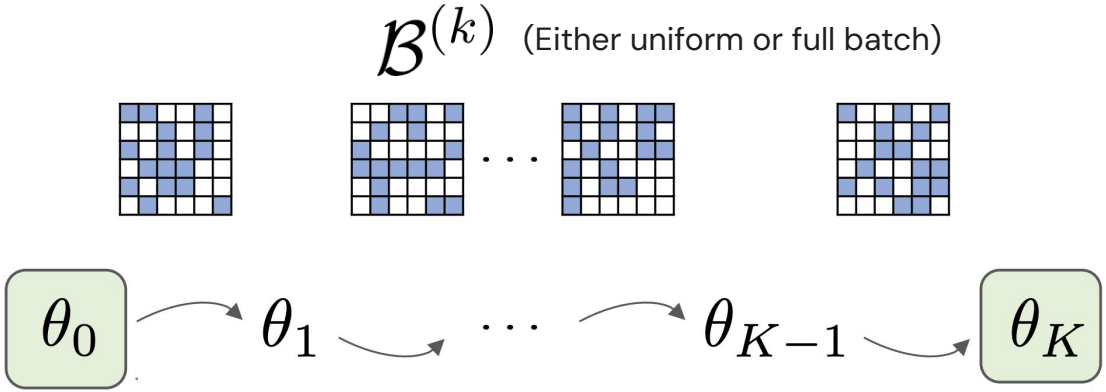
Accelerating INRs through Meta-Learning





Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

Accelerating INRs through Meta-Learning

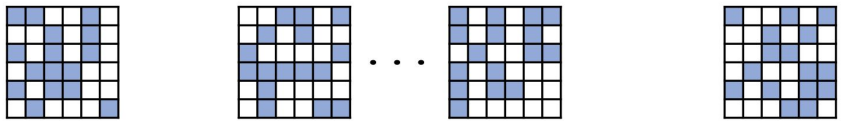


Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

Accelerating INRs through Meta-Learning

Limited to 5–10 steps in practice

$\mathcal{B}^{(k)}$ (Either uniform or full batch)

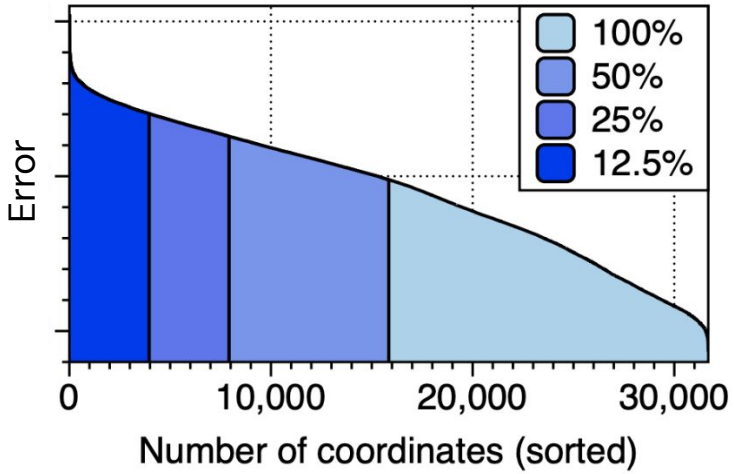


$\theta_0 \rightarrow \theta_1 \rightarrow \cdots \rightarrow \theta_{K-1} \rightarrow \theta_K$

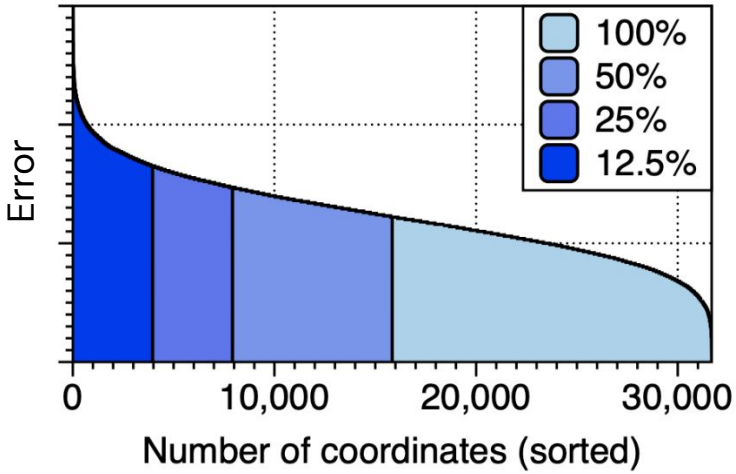
Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

Accelerating INRs through Meta-Learning

Runtime can be addressed through Meta-Learning:



(a) $k = 0$

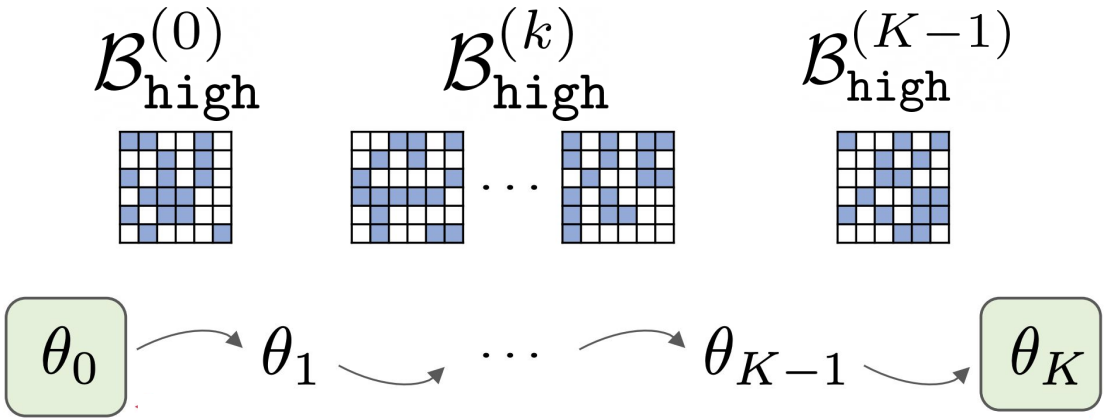


(b) $k = 15$

Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

Accelerating INRs through Meta-Learning

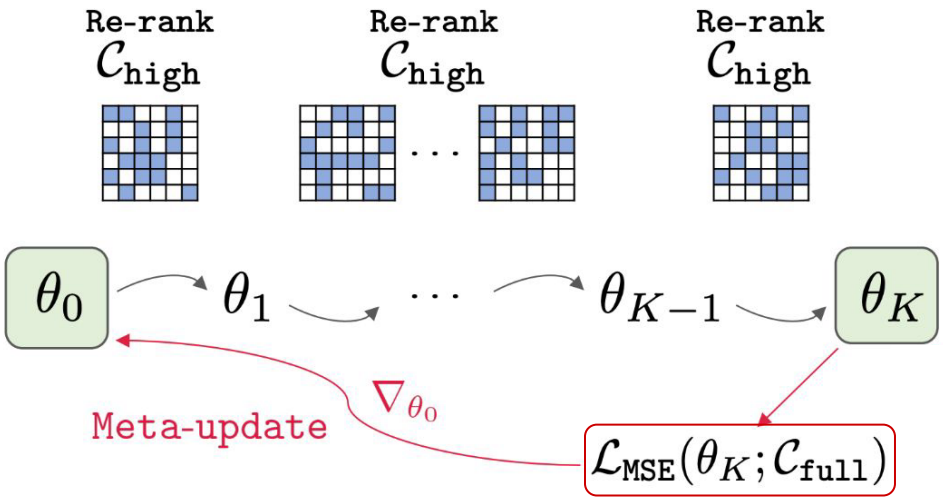
(Connections to Curriculum Learning)



$$\mathcal{B}_{\texttt{high}}^{(k)} = \mathrm{TopK}(\mathcal{D}, \mathcal{R}_k, \gamma|\mathcal{D}|)$$

Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

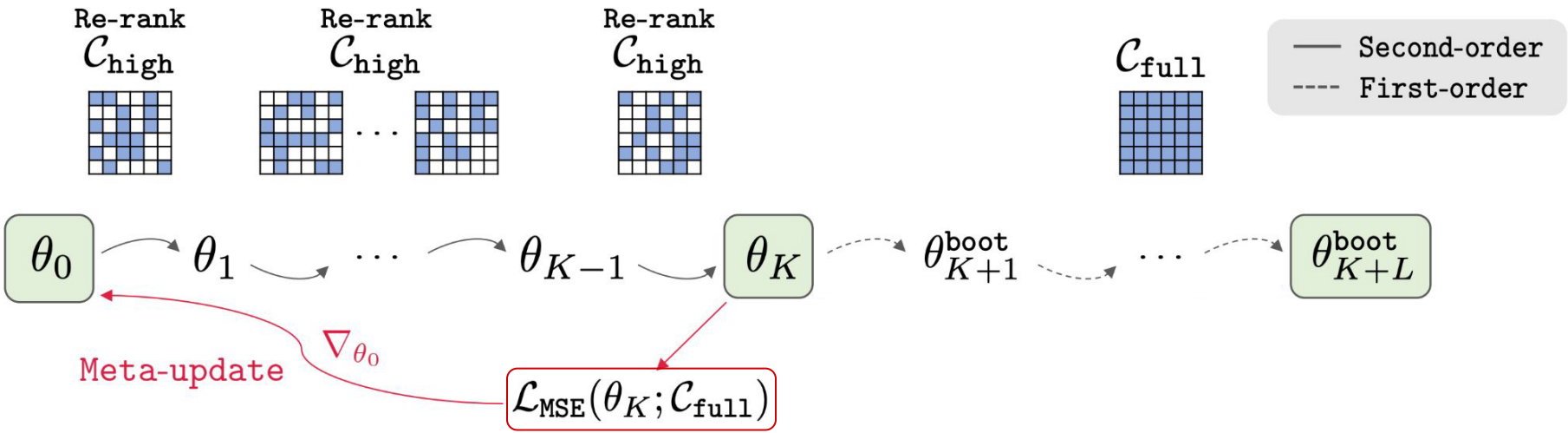
Meta-Learning with Bootstrap correction



Second-order
First-order

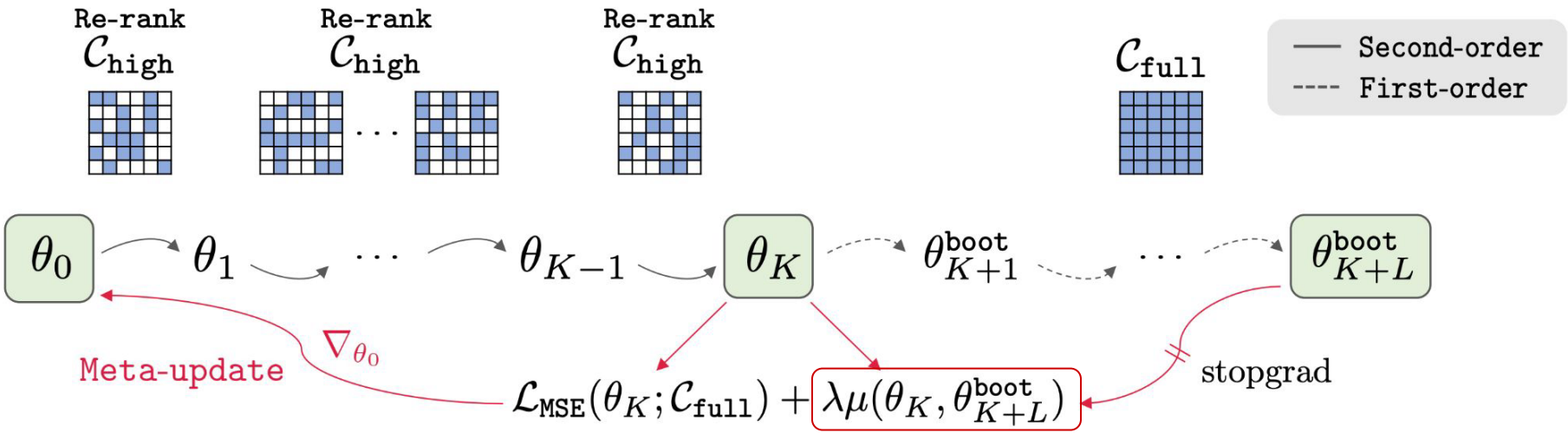
Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

Meta-Learning with Bootstrap correction



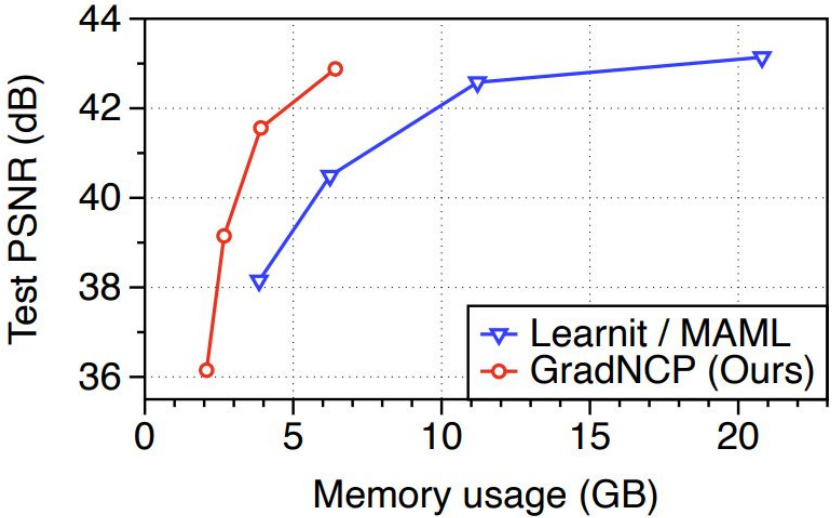
Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS 2023

Meta-Learning with Bootstrap correction



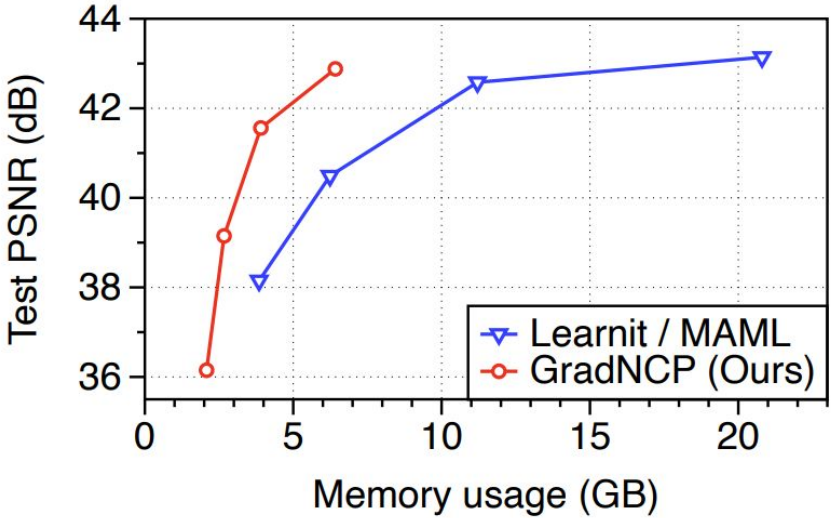
Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS 2023

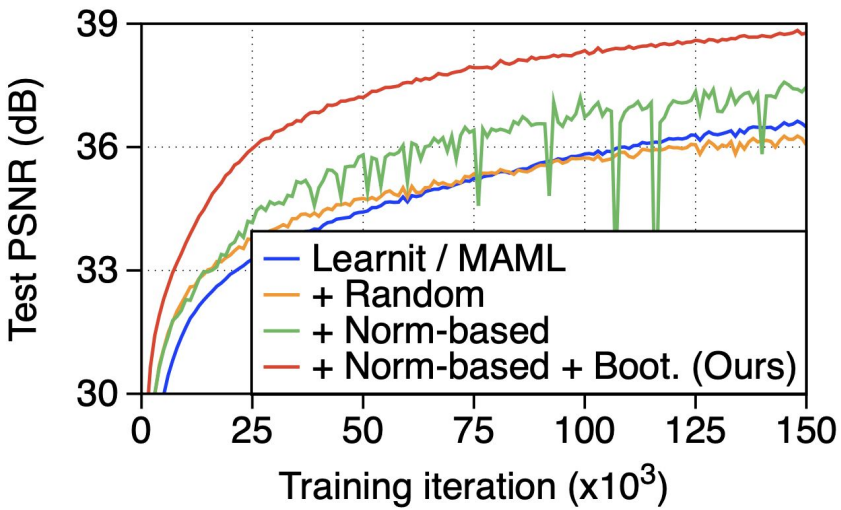
Accelerating INRs through Meta-Learning



Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

Accelerating INRs through Meta-Learning





Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS 2023

$\mathcal{R}_k^{\texttt{GradNCP}}(\cdot)$ implements a curriculum

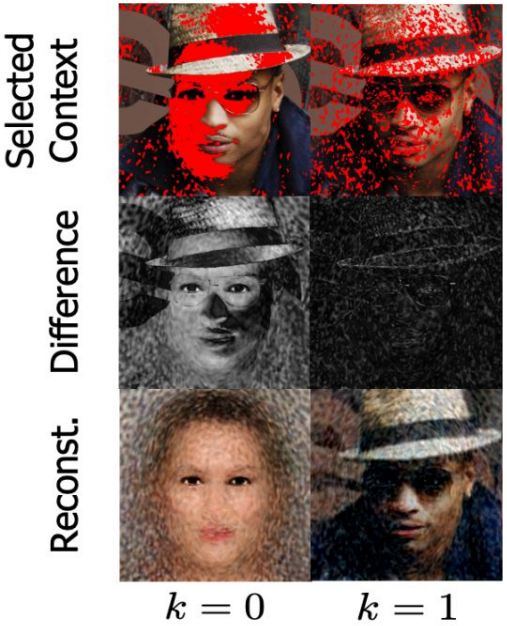
$k = 0$

Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

$\mathcal{R}_k^{\texttt{GradNCP}}(\cdot)$ implements a curriculum

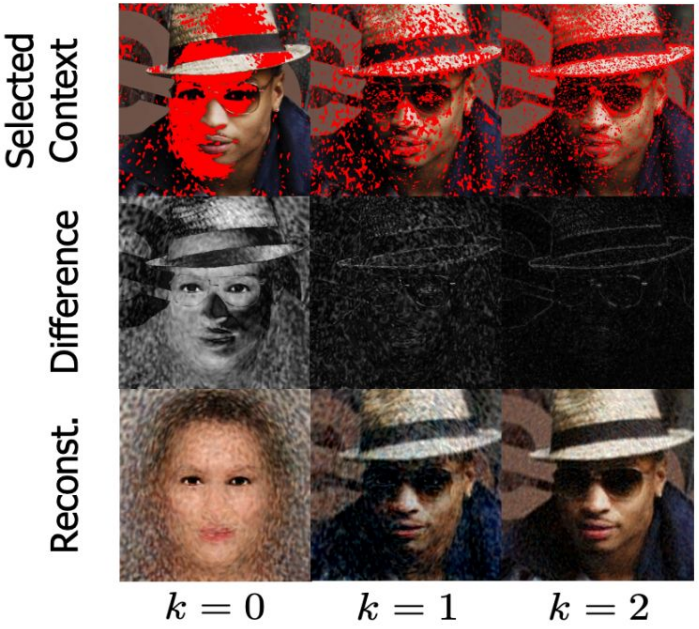
Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

$\mathcal{R}_k^{\texttt{GradNCP}}(\cdot)$ implements a curriculum



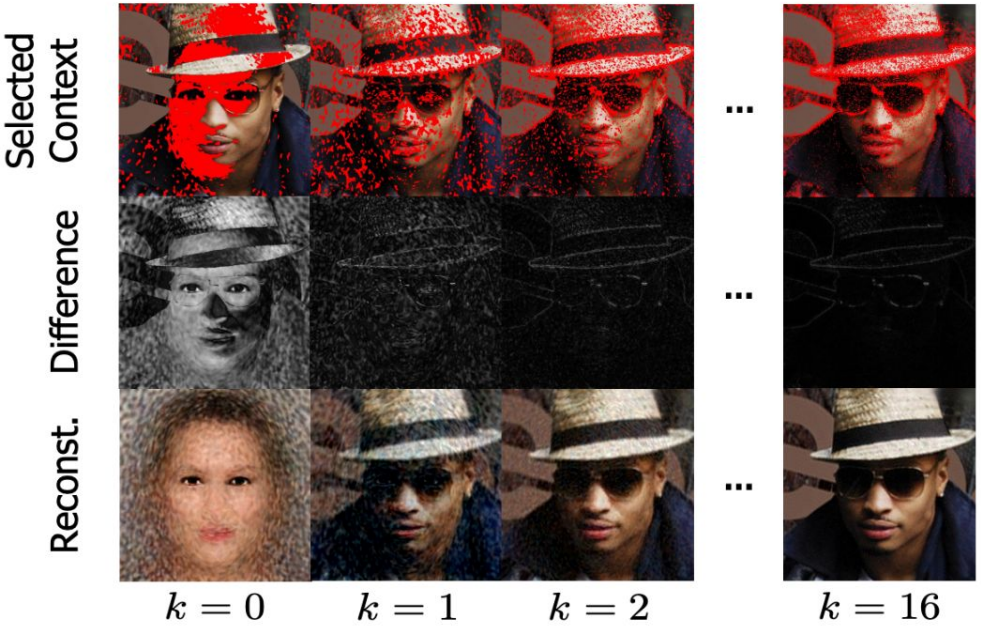
Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

$\mathcal{R}_k^{\texttt{GradNCP}}(\cdot)$ implements a curriculum



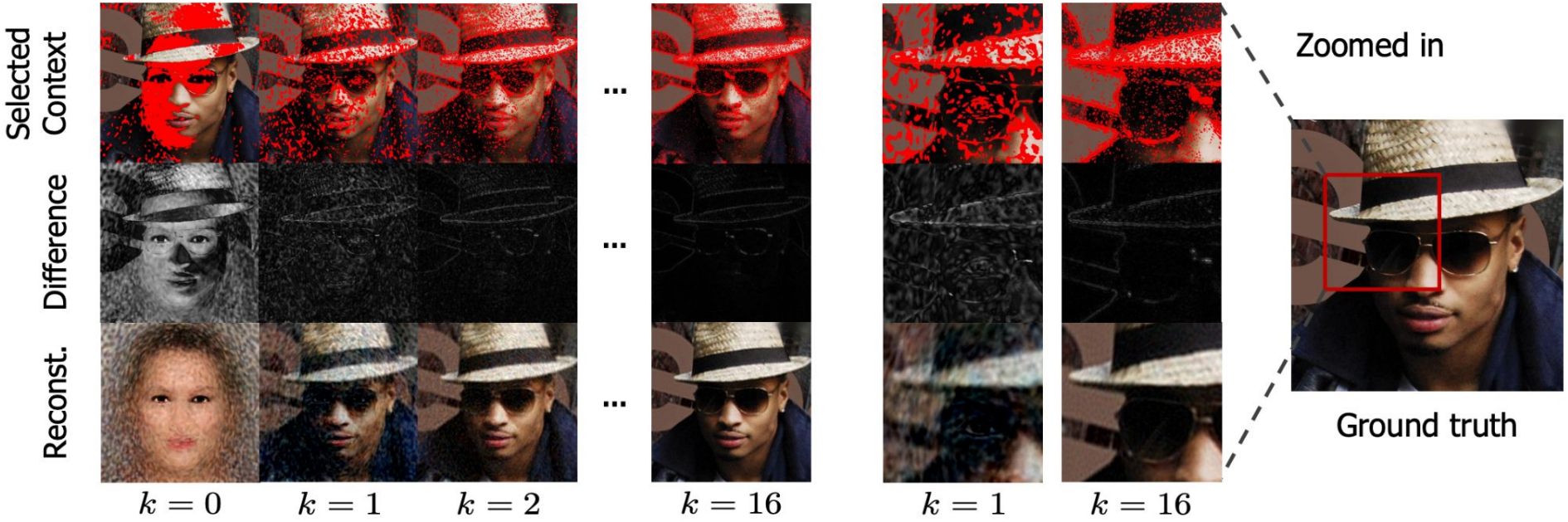
Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

$\mathcal{R}_k^{\texttt{GradNCP}}(\cdot)$ implements a curriculum



Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS 2023

$\mathcal{R}_k^{\texttt{GradNCP}}(\cdot)$ implements a curriculum



Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS 2023

Finding a suitable ranking metric

Target Prediction Gain

$$\mathcal{R}_k^{\texttt{tpg}}(\mathbf{x}, \mathbf{y}) := \ell(\theta_k, \mathcal{D} \setminus (\mathbf{x}, \mathbf{y})) - \ell(\theta'_k, \mathcal{D} \setminus (\mathbf{x}, \mathbf{y}))$$

Leave-one-out cross validation

Network updated using (**x, y**)

[Graves at al., "Automated Curriculum Learning", 2017]

Finding a suitable ranking metric

Target Prediction Gain

$$\mathcal{R}_k^{\texttt{tpg}}(\mathbf{x}, \mathbf{y}) := \ell(\theta_k, \mathcal{D} \setminus (\mathbf{x}, \mathbf{y})) - \ell(\theta'_k, \mathcal{D} \setminus (\mathbf{x}, \mathbf{y}))$$

Leave-one-out cross validation

Network updated using (**x, y**)

Self Prediction Gain

$$\mathcal{R}_k^{\texttt{spg}}(\mathbf{x}, \mathbf{y}) := \ell(\theta_k, \mathbf{x}, \mathbf{y}) - \ell(\theta'_k, \mathbf{x}, \mathbf{y})$$

[Graves at al., "Automated Curriculum Learning", 2017]

Finding a suitable ranking metric

Self prediction Gain

$$\ell\left(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\right) - \ell\left(\theta'_k; \{(\mathbf{x}, \mathbf{y})\}\right)$$

[Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS Sub. 2023]

Finding a suitable ranking metric

$$g_k = [\mathbf{0}, \nabla_{W_k}\ell, \nabla_{b_k}\ell]$$

Self prediction Gain

$$\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta'_k; \{(\mathbf{x}, \mathbf{y})\}\big)$$

[Tack et al. “Learning Large-scale Neural Fields via Context Pruned Meta-Learning”, NeurIPS Sub. 2023]

Finding a suitable ranking metric

$$g_k = [\mathbf{0}, \nabla_{W_k}\ell, \nabla_{b_k}\ell]$$

Self prediction Gain

$$\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta'_k; \{(\mathbf{x}, \mathbf{y})\}\big)$$
$$\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta_k - \alpha g_k; \{(\mathbf{x}, \mathbf{y})\}\big)$$

(Last layer update only)

[Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS Sub. 2023]

Finding a suitable ranking metric

$$g_k = [\mathbf{0}, \nabla_{W_k}\ell, \nabla_{b_k}\ell]$$

Self prediction Gain

$$\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta'_k; \{(\mathbf{x}, \mathbf{y})\}\big)$$

$$\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta_k - \alpha g_k; \{(\mathbf{x}, \mathbf{y})\}\big) \quad \text{(Last layer update only)}$$

$$\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \Big(\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \alpha g_k^\top \nabla_{\theta_k}\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big)\Big) \quad \text{(Taylor approximation)}$$

[Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS Sub. 2023]

Finding a suitable ranking metric

$$g_k = [\mathbf{0}, \nabla_{W_k}\ell, \nabla_{b_k}\ell]$$

Self prediction Gain

$$\begin{aligned}
&\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta'_k; \{(\mathbf{x}, \mathbf{y})\}\big) \\
&\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta_k - \alpha g_k; \{(\mathbf{x}, \mathbf{y})\}\big) && \text{(Last layer update only)} \\
&\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \Big(\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \alpha g_k^\top \nabla_{\theta_k} \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big)\Big) && \text{(Taylor approximation)} \\
&= \alpha g_k^\top \nabla_{\theta_k} \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big)
\end{aligned}$$

[Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS Sub. 2023]

Finding a suitable ranking metric

$$g_k = [\mathbf{0}, \nabla_{W_k}\ell, \nabla_{b_k}\ell]$$

Self prediction Gain

$$\begin{aligned}
&\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta'_k; \{(\mathbf{x}, \mathbf{y})\}\big) \\
&\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta_k - \alpha g_k; \{(\mathbf{x}, \mathbf{y})\}\big) && \text{(Last layer update only)} \\
&\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \Big(\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \alpha g_k^\top \nabla_{\theta_k} \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big)\Big) && \text{(Taylor approximation)} \\
&= \alpha g_k^\top \nabla_{\theta_k} \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) = \alpha \|g_k\|_2^2.
\end{aligned}$$

[Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS Sub. 2023]

Finding a suitable ranking metric

$$g_k = [\mathbf{0}, \nabla_{W_k}\ell, \nabla_{b_k}\ell]$$

Self prediction Gain

$$\begin{aligned}
&\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta'_k; \{(\mathbf{x}, \mathbf{y})\}\big) \\
&\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \ell\big(\theta_k - \alpha g_k; \{(\mathbf{x}, \mathbf{y})\}\big) && \text{(Last layer update only)} \\
&\approx \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \Big(\ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) - \alpha g_k^\top \nabla_{\theta_k} \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big)\Big) && \text{(Taylor approximation)} \\
&= \alpha g_k^\top \nabla_{\theta_k} \ell\big(\theta_k; \{(\mathbf{x}, \mathbf{y})\}\big) = \alpha \|g_k\|_2^2.
\end{aligned}$$

$$\mathcal{R}_k^{\texttt{GradNCP}}(\mathbf{x}, \mathbf{y}) := ||(\mathbf{y} - f_{\theta_k}(\mathbf{x}))[\phi_{\theta_k^{\text{base}}}(\mathbf{x}), \mathbf{1}]^\top||$$

Cheap & Easy

[Tack et al. "Learning Large-scale Neural Fields via Context Pruned Meta-Learning", NeurIPS Sub. 2023]

Transfer between task distributions

Init. \ Task	CelebA	Imagenette	Text	SDF
CelebA	**30.37**	26.44	21.53	36.45
Imagenette	28.51	**27.07**	22.63	34.80
Text	14.65	15.83	**27.85**	23.14
SDF	19.80	20.05	17.23	**51.73**

Tancik & Mildenhall et al. “Learned Initializations for Optimizing Coordinate-Based Neural Representations”, CVPR 2021

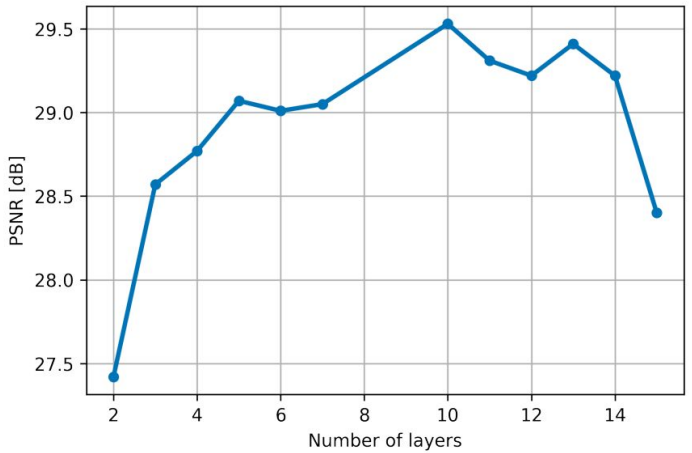
Architectural Improvements

Desirable for several reasons

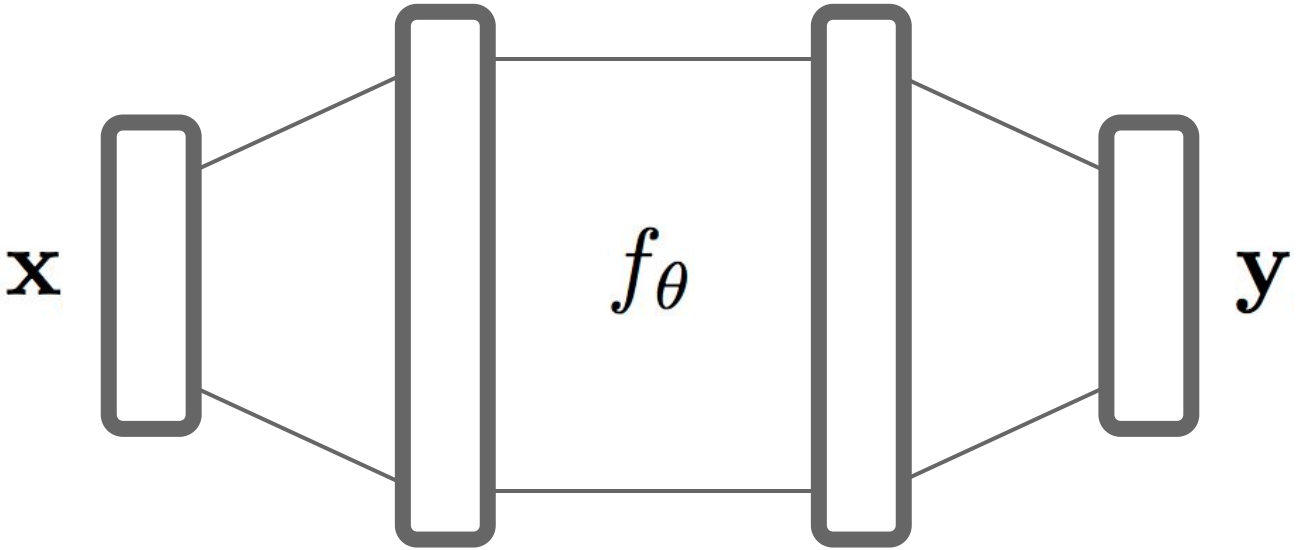
- Avoid costly architecture search
- Drastically reduce parameters to be encoded per signal (compression, functa)
- Meta-Learning most effective when combined with inductive bias
- (Specialized architectures can also reduce encoding/decoding time)

→ *Parameter-Efficient Fine-Tuning*

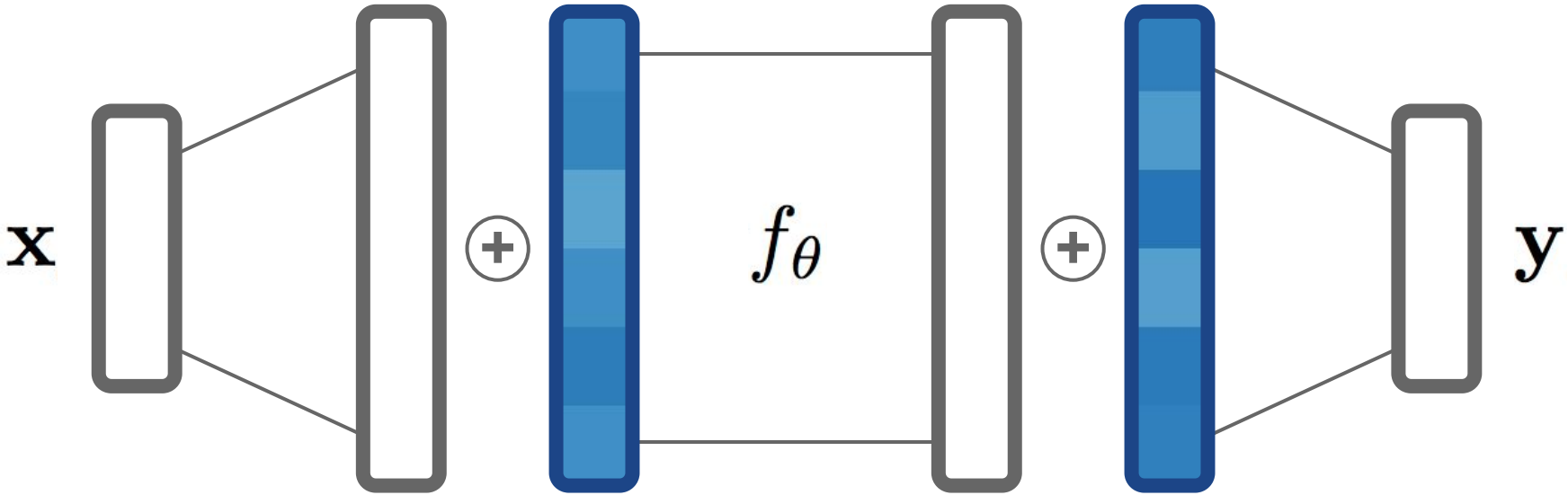
Different architectures @ 0.3 bpp



Compact representations

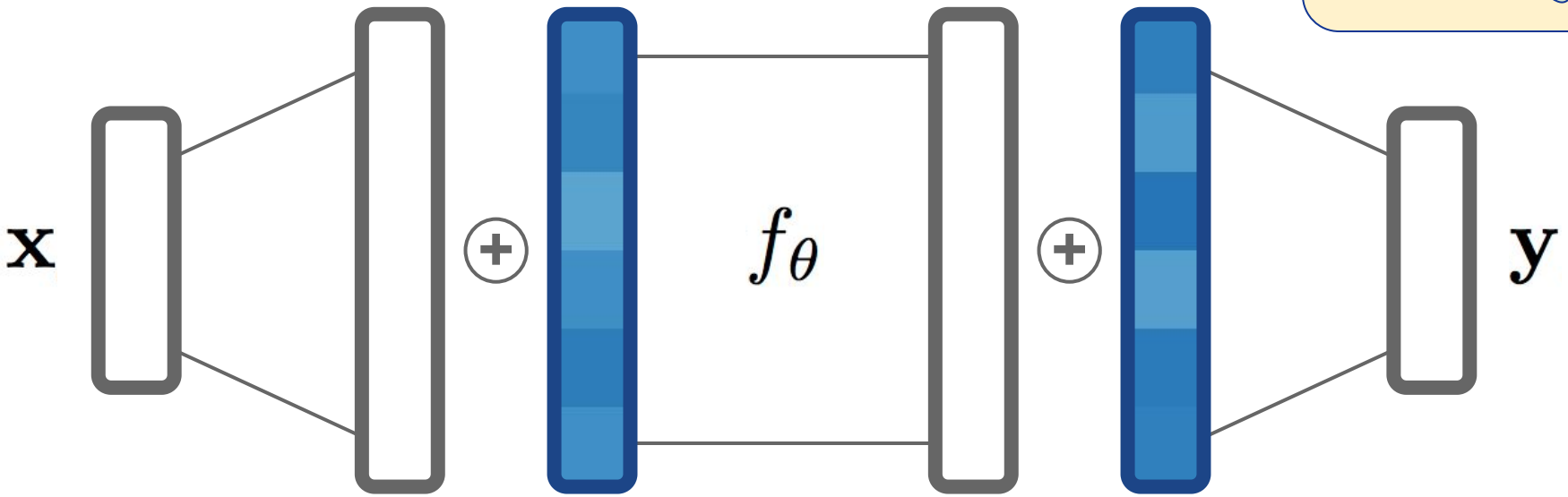


Compact representations: Modulations

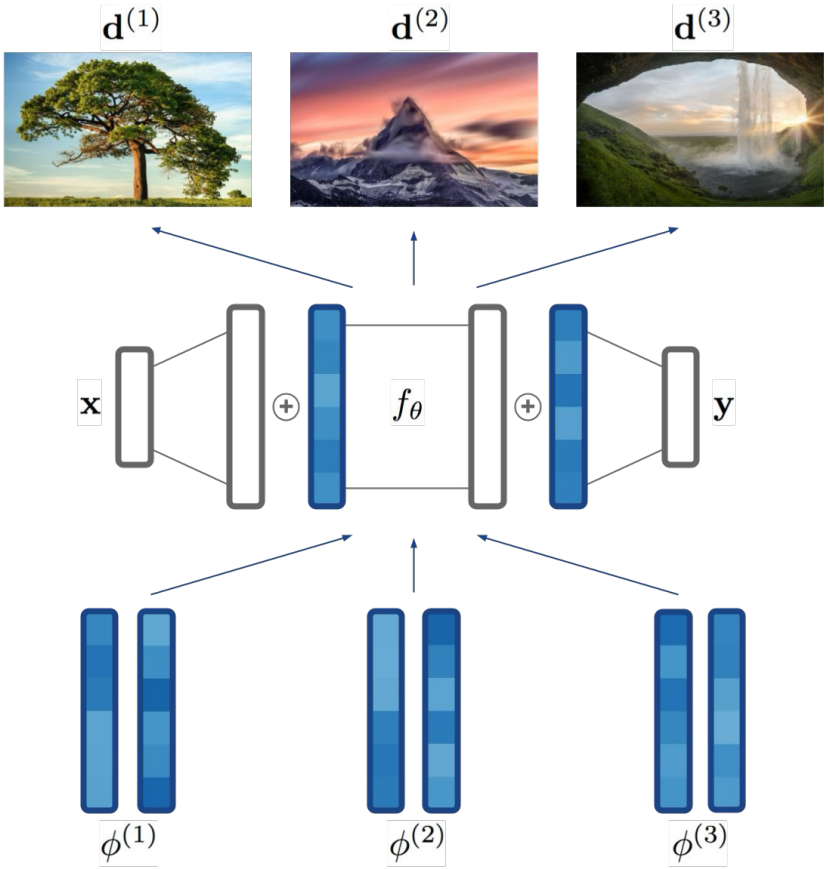


Perez et al. “FiLM: Visual Reasoning with a General Conditioning Layer”, AAAI 2018

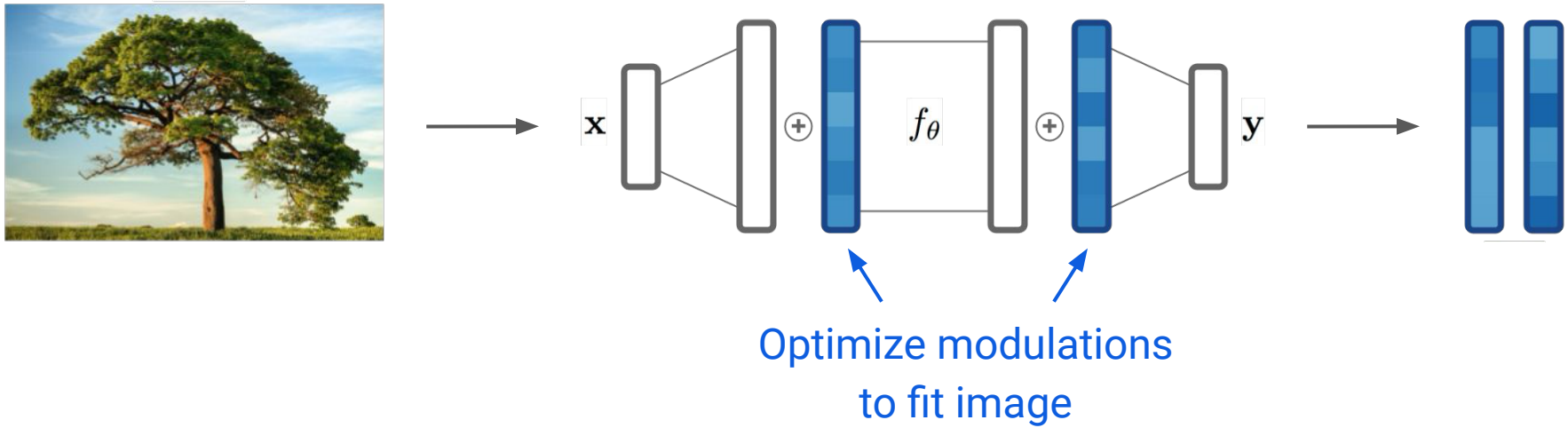
Compact representations: Modulations



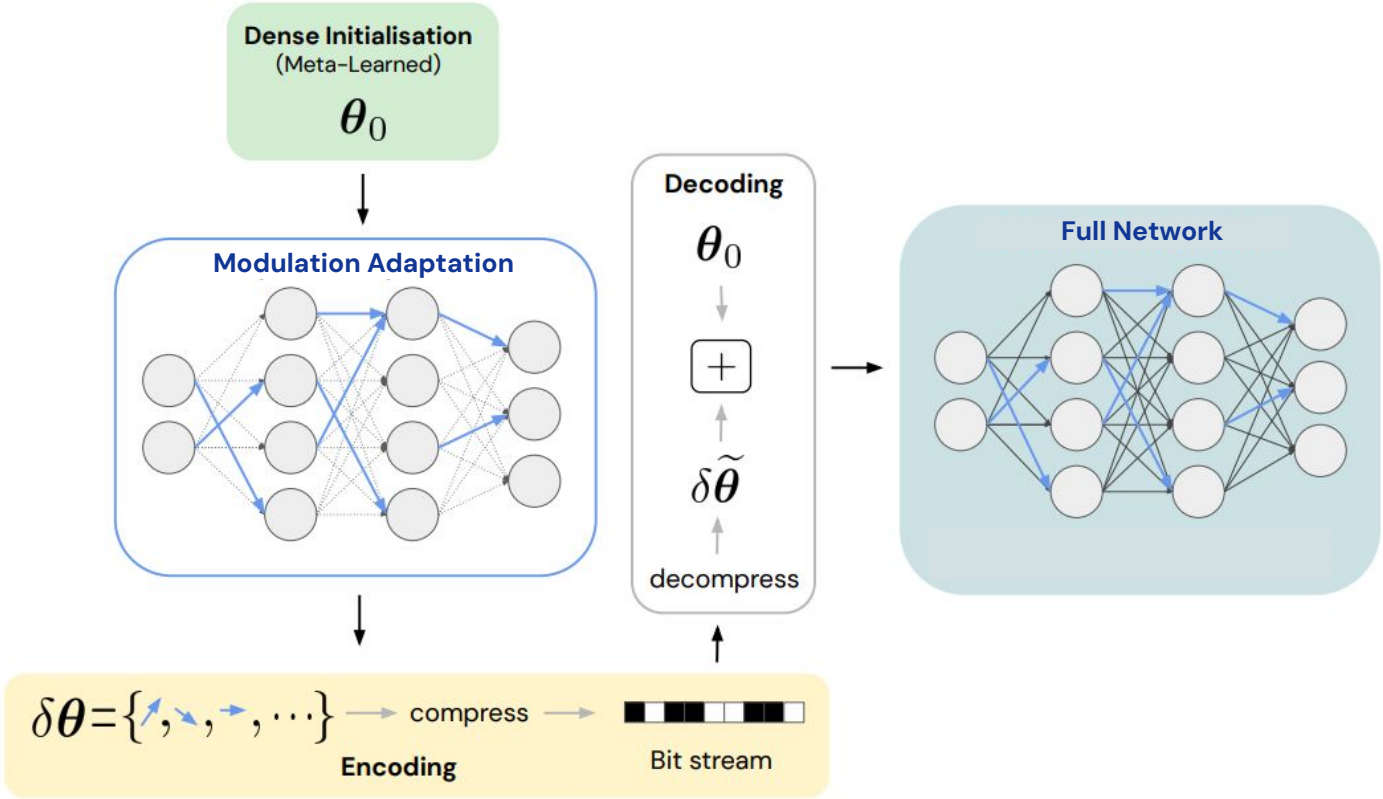
Compact representations: Modulations



Compact representations: Modulations



Modulations & Compression



Schwarz & Teh “Meta Learning Sparse Compression Networks”, TMLR 2022

Compact latent representations: Latent Modulations

Algorithm 1 Meta-learning functa

Randomly initialize shared base network θ
while not done **do**
  Sample batch $\mathcal{B}$ of data $\{\{\mathbf{x}_i^{(j)}, \mathbf{f}_i^{(j)}\}_{i \in \mathcal{I}}\}_{j \in \mathcal{B}}$
  Set batch modulations to zero $\phi_j \leftarrow 0 \; \forall j \in \mathcal{B}$
  for all step $\in \{1, \ldots, N_{inner}\}$ and $j \in \mathcal{B}$ **do**

  end for

end while

Kim & Dupont et al. “Functa”, ICML 2022 / Dupont et al. "COIN++", 2022

Compact latent representations: Latent Modulations

Algorithm 1 Meta-learning functa

1: Randomly initialize shared base network θ
2: **while** not done **do**
3: Sample batch $\mathcal{B}$ of data $\{\{\mathbf{x}_i^{(j)}, \mathbf{f}_i^{(j)}\}_{i \in \mathcal{I}}\}_{j \in \mathcal{B}}$
4: Set batch modulations to zero $\phi_j \leftarrow 0 \; \forall j \in \mathcal{B}$ Modulation
5: **for all** step $\in \{1, \ldots, N_{inner}\}$ and $j \in \mathcal{B}$ **do**
6:
7: **end for**
8:
9: **end while**

Kim & Dupont et al. “Functa”, ICML 2022 / Dupont et al. "COIN++", 2022

Compact latent representations: Latent Modulations

Algorithm 1 Meta-learning functa

1: Randomly initialize shared base network θ
2: **while** not done **do**
3: Sample batch $\mathcal{B}$ of data $\{\{\mathbf{x}_i^{(j)}, \mathbf{f}_i^{(j)}\}_{i \in \mathcal{I}}\}_{j \in \mathcal{B}}$
4: Set batch modulations to zero $\phi_j \leftarrow 0 \; \forall j \in \mathcal{B}$
5: **for all** step $\in \{1, \ldots, N_{inner}\}$ and $j \in \mathcal{B}$ **do**
6: $\phi_j \leftarrow \phi_j - \epsilon \nabla_\phi \mathcal{L}(f_{\theta,\phi}, \{\mathbf{x}_i^{(j)}, \mathbf{f}_i^{(j)}\}_{i \in \mathcal{I}})|_{\phi=\phi_j}$
7: **end for**
8:
9: **end while**

Modulation

Kim & Dupont et al. “Functa”, ICML 2022 / Dupont et al. "COIN++", 2022

Compact latent representations: Latent Modulations

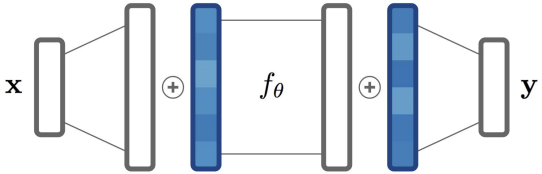
Algorithm 1 Meta-learning functa

1: Randomly initialize shared base network θ
2: **while** not done **do**
3: Sample batch $\mathcal{B}$ of data $\{\{\mathbf{x}_i^{(j)}, \mathbf{f}_i^{(j)}\}_{i \in \mathcal{I}}\}_{j \in \mathcal{B}}$
4: Set batch modulations to zero $\phi_j \leftarrow 0 \; \forall j \in \mathcal{B}$
5: **for all** step $\in \{1, \ldots, N_{inner}\}$ and $j \in \mathcal{B}$ **do**
6: $\phi_j \leftarrow \phi_j - \epsilon \nabla_\phi \mathcal{L}(f_{\theta,\phi}, \{\mathbf{x}_i^{(j)}, \mathbf{f}_i^{(j)}\}_{i \in \mathcal{I}})|_{\phi=\phi_j}$
7: **end for**
8: $\theta \leftarrow \theta - \epsilon' \frac{1}{|\mathcal{B}|} \sum_{j \in \mathcal{B}} \nabla_\theta \mathcal{L}(f_{\theta,\phi}, \{\mathbf{x}_i^{(j)}, \mathbf{f}_i^{(j)}\}_{i \in \mathcal{I}})|_{\phi=\phi_j}$
9: **end while**

Shared Network

Kim & Dupont et al. “Functa”, ICML 2022 / Dupont et al. "COIN++", 2022

Modulation Variants



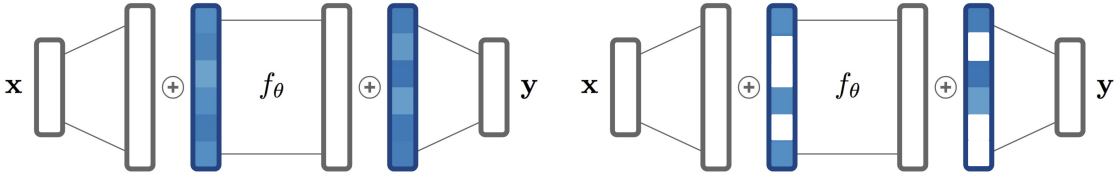
FiLM Style Modulations:
[Perez et al, 2018, Mehta et al. 2021]

- Parameter-compact shift and scale modulations

$$f(\boldsymbol{\gamma} \odot (\mathbf{W}\mathbf{x} + \mathbf{b}) + \boldsymbol{\beta})$$

- Drastically reduce per-signal parameter count
- Play nicely with Meta-Learning

Modulation Variants



FiLM Style Modulations:
[Perez et al, 2018, Mehta et al. 2021]

- Parameter-compact shift and scale modulations

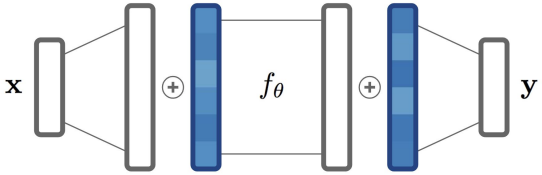
$$f(\boldsymbol{\gamma} \odot (\mathbf{W}\mathbf{x} + \mathbf{b}) + \boldsymbol{\beta})$$

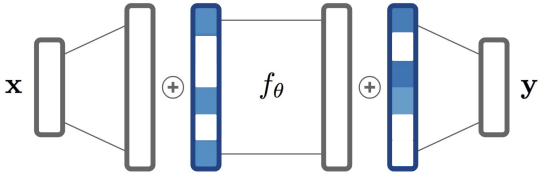
- Drastically reduce per-signal parameter count
- Play nicely with Meta-Learning

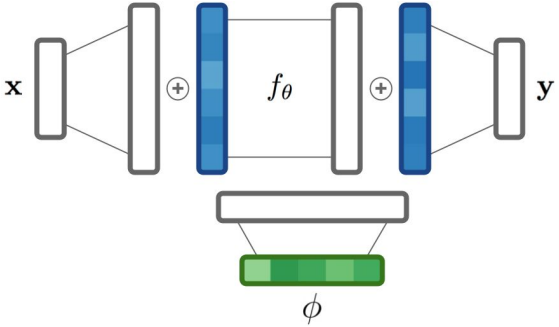
Sparse Modulations:
[Schwarz et al, 2022]

- Inspired by NN Pruning Literature
- "Learning where to learn" property
- Introduces Approximate Inference in the inner loop

Modulation Variants







FiLM Style Modulations:
[Perez et al, 2018, Mehta et al. 2021]

- Parameter-compact shift and scale modulations

$$f(\boldsymbol{\gamma} \odot (\mathbf{W}\mathbf{x} + \mathbf{b}) + \boldsymbol{\beta})$$

- Drastically reduce per-signal parameter count
- Play nicely with Meta-Learning

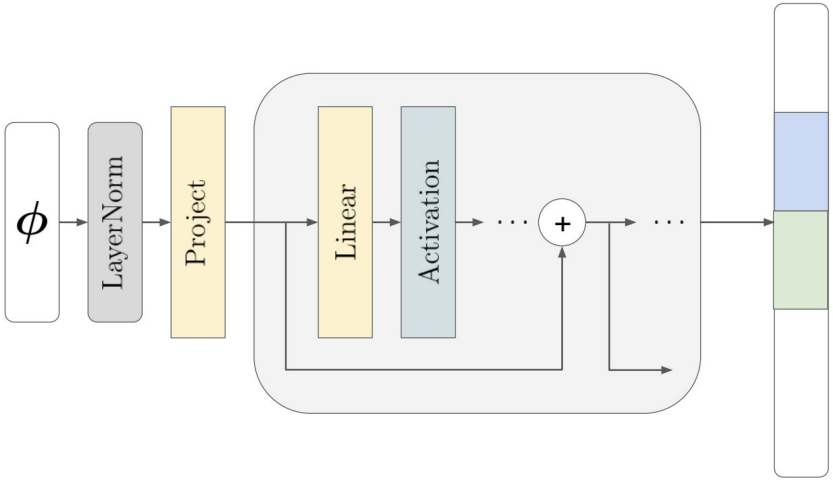
Sparse Modulations:
[Schwarz et al, 2022]

- Inspired by NN Pruning Literature
- "Learning where to learn" property
- Introduces Approximate Inference in the inner loop

Latent Modulations:
[Dupont et al. 2022a & 2022b,
Bauer et al. 2023, Schwarz et al 2023]

- Show best performance
- Can be tricky to optimize (HyperNets)
- Non-linear latent -> modulation mappings can be computationally expensive

Latent Mapping + Sparsity (VC-INR)



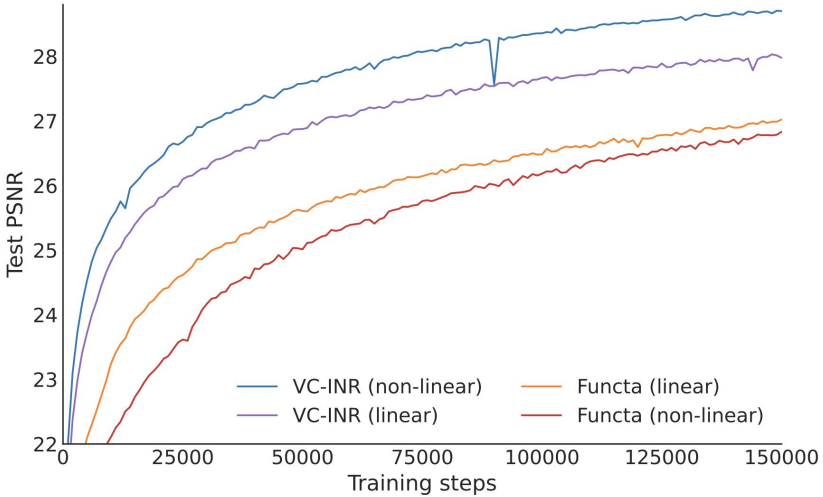
$$\mathbf{x} \mapsto f((\mathbf{G}_{\texttt{low}}^{(l)} \odot \mathbf{W}^{(l)})\mathbf{x} + \mathbf{b}^{(l)})$$

$$\mathbf{G}_{\texttt{low}}^{(l)} := \sigma\left(\mathbf{U}^{(l)} \begin{matrix} \mathbf{V}^{(l)^T} \\ \ddots \end{matrix} \right)$$

Non-linear projection from latent representation ϕ to $\mathbf{G}_{\texttt{low}}^{(l)}$.

Schwarz & Tack et al. "Modality-Agnostic Variational Compression of Implicit Neural Representations", ICML 2023

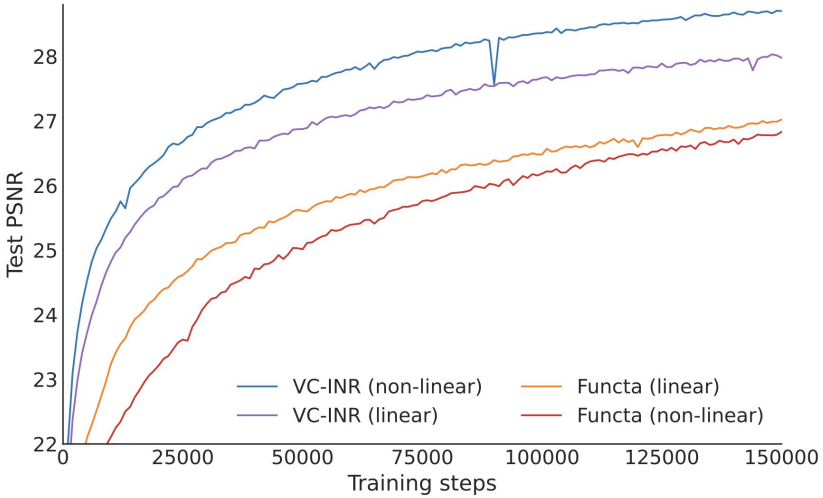
Improving Sparse Adaptation through Meta-Learning



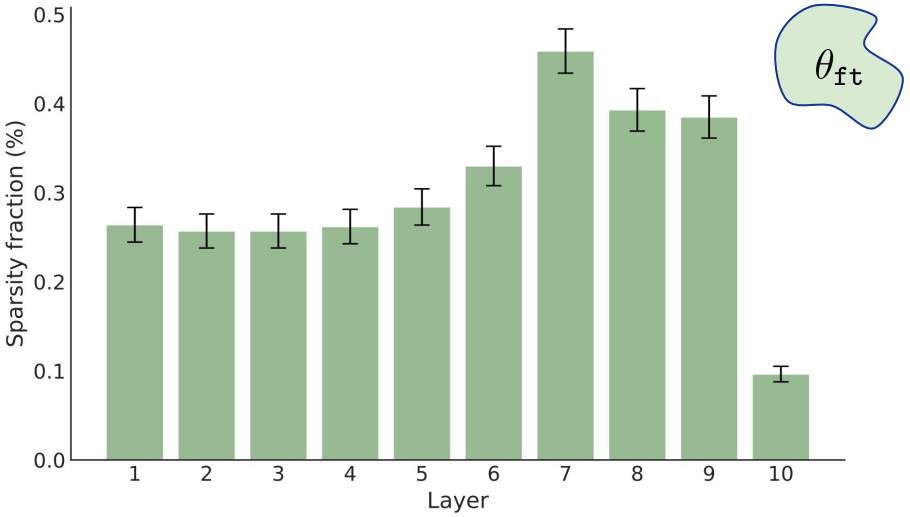
(a) Learning curves

Schwarz & Tack et al. “Modality-Agnostic Variational Compression of Implicit Neural Representations”, ICML 2023

Improving Sparse Adaptation through Meta-Learning



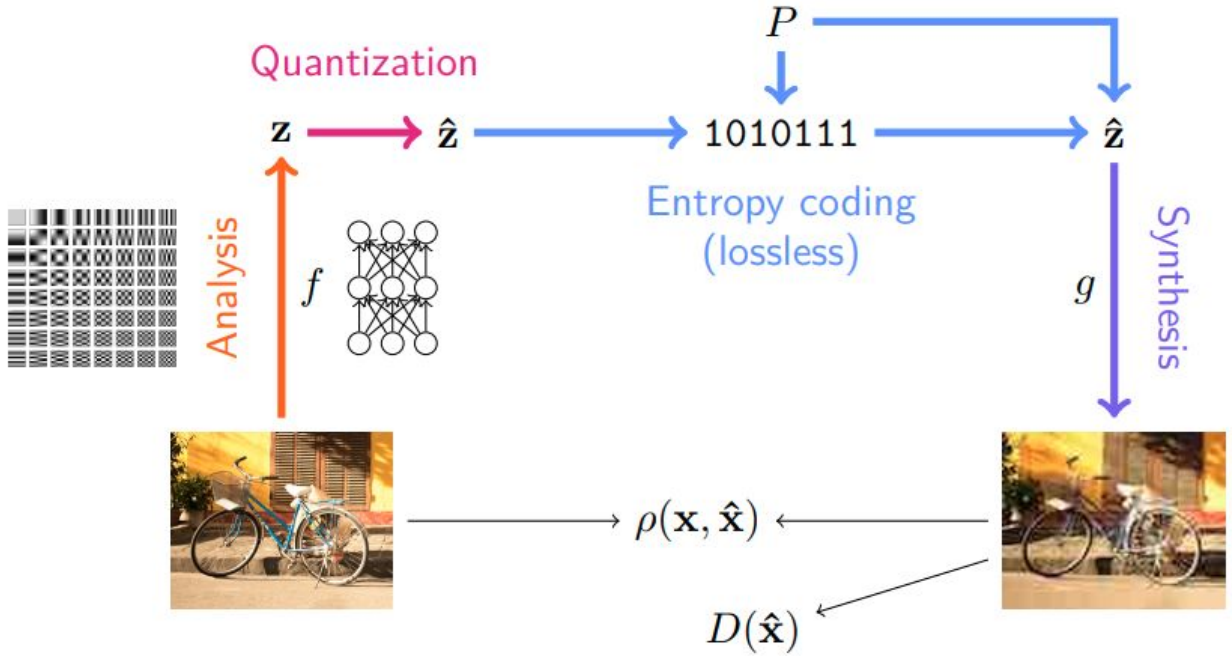
(a) Learning curves



(b) Sparsity pattern

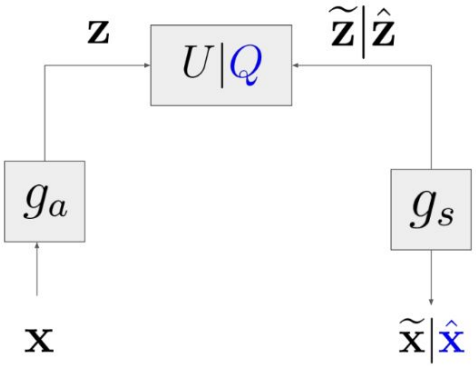
Schwarz & Tack et al. "Modality-Agnostic Variational Compression of Implicit Neural Representations", ICML 2023

Neural Image Compression: Autoencoder-based approach



Quantization & Entropy Coding

- Inference path

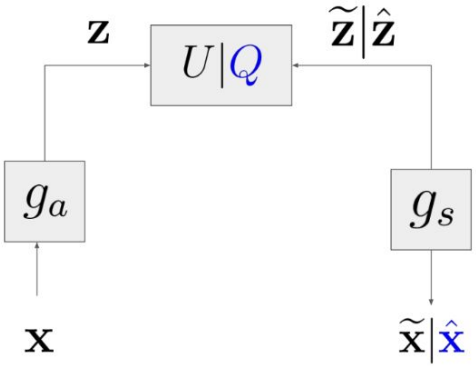


(a)

Neural Data Compression with non-linear transform coding [Balle et al., 2018]

Quantization & Entropy Coding

- Inference path

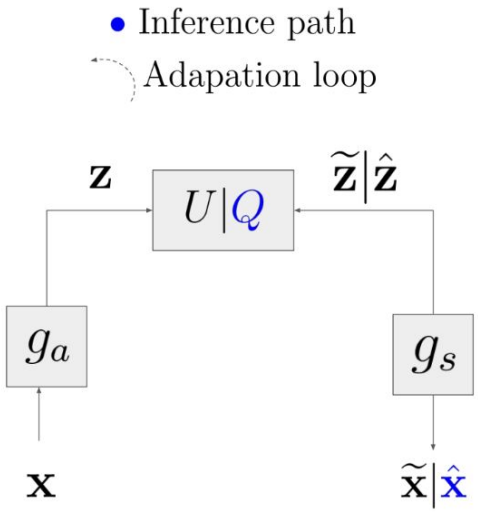


(a)

Neural Data Compression with non-linear transform coding [Balle et al., 2018]

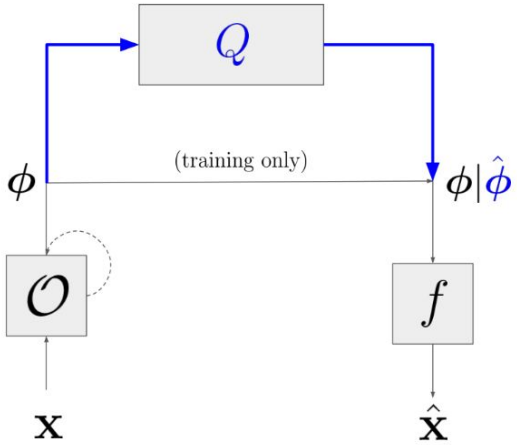
- Uniform Quantization
- Additive Uniform Noise
- Deep CDF Model

Quantization & Entropy Coding



(a)

Neural Data Compression with non-linear transform coding [Balle et al., 2018]

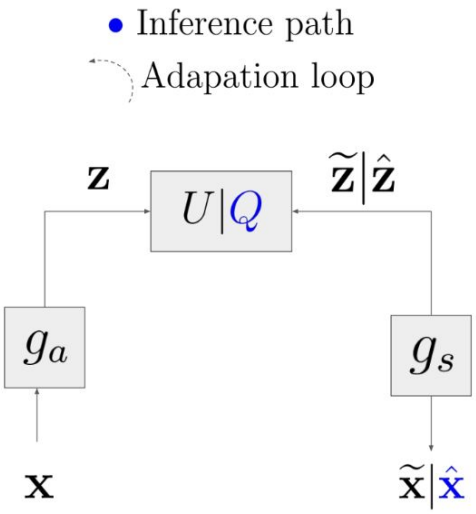


(b)

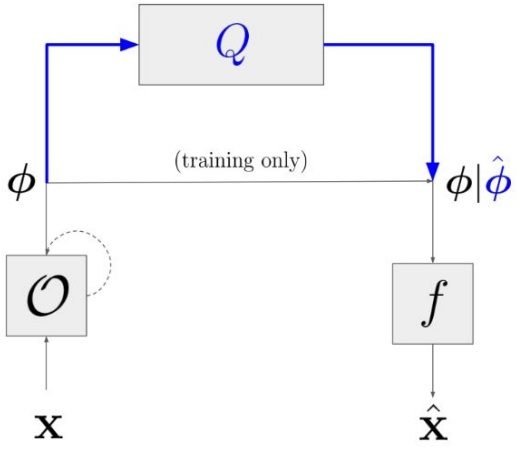
Data compression with INRs (e.g. COIN, modulated models)

- Post-hoc quantisation
- Probability model based on training statistics

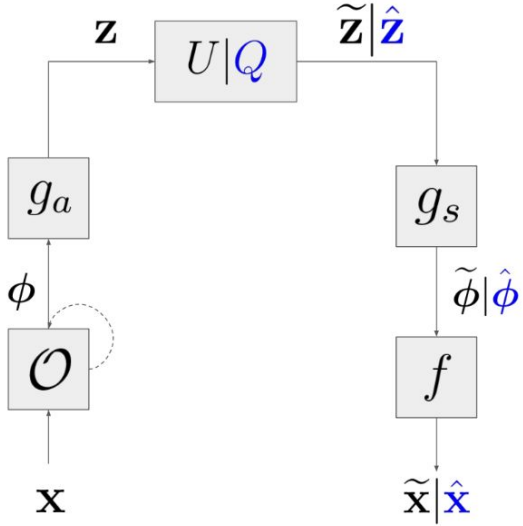
Quantization & Entropy Coding



(a)
Neural Data Compression with non-linear transform coding [Balle et al., 2018]

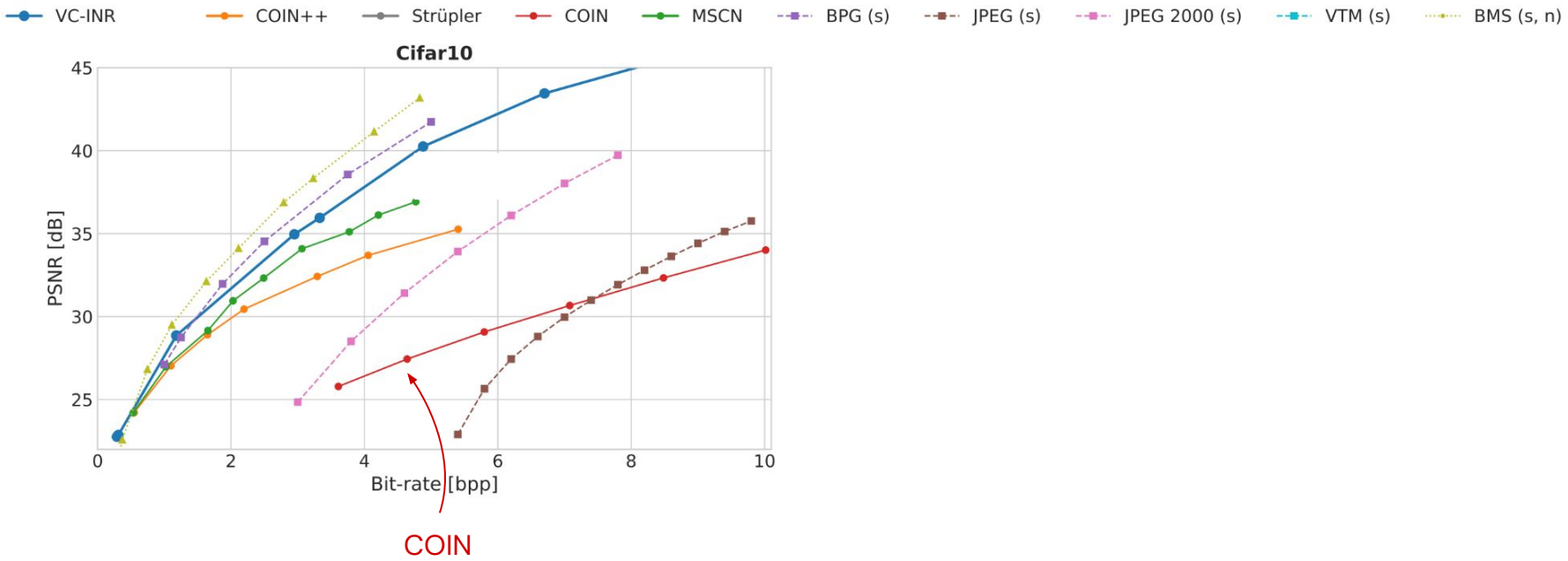


(b)
Data compression with INRs (e.g. COIN, modulated models)

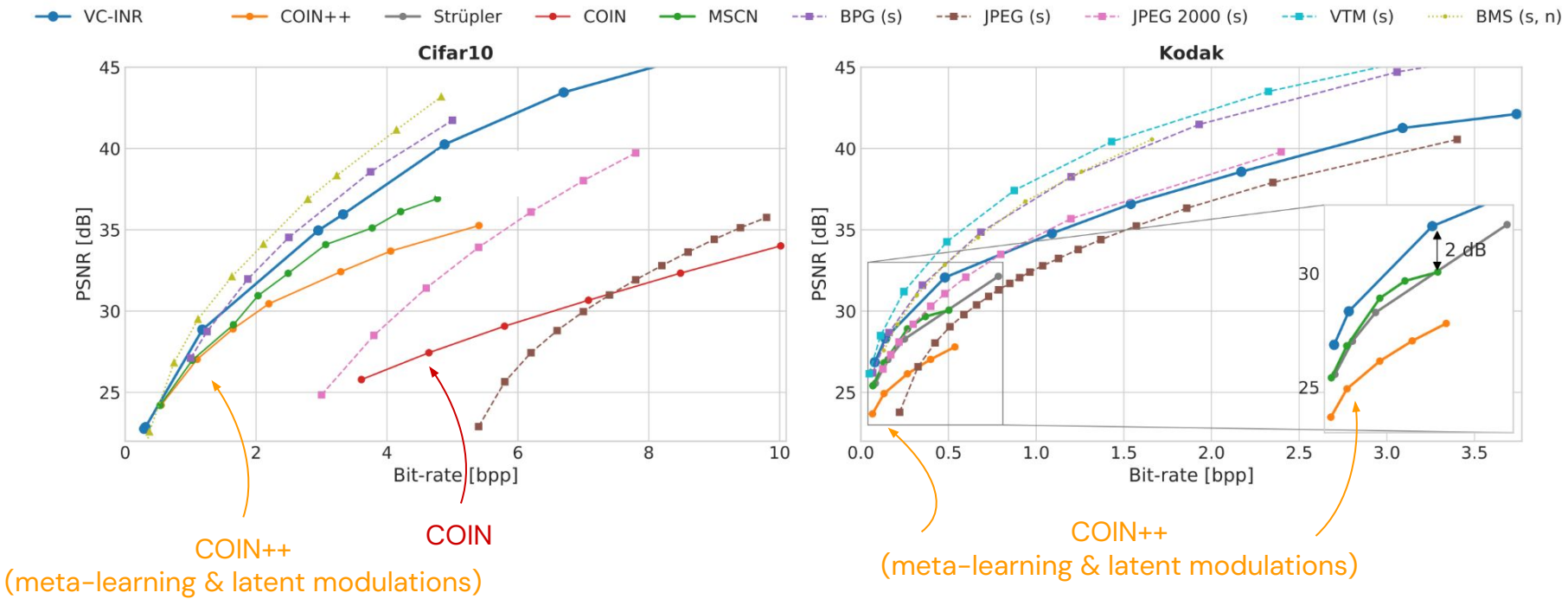


(c)
Non-linear transform coding on Functasets (VC-INR)

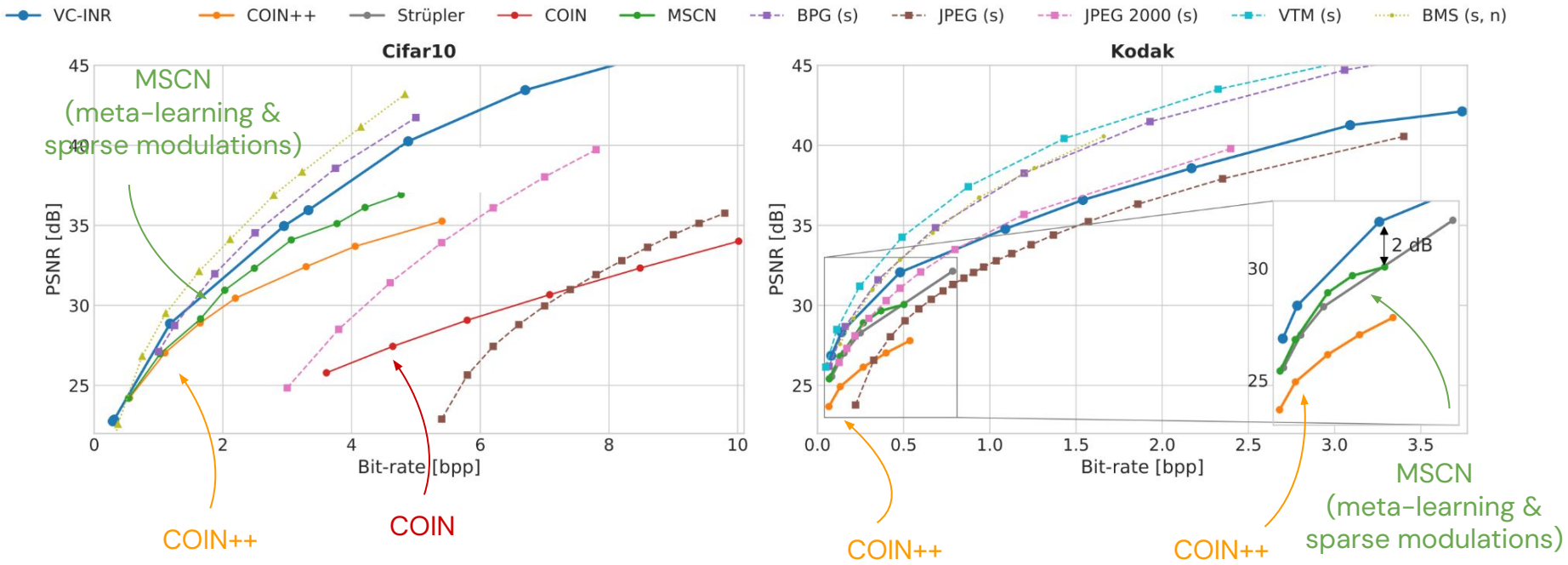
Progress since COIN



Progress since COIN

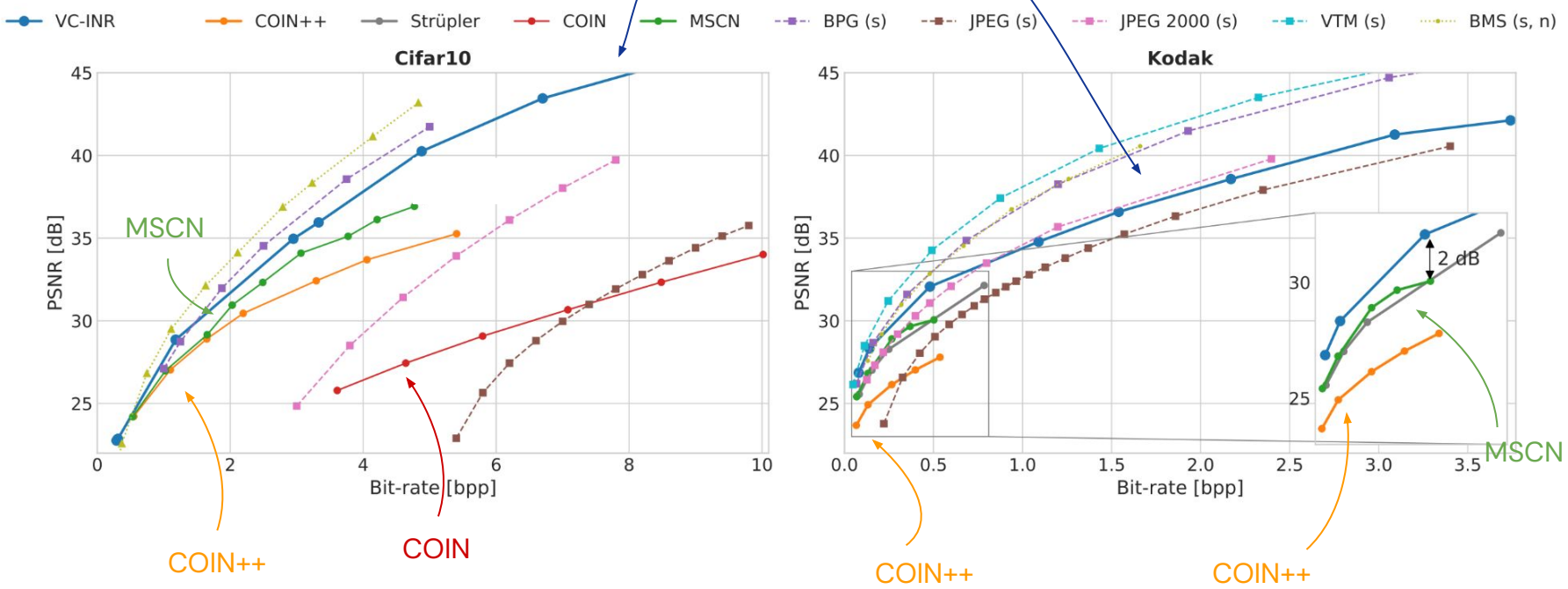


Progress since COIN



Progress since COIN

VC-INR
(meta-learning, sparsity + latent modulations, deep entropy coding)



Two approaches in neural image compression

Compress a distribution of images

Balle 2018 & follow ups (autoencoder)

COIN++ (neural field)

VC-INR (neural field)

...

Compress a single image

COIN (neural field)

NeRV (neural field)

Two approaches in neural image compression

Compress a distribution of images

Balle 2018 & follow ups (autoencoder)

COIN++ (neural field)

VC-INR (neural field)

...

Compress a single image

COIN (neural field)

NeRV (neural field)

COOL-CHIC (neural field)

C3 (neural field)

An issue with neural compression

"[...] many practical and theoretical and challenges remain. Chief among them is **computational complexity**, which stands in the way of the wider adoption of neural compression."

An introduction to Neural Data Compression, Yang et al, 2022

An issue with neural compression

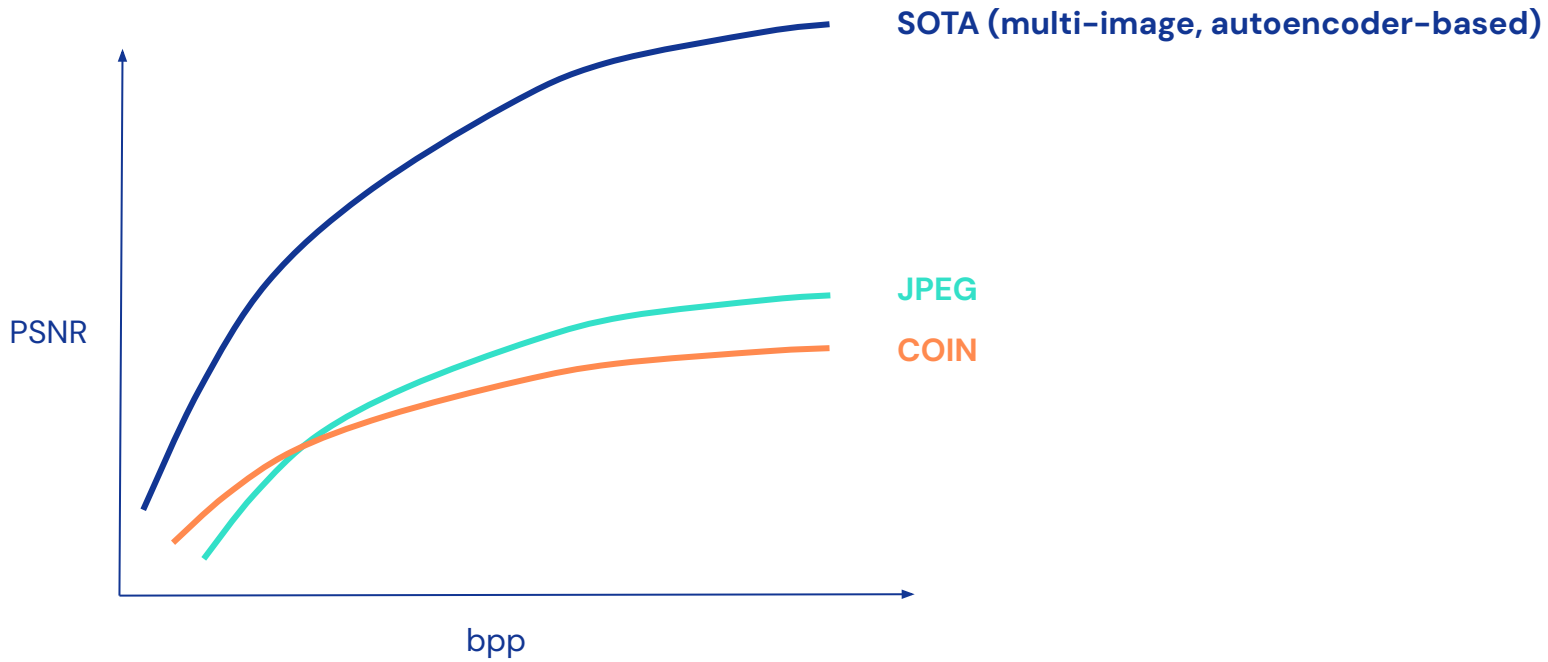
"[...] many practical and theoretical and challenges remain. Chief among them is **computational complexity**, which stands in the way of the wider adoption of neural compression."

An introduction to Neural Data Compression, Yang et al, 2022

Single image compression has computationally cheap decoding → potential solution to this problem

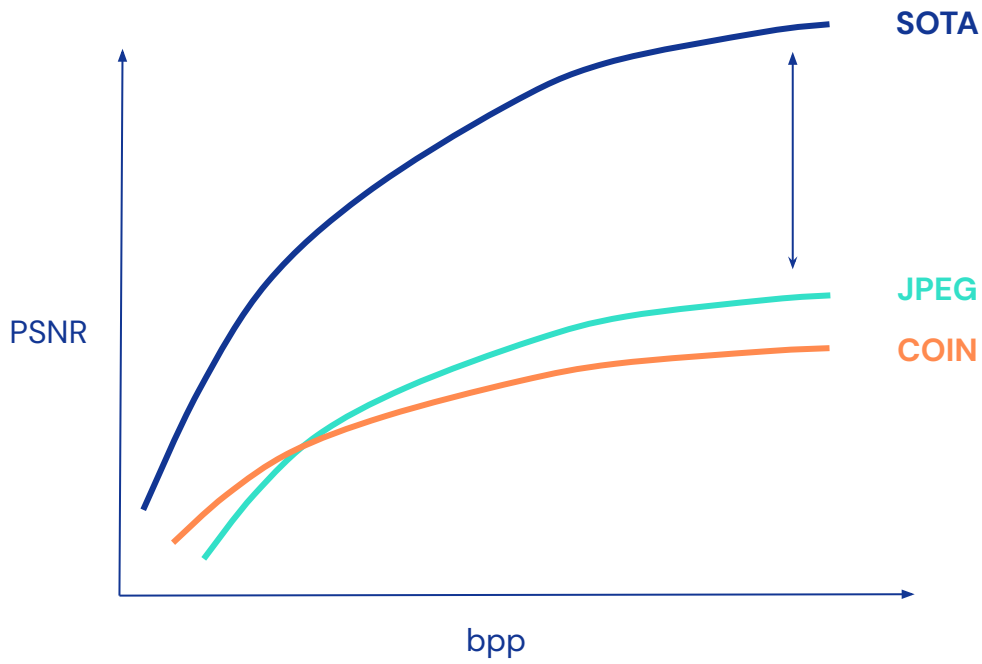
The main issue with single image compression

Typical rate distortion plot



The main issue with single image compression

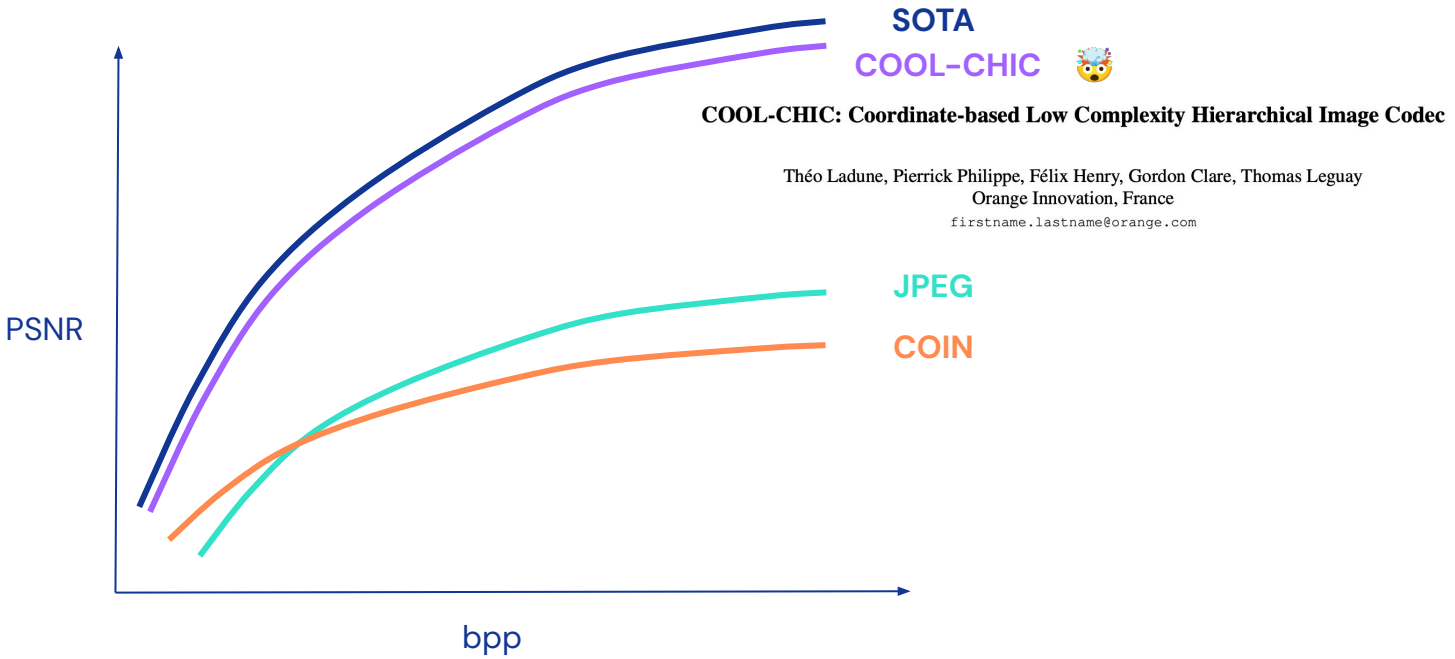
Typical rate distortion plot



Possible explanation: Natural **images share a lot of structure**. To narrow down the gap, model needs to capture shared structure.

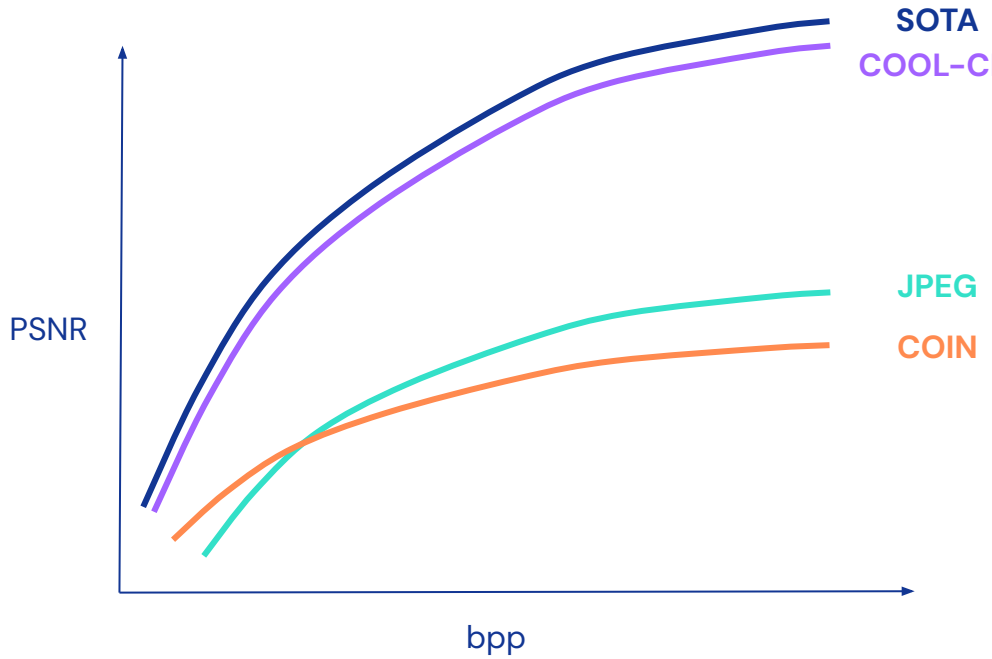
The main issue with single image compression

Typical rate distortion plot



The main issue with single image compression

Typical rate distortion plot

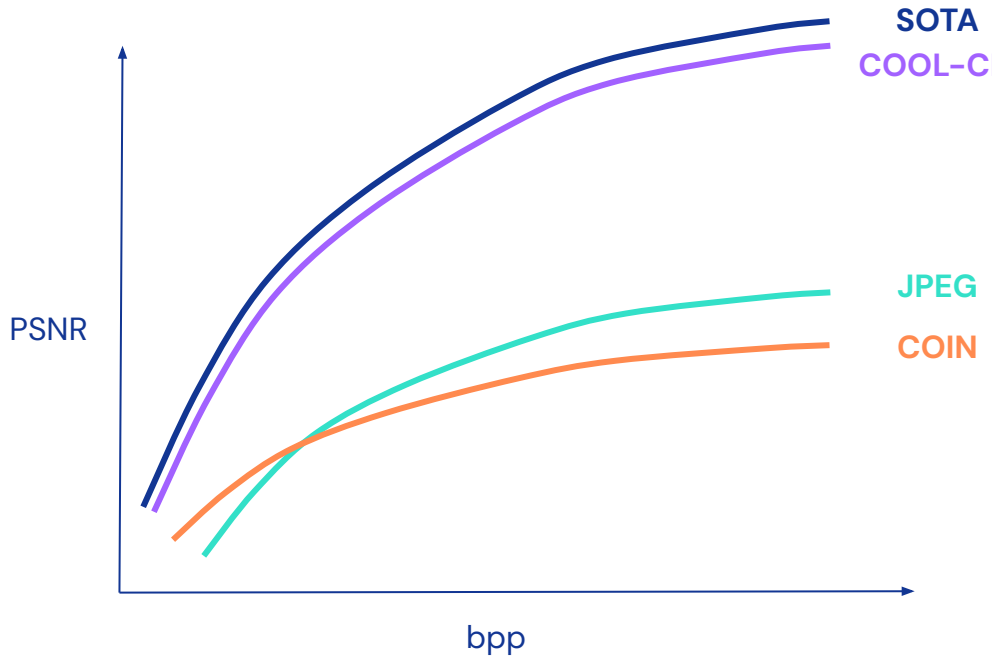


Shared structure not necessarily needed!

We can get close to SOTA by training on a single image. So most of the R-D gain you can get does *not* actually come from information shared across images

The main issue with single image compression

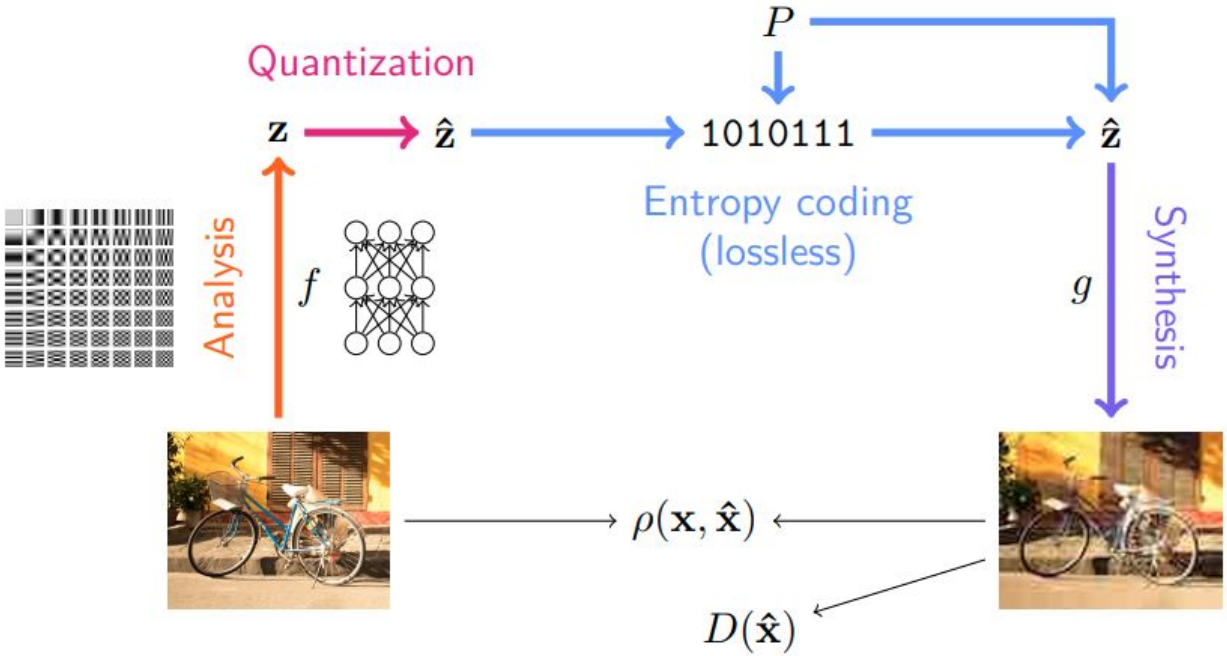
Typical rate distortion plot



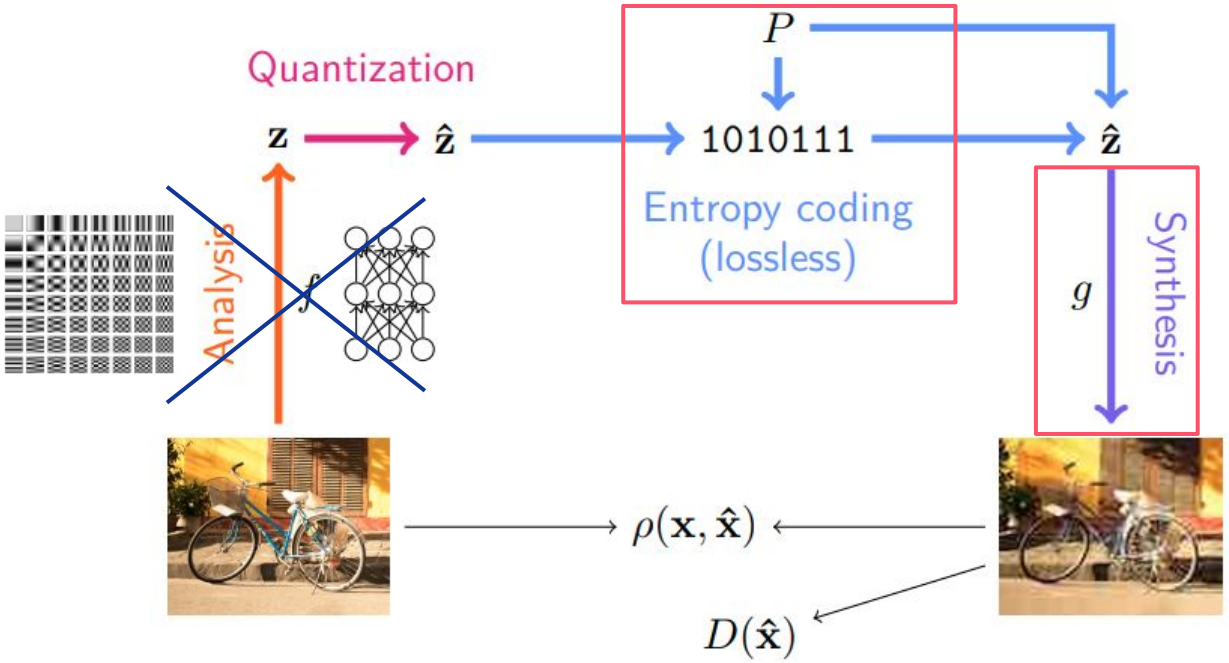
Also COOL-CHIC is **orders of magnitudes fewer flops for decoding** compared to SOTA!

(*encoding is very slow for now)

Neural image compression (again)



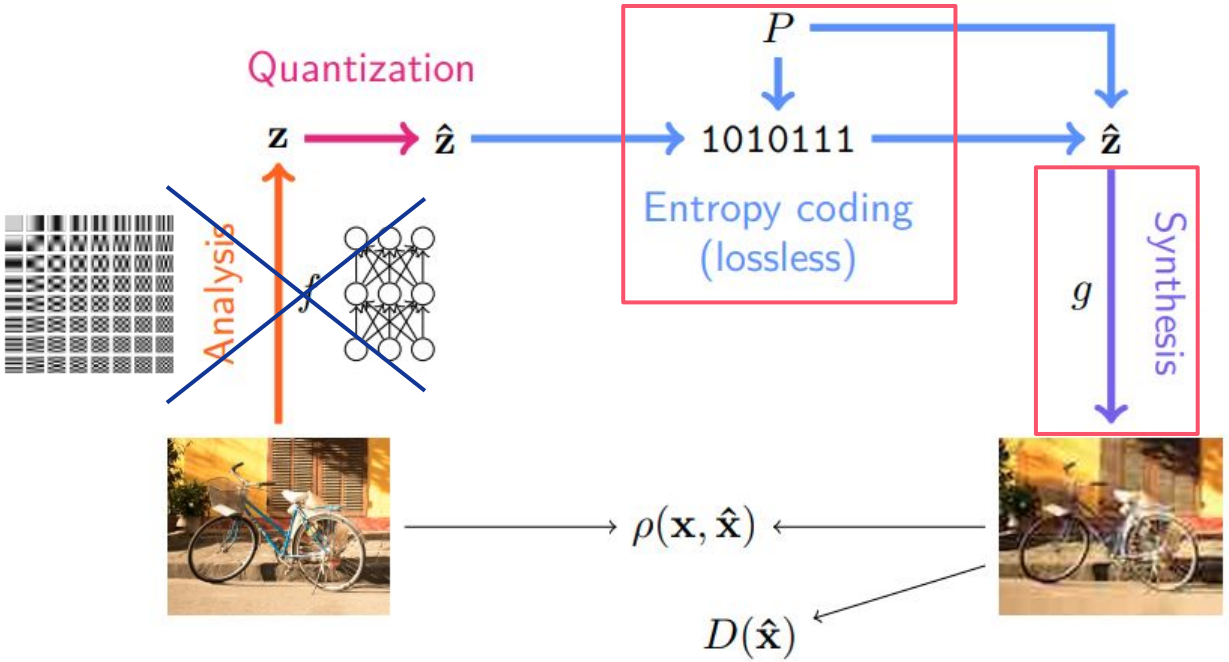
Neural image compression (again)



All three elements are learned **per image:**

- Latents (z)
- Entropy model (P)
- Synthesis model (g)

Neural image compression (again)

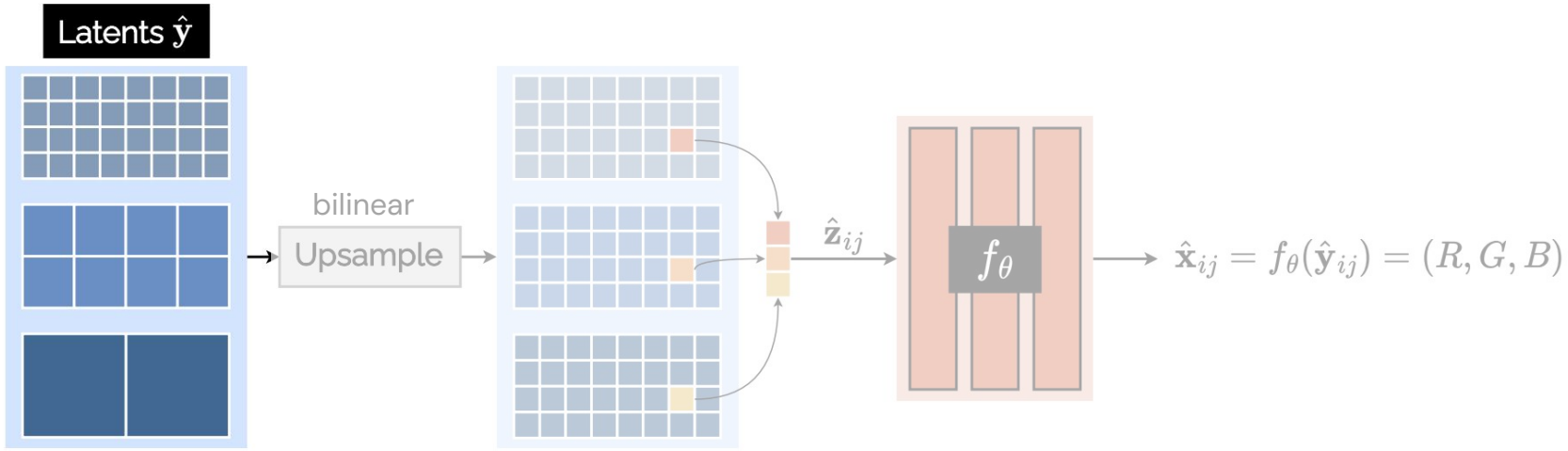


All three elements are learned **per image:**

- Latents (z)
- Entropy model (P)
- Synthesis model (g)

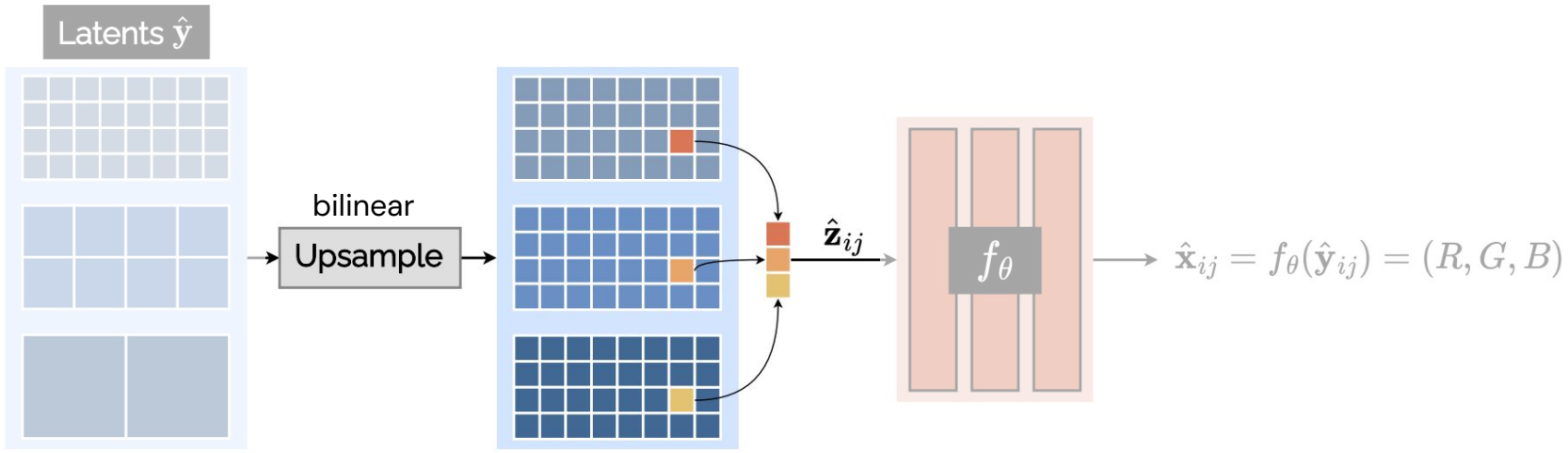
All three need to be transmitted!

Latents & synthesis network

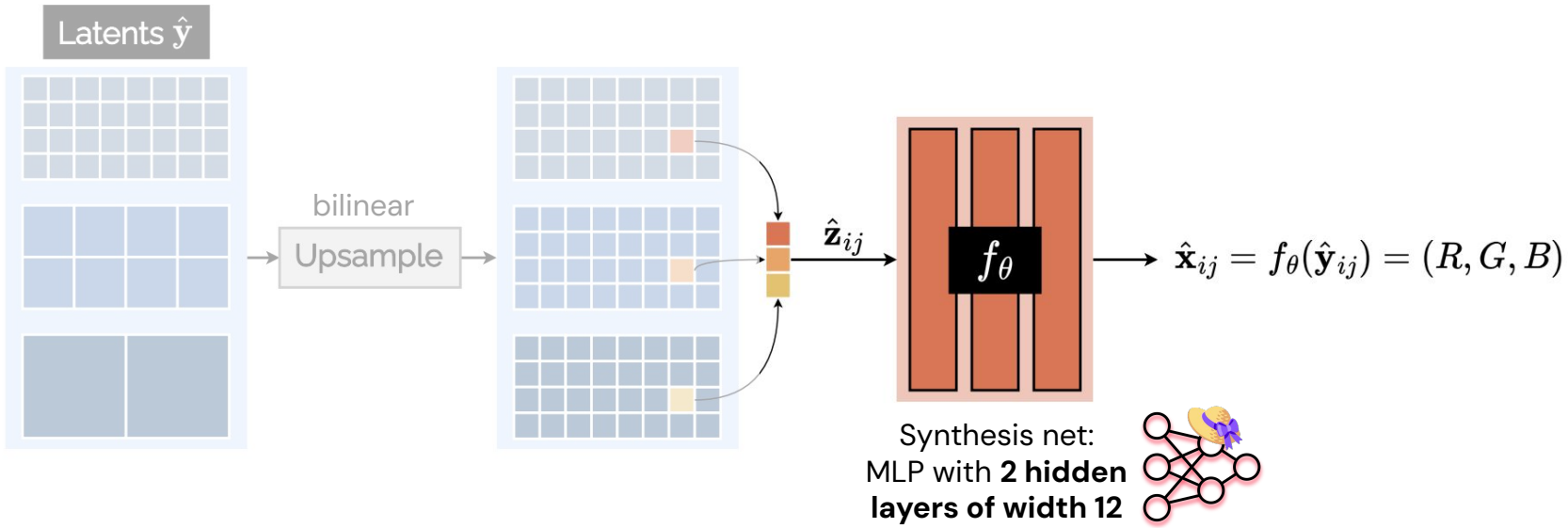


Several latent grids at different resolutions

Latents & synthesis network



Latents & synthesis network



Compression != Dimensionality Reduction

Note that the image is expressed using **latents** that can be **even higher dimensional than the image itself**.

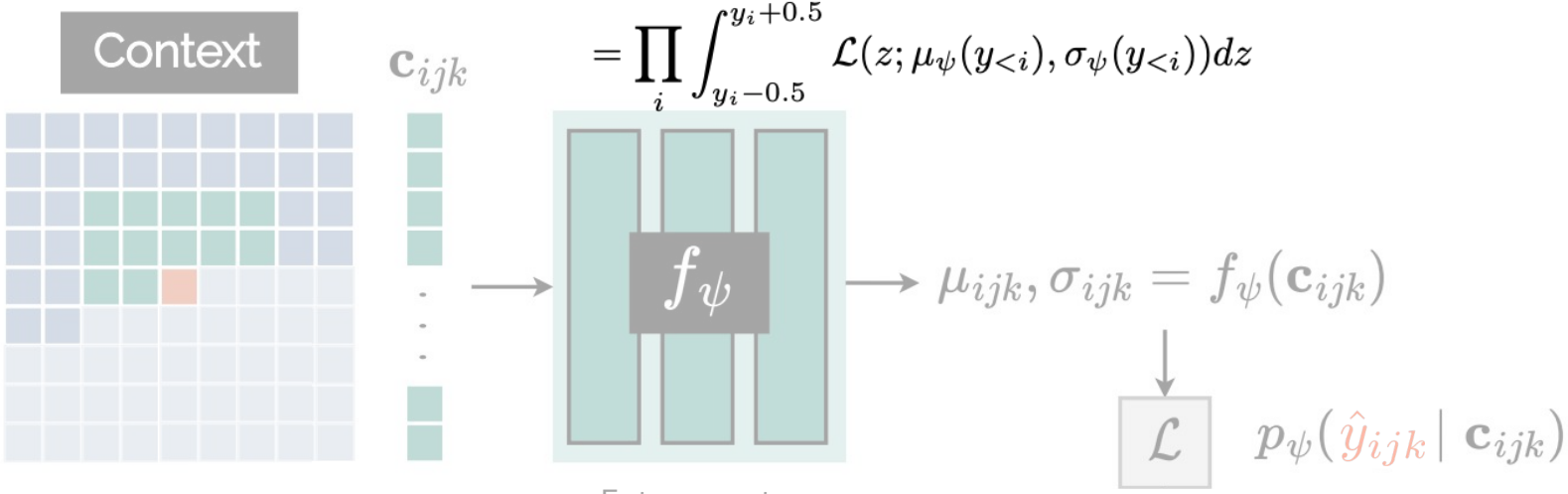
This learned representation allows to represent the image in a latent space in which the **latent dims have low entropy** i.e. highly compressible via entropy coding.

So compression doesn't necessarily give us representations useful for downstream ML tasks.

Autoregressive Entropy Model for Latents

$$p_\psi(y) = \prod_{i=1}^{\dim(y)} p_\psi(y_i|y_{<i})$$

$$= \prod_i \int_{y_i-0.5}^{y_i+0.5} \mathcal{L}(z; \mu_\psi(y_{<i}), \sigma_\psi(y_{<i}))dz$$

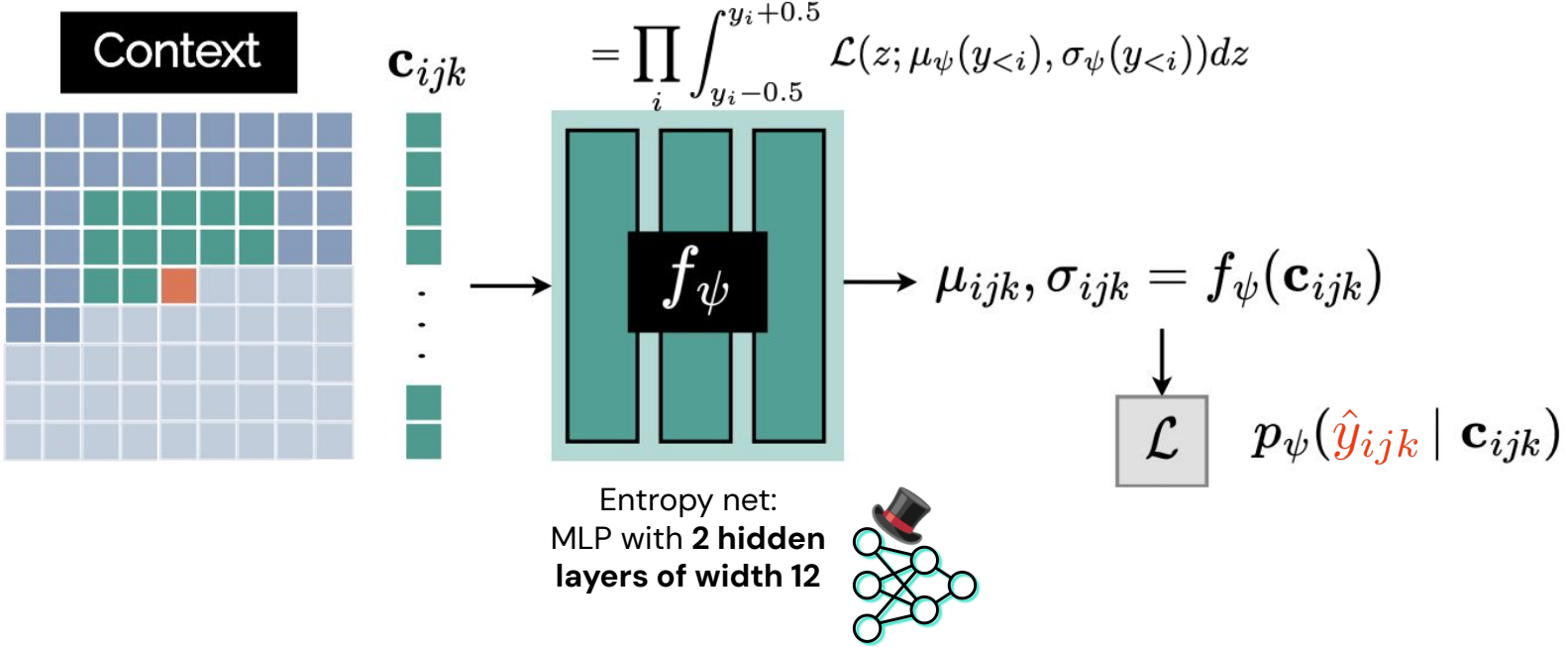


Entropy net:
MLP with **2 hidden layers of width 12**

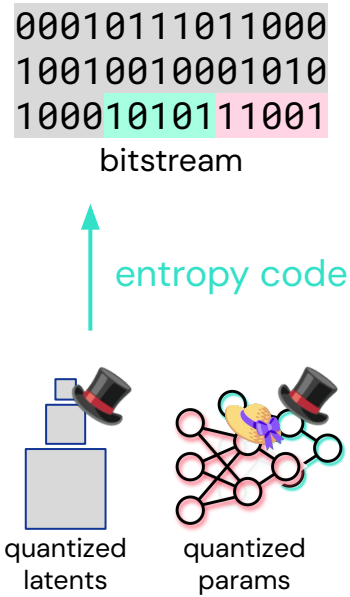
Autoregressive Entropy Model for Latents

$$p_\psi(y) = \prod_{i=1}^{\dim(y)} p_\psi(y_i|y_{<i})$$

$$= \prod_i \int_{y_i-0.5}^{y_i+0.5} \mathcal{L}(z; \mu_\psi(y_{<i}), \sigma_\psi(y_{<i}))dz$$



Entropy encoding



Quantized latents

Encoded via autoregressive Laplace distrib

$$p_\psi(y) = \prod_{i=1}^{\dim(y)} p_\psi(y_i|y_{<i})$$

$$= \prod_i \int_{y_i-0.5}^{y_i+0.5} \mathcal{L}(z; \mu_\psi(y_{<i}), \sigma_\psi(y_{<i}))dz$$

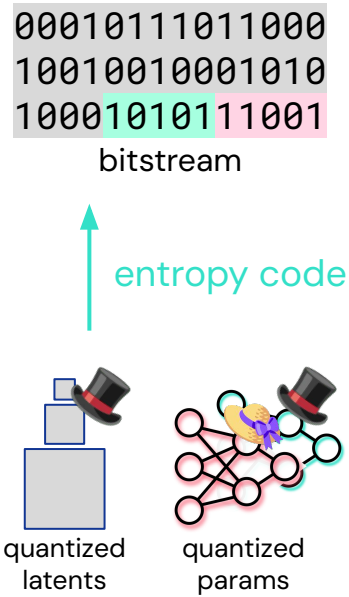
Quantized network parameters

Factorised Laplace distribution

$$p(\hat{\theta}) = \prod_j p(\hat{\theta}_j)$$

$$= \prod_j \int_{\hat{\theta}_j-0.5}^{\hat{\theta}_j+0.5} \mathcal{L}(z; 0, \hat{\sigma})dz, \quad \text{where } \hat{\sigma} = \text{stddev}(\hat{\theta})$$

Entropy encoding



Quantized latents

Encoded via autoregressive Laplace distrib

$$p_\psi(y) = \prod_{i=1}^{\dim(y)} p_\psi(y_i|y_{<i})$$

$$= \prod_i \int_{y_i-0.5}^{y_i+0.5} \mathcal{L}(z; \mu_\psi(y_{<i}), \sigma_\psi(y_{<i}))dz$$

Quantized network parameters

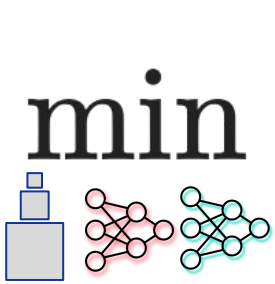
Factorised Laplace distribution

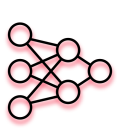
$$p(\hat{\theta}) = \prod_j p(\hat{\theta}_j)$$

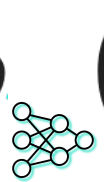
$$= \prod_j \int_{\hat{\theta}_j-0.5}^{\hat{\theta}_j+0.5} \mathcal{L}(z; 0, \hat{\sigma})dz, \quad \text{where } \hat{\sigma} = \text{stddev}(\hat{\theta})$$

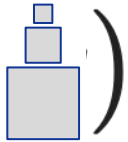
Loss: "Overfit" each image with rate-distortion loss

$$\min \left\| \quad \right\|^2 - \lambda \log p$$









reconstruction/distortion loss

(autoregressive) rate loss

larger λ lower rate, worse reconstruction

smaller λ larger rate, better reconstruction

Training

What COOL-CHIC wants to optimize: Objective over **quantized** parameters

$$\min \left\| \text{decoder}(\text{latents}) - \text{image} \right\|^2 - \lambda \log p_{\text{entropy model}}(\text{latents})$$

reconstruction/distortion loss | (autoregressive) rate loss

What COOL-CHIC can optimize: Objective over **continuous** parameters

$$\min \left\| \text{decoder}(\text{latents}) - \text{image} \right\|^2 - \lambda \log p_{\text{entropy model}}(\text{latents})$$

reconstruction/distortion loss | (autoregressive) rate loss

Training

In particular, COOL-CHIC uses 2 stages of optimization:

Stage 1 (long: 50-100k iterations)

- Add U[-0.5, 0.5] noise to latents - **mimic quantization**

Stage 2 (short: 5-10k iterations)

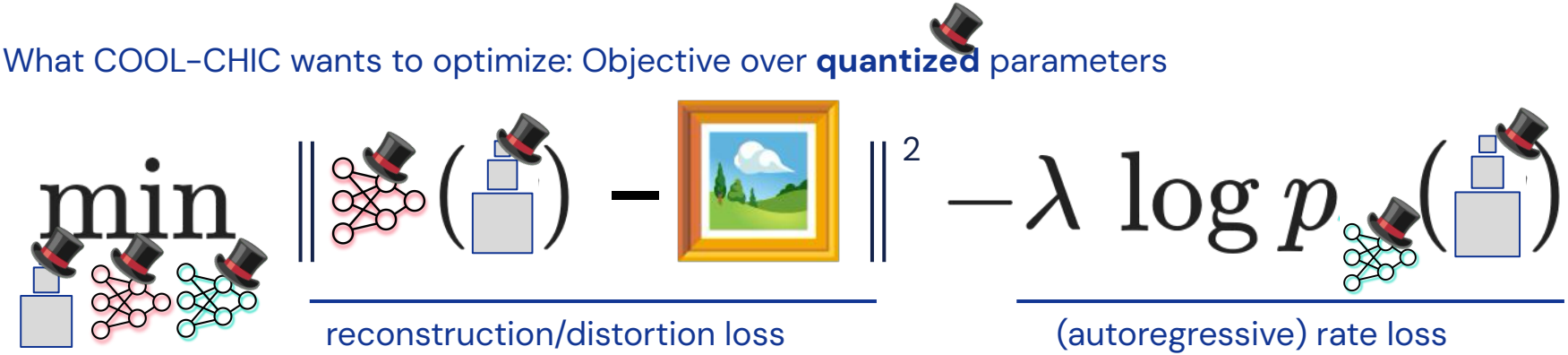
- Use quantized value of z with **straight-through gradient estimation**

Stage 1	$\nabla_{\mathbf{z},\theta,\psi}\mathcal{L}_{\theta,\psi}(\mathbf{z}+\mathbf{u}); \qquad \mathbf{u} \sim \text{Uniform}(0,1)$
Stage 2	$\nabla_{\theta,\psi}\mathcal{L}_{\theta,\psi}(\lfloor \mathbf{z} \rceil)$ and $\widetilde{\nabla}_{\mathbf{z}}\mathcal{L}_{\theta,\psi}(\lfloor \mathbf{z} \rceil)$

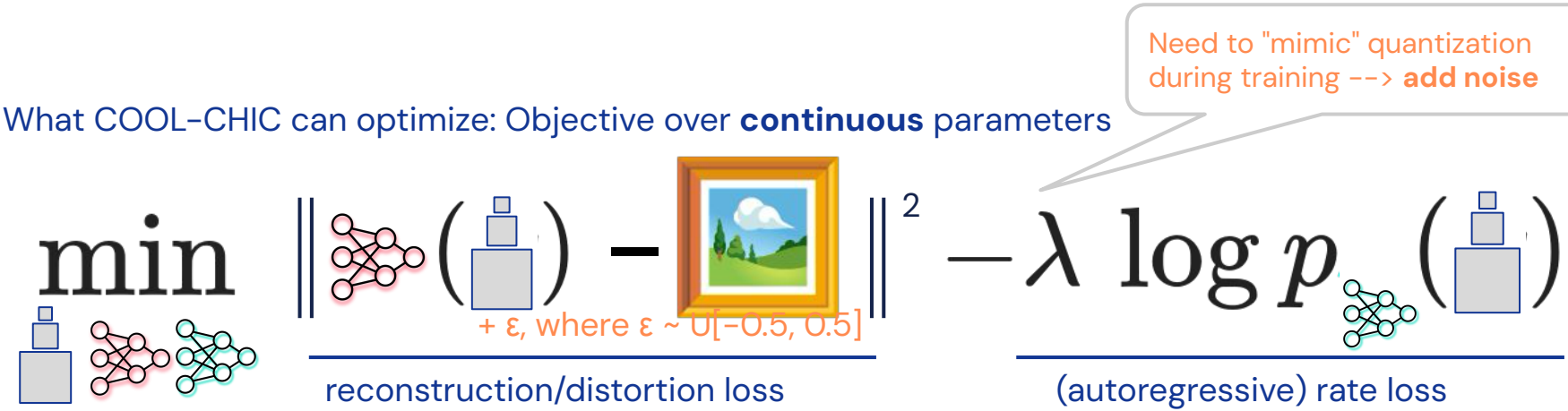
Table 1. Two-stage optimization; $\widetilde{\nabla}_{\mathbf{z}}$ is straight-through estimation.

Training

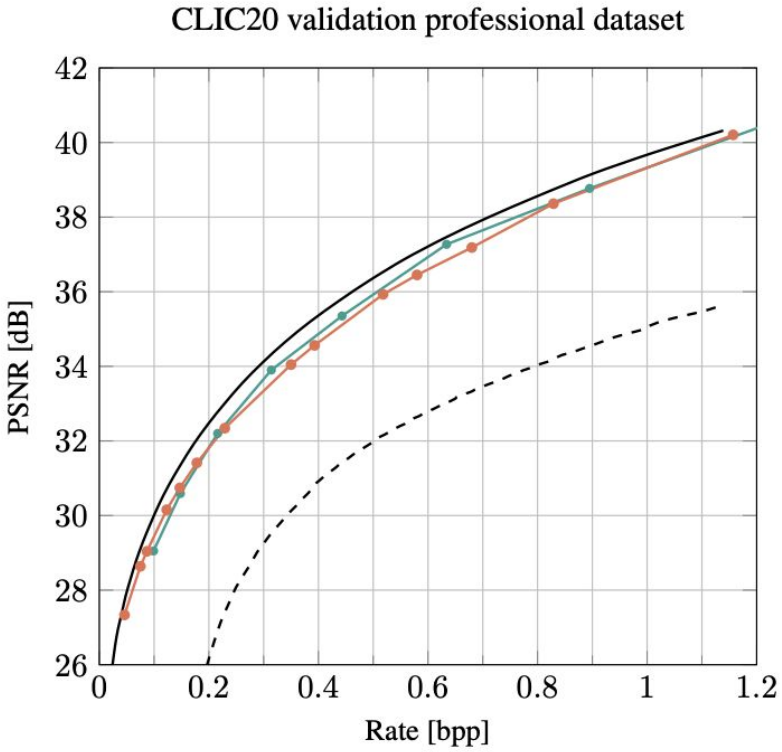
What COOL-CHIC wants to optimize: Objective over **quantized** parameters

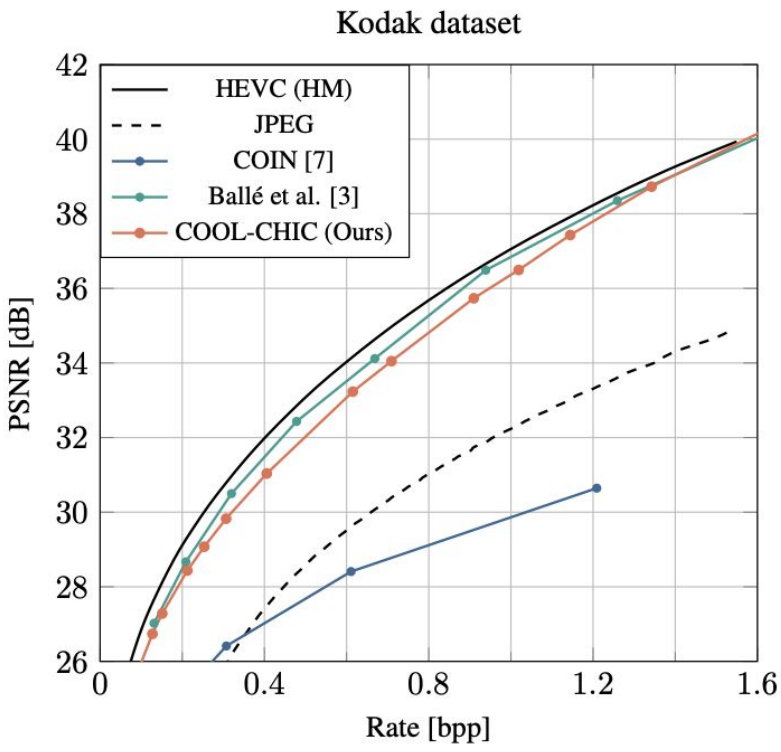


What COOL-CHIC can optimize: Objective over **continuous** parameters



Results - RD curve on image datasets





Results - latent bits vs MLP bits

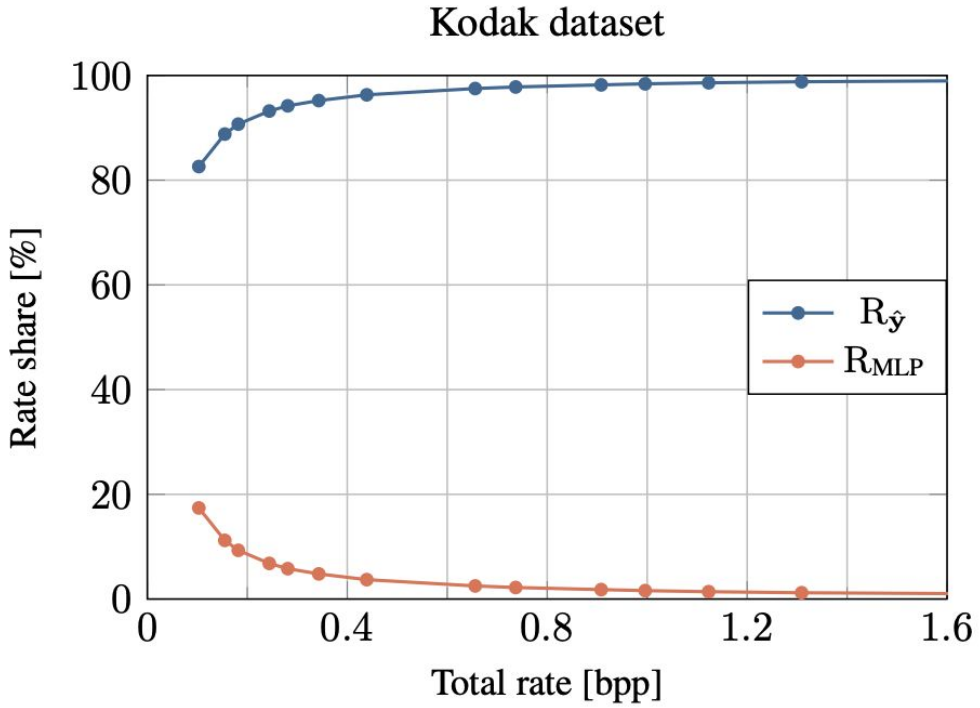
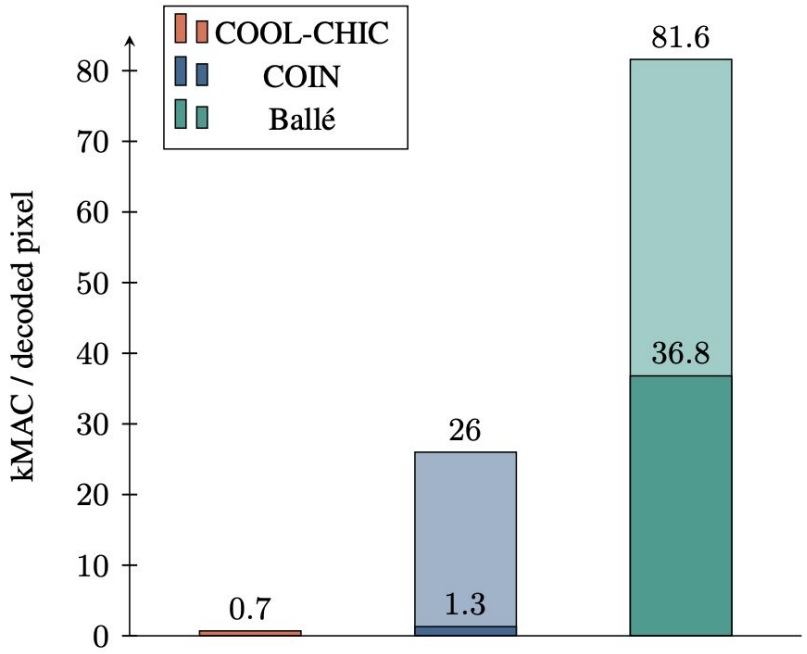


Figure 5: Contributions of the two rate terms: the MLPs rate R_{MLP} and the latent variable rate $R_{\hat{y}}$.

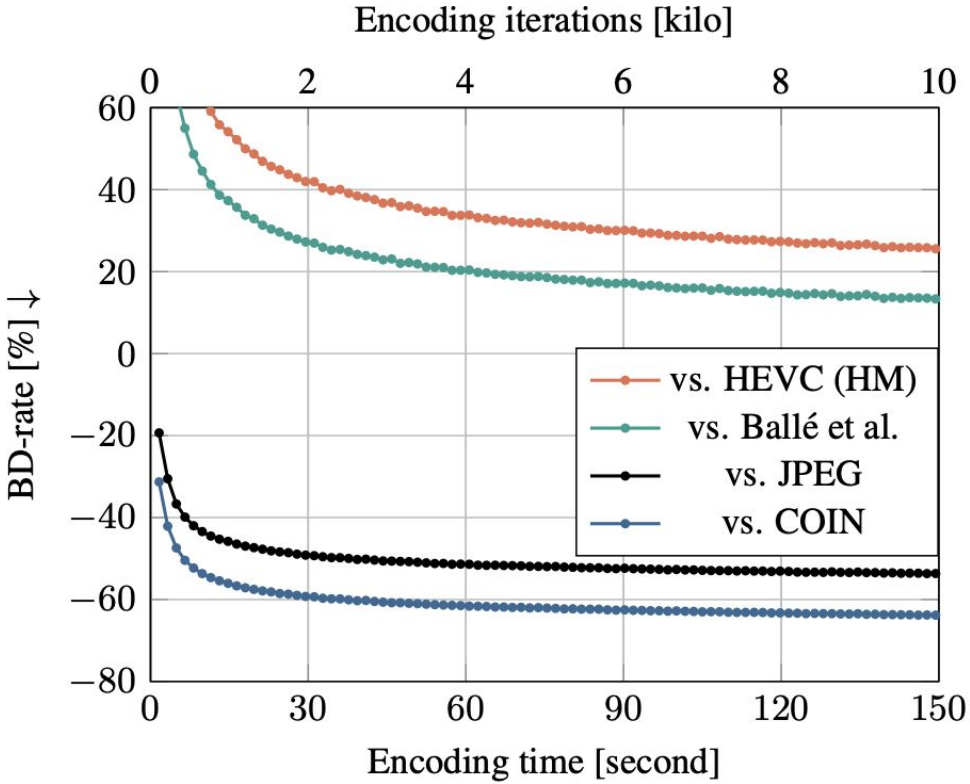
Results - Decoding complexity & performance



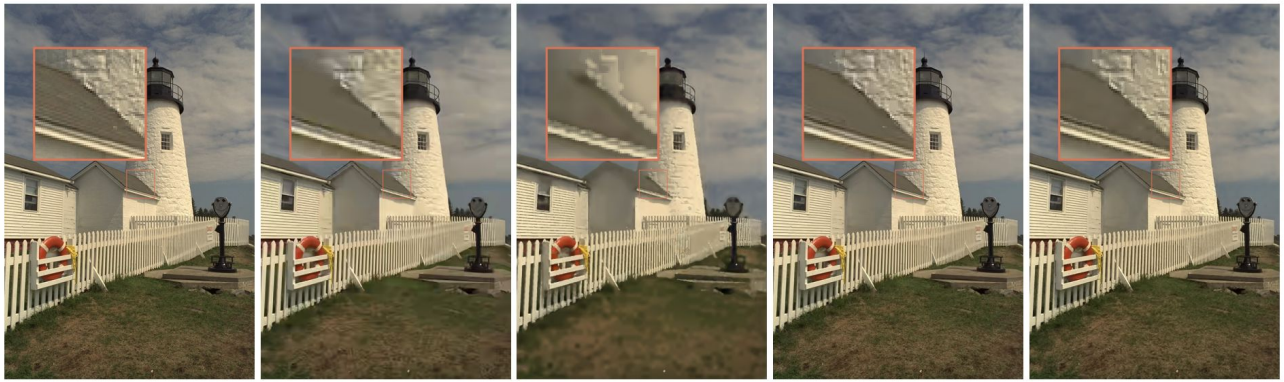
(a) Learned decoders minimum and maximum complexity.

Results - Encoding Time

the encoding of a 768 × 512 image takes around 10 minutes and 40 000 iterations. Yet, better implementation (e.g.

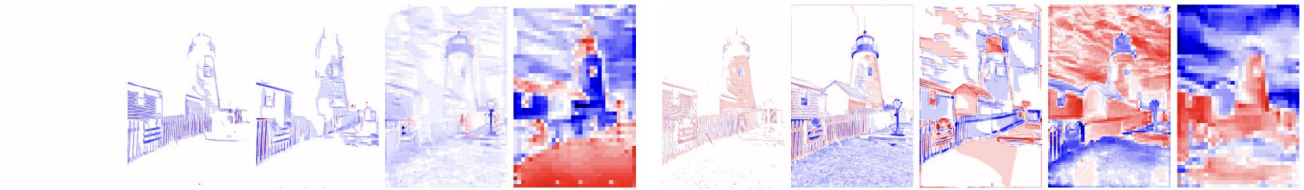


Results - Qualitative



(a) Original (b) HEVC 0.142 bpp (c) Ours 0.145 bpp (d) HEVC 0.578 bpp (e) Ours 0.589 bpp

Figure 8: Comparison of HEVC and COOL-CHIC on Kodak image *kodim19* at two different rates.



(a) Rate is 0.145 bpp. (b) Rate is 0.589 bpp.

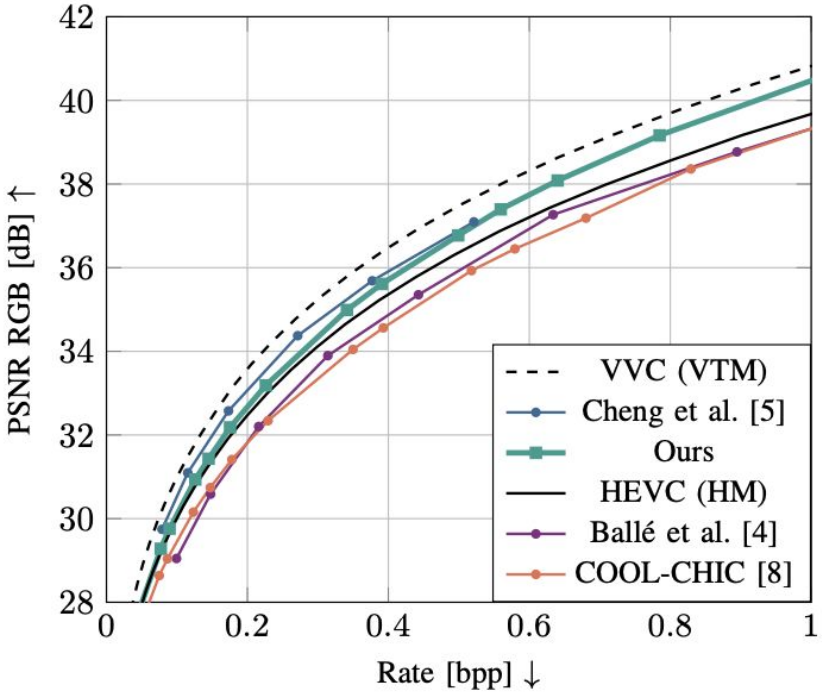
Figure 9: Latent variables for two rates (best viewed on screen). Captions indicate the resolution of each latent channel.

COOL-CHIC-v2 (follow up)

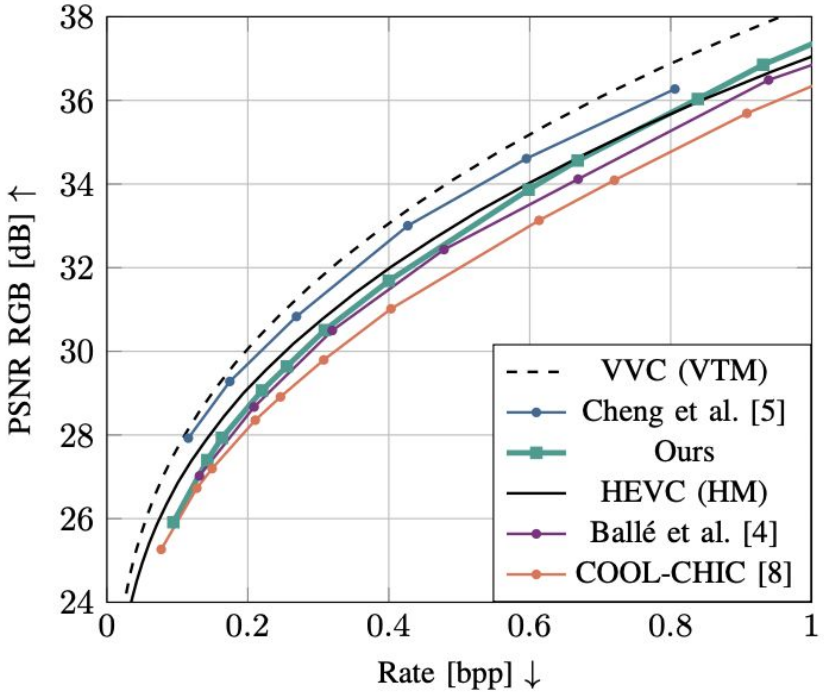
Low-complexity Overfitted Neural Image Codec

Thomas Leguay, Théo Ladune, Pierrick Philippe, Gordon Clare, Félix Henry
Orange Innovation, France
firstname.lastname@orange.com

Olivier Déforges
IETR, France
olivier.deforges@insa-rennes.fr



(a) CLIC 2020 professional validation set.



(b) Kodak dataset.

COOL-CHIC-v2 (follow up)

Low-complexity Overfitted Neural Image Codec

Thomas Leguay, Théo Ladune, Pierrick Philippe, Gordon Clare, Félix Henry
Orange Innovation, France
firstname.lastname@orange.com

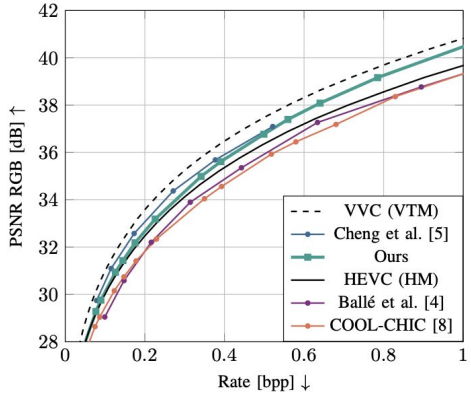
Olivier Déforges
IETR, France
olivier.deforges@insa-rennes.fr

COOL-CHIC-v2 surpasses both Balle & HEVC. 3 Changes:

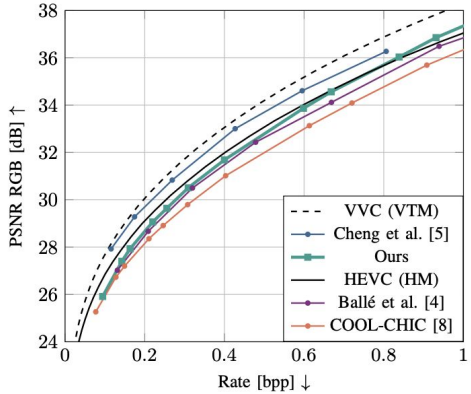
1. Learned Upsampling of latents

2. Extra residual 3x3 Conv layers used for synthesis

3. STE basically sets quantization derivatives to 1. Instead set it to ε and tune hyperparam ε (=0.01)

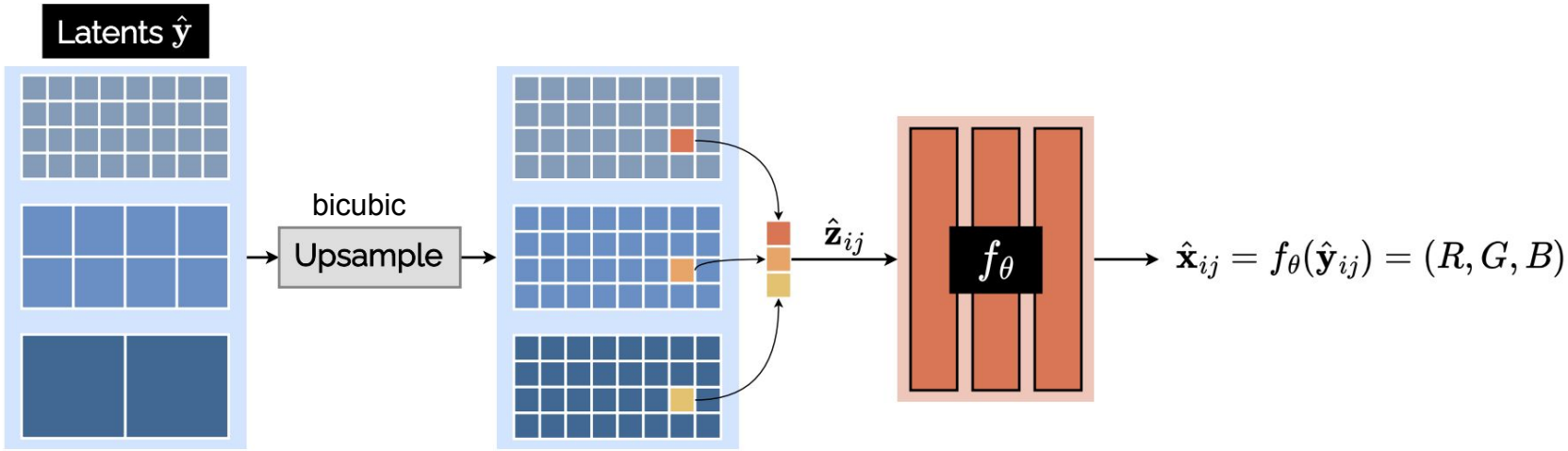


(a) CLIC 2020 professional validation set.



(b) Kodak dataset.

1. Learned upsampling of Latents



Observation: Bicubic interpolation is equivalent to a convolution.

Idea: Replace bicubic interpolation with convolution initialized to bicubic interpolation, and learn this convolution.

2. Extra 3x3 Residual Convs for Synthesis

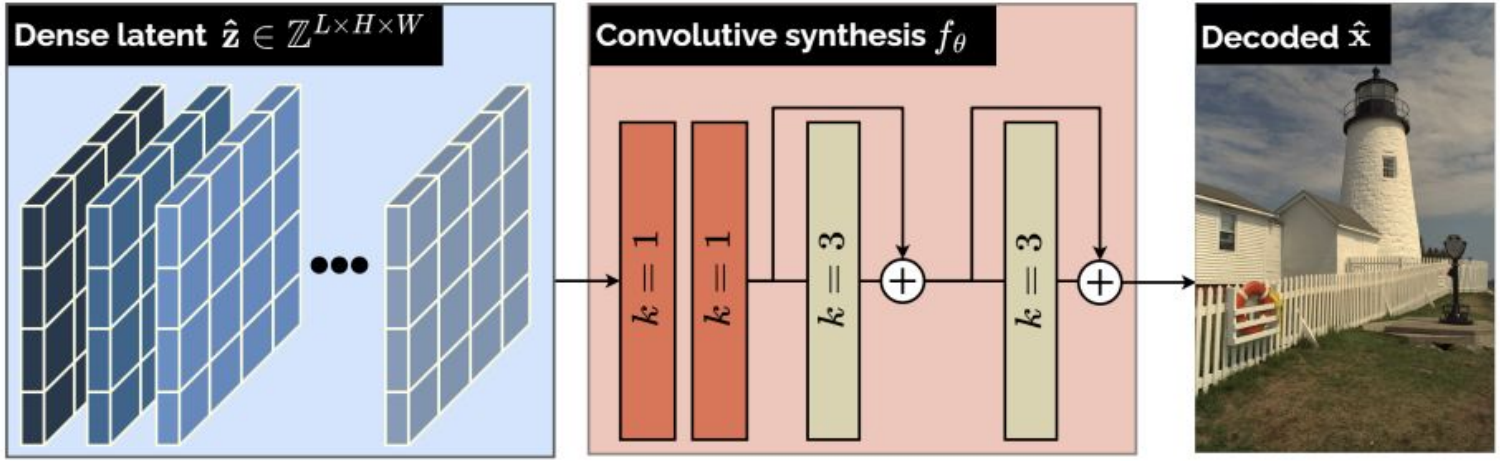


Fig. 4: Convolution-based synthesis function $f_{\boldsymbol{\theta}}$. The kernel size of each layer is denoted by k. More details in Table I.

3. Tweak STE

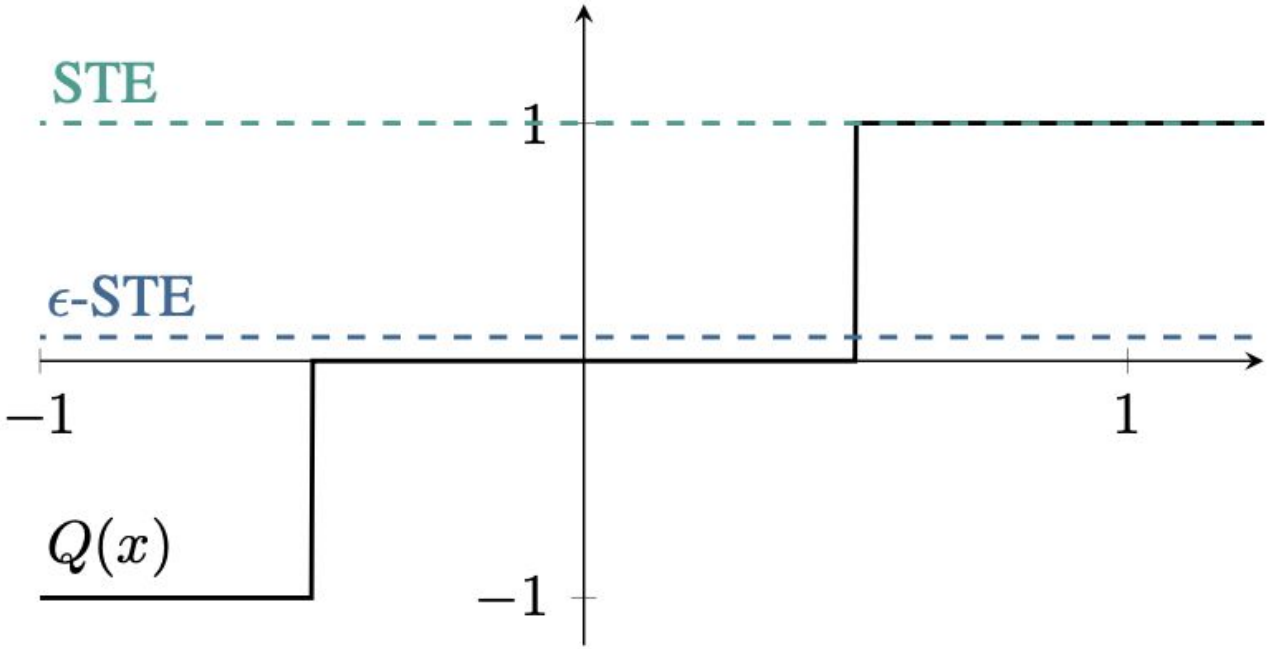


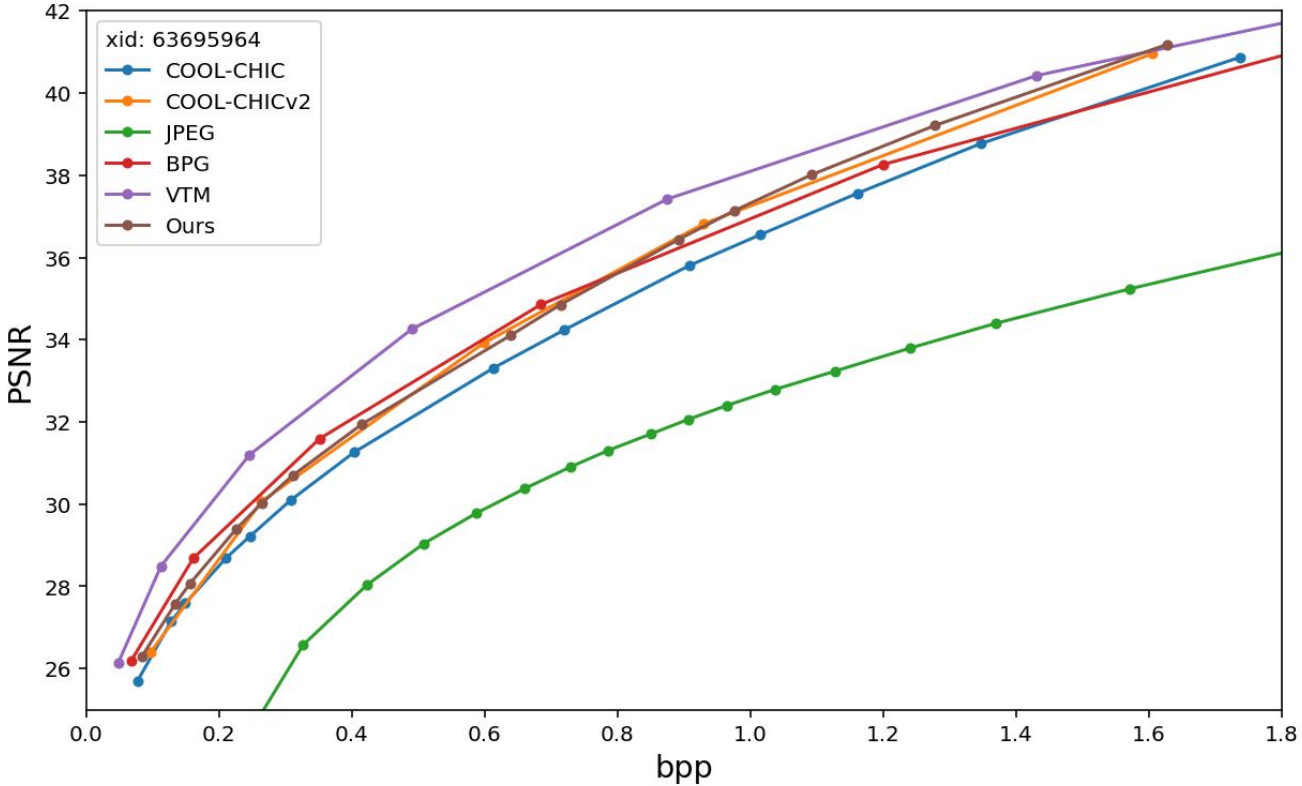
Fig. 6: Different approximations of the quantization derivative.

Ablations

Conv. synthesis	Learned upsample	ϵ-STE	Complexity [kMAC / decoded pix] Synthesis	Upsampling	Total	BD-rate vs. full model [%] ↓
			0.8	0.03	2.5	14.0
✓			0.6	0.03	2.2	9.6
✓	✓		0.6	0.1	2.3	6.0
✓	✓	✓	0.6	0.1	2.3	0.0

TABLE III: Ablation study of the main configuration on CLIC 2020. ARM f_ψ remains identical for all tests and has a complexity of 1.6 kMAC / decoded pixel. When disabled, ϵ-STE is replaced by STE.

Reproducing COOL-CHIC on images



RD results on images (Kodak, CLIC2020)

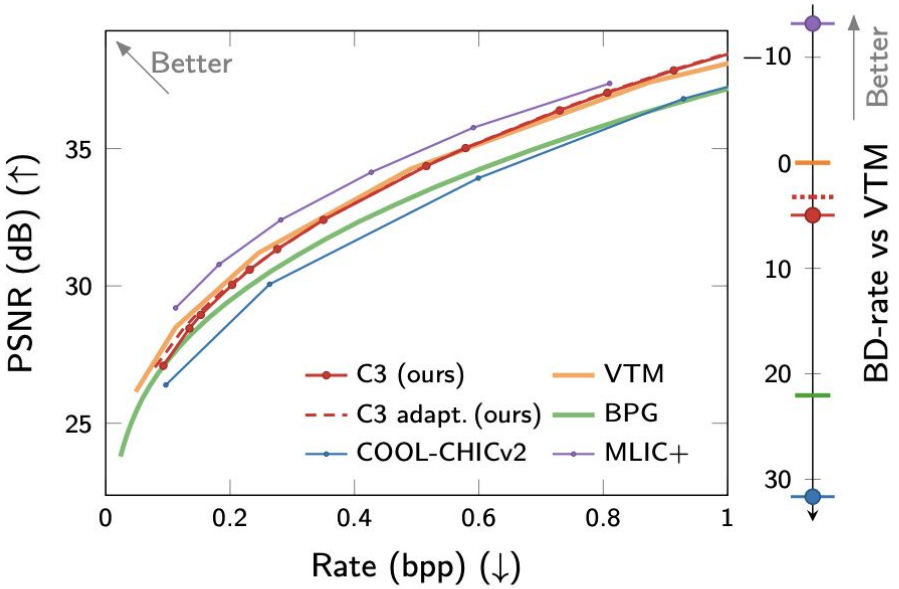


Figure 5. Rate-distortion curve and BD-rate on Kodak.

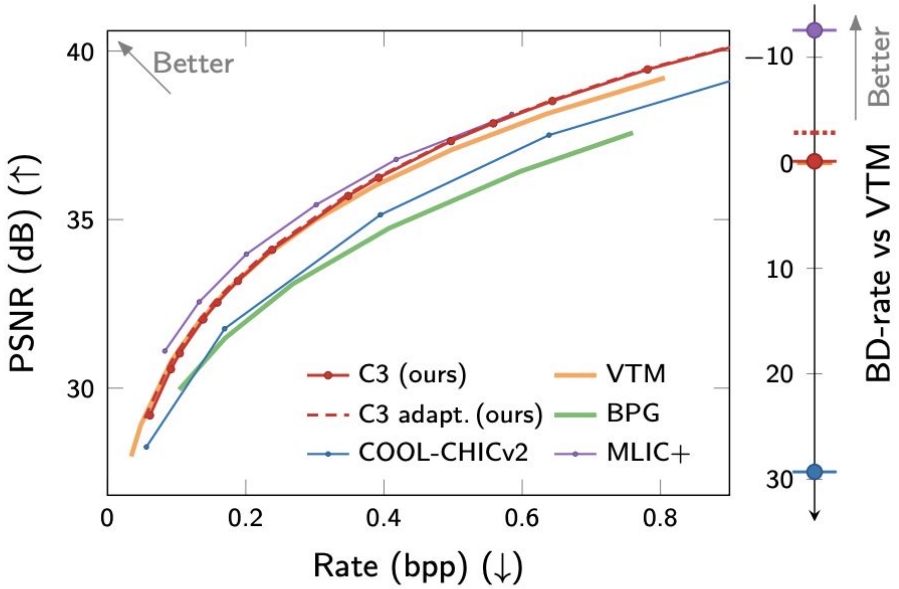


Figure 6. Rate-distortion curve and BD-rate on CLIC2020.

BD-rate vs MACs on images

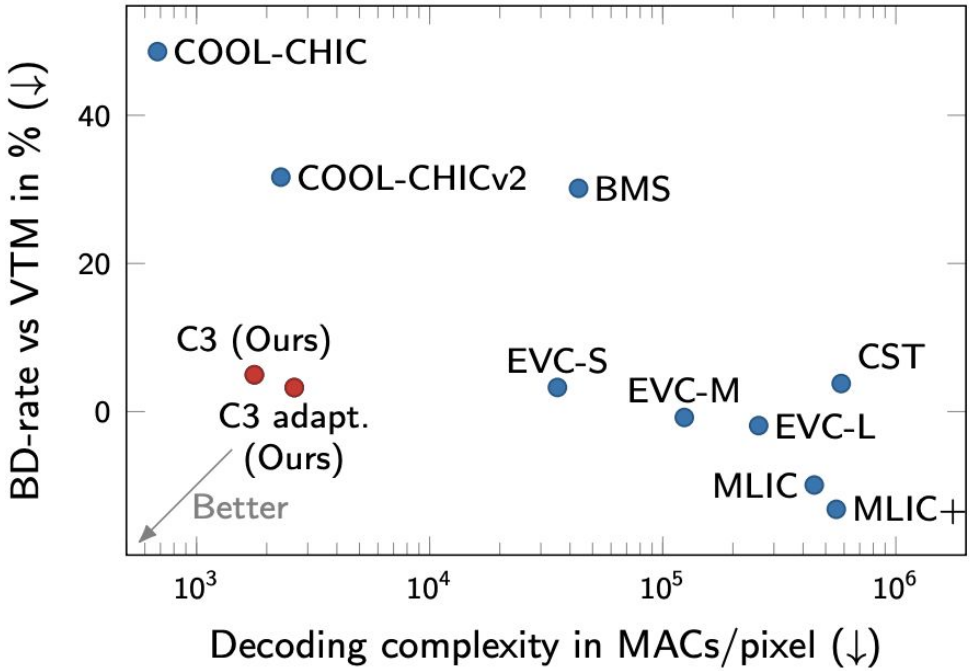


Figure 1. Rate-distortion performance (BD-rate) vs. decoding complexity on the Kodak image benchmark. Our method, C3, achieves a better trade-off than existing neural codecs.

Results on video (UVG)

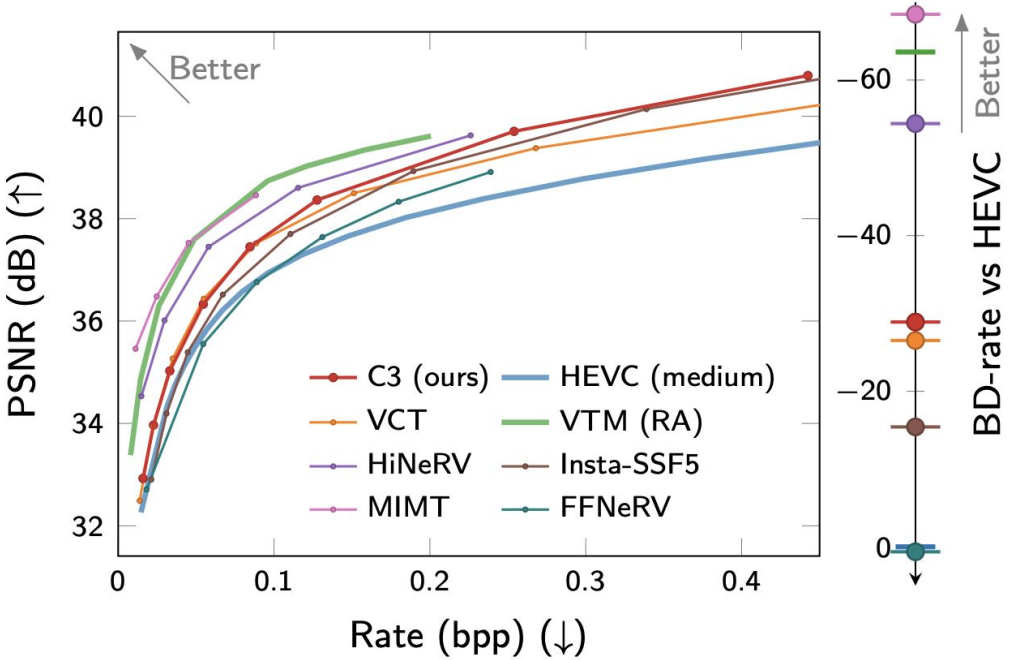


Figure 8. Rate-distortion curve and BD-rate on UVG.

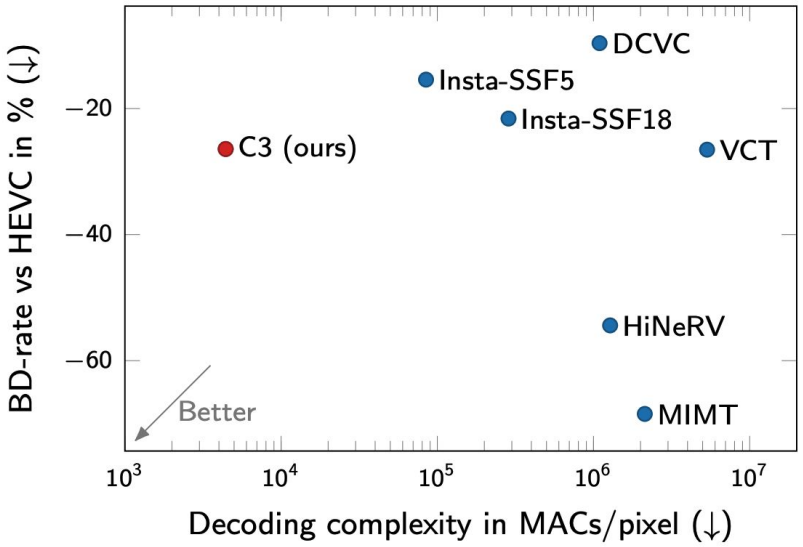
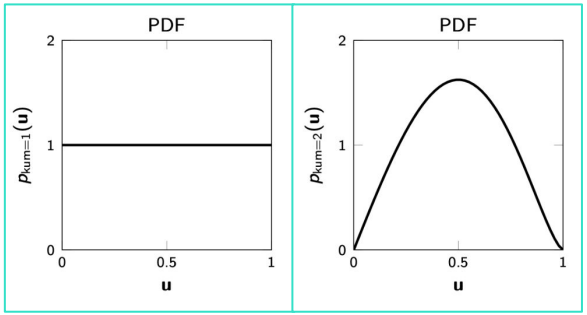


Figure 9. BD-rate vs decoding complexity relative to HEVC (medium). For methods with varying MACs for different bitrates (e.g. C3, HiNeRV), we report the largest MACs/pixel.

Methodological improvements (image & video)

1) Soft-rounding
2) Replace uniform noise with Kumaraswamy noise



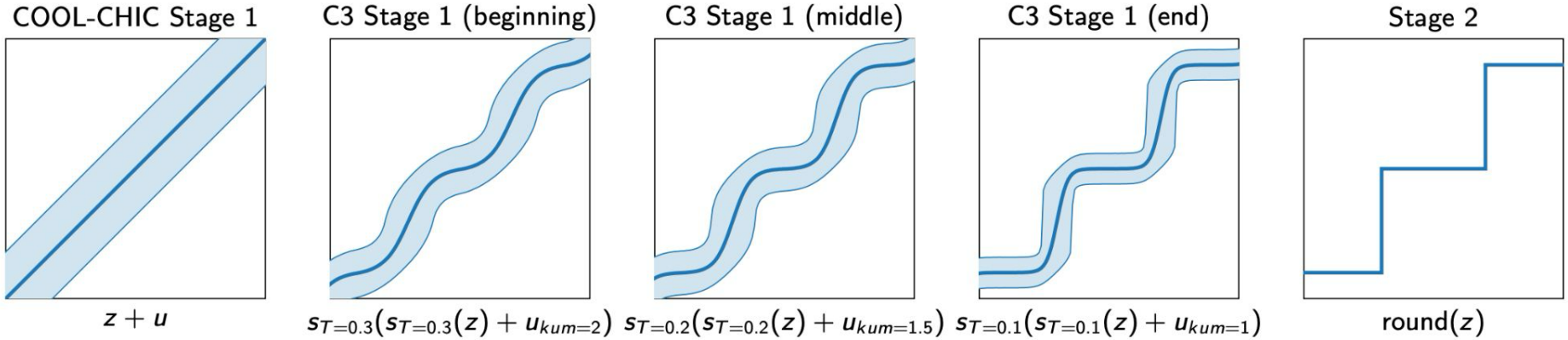


Figure 3. Approximating the round$(\mathbf{z})$ function during Stage 1 of optimization. COOL-CHIC adds uniform noise $\mathbf{u}$, whereas C3 uses soft-rounding s_T with varying temperatures T and Kumaraswamy noise of different strengths, $\mathbf{u}_{\text{kum}}$. We plot the mean and 95% interval.

Methodological improvements (image & video)

- smaller quantization bins
- adaptive learning rate according to loss
- independent entropy model per latent grid
- ReLU -> GELU
- gradient norm clipping
- ...
- (optional) **Adaptive setting**
  - sweep over a handful of architecture choices per image/video to find the best RD-tradeoff on a per-instance basis

Video-specific methodology

1) 2D -> 3D
2) Learned custom masking to capture fast motion
3) Use video patches

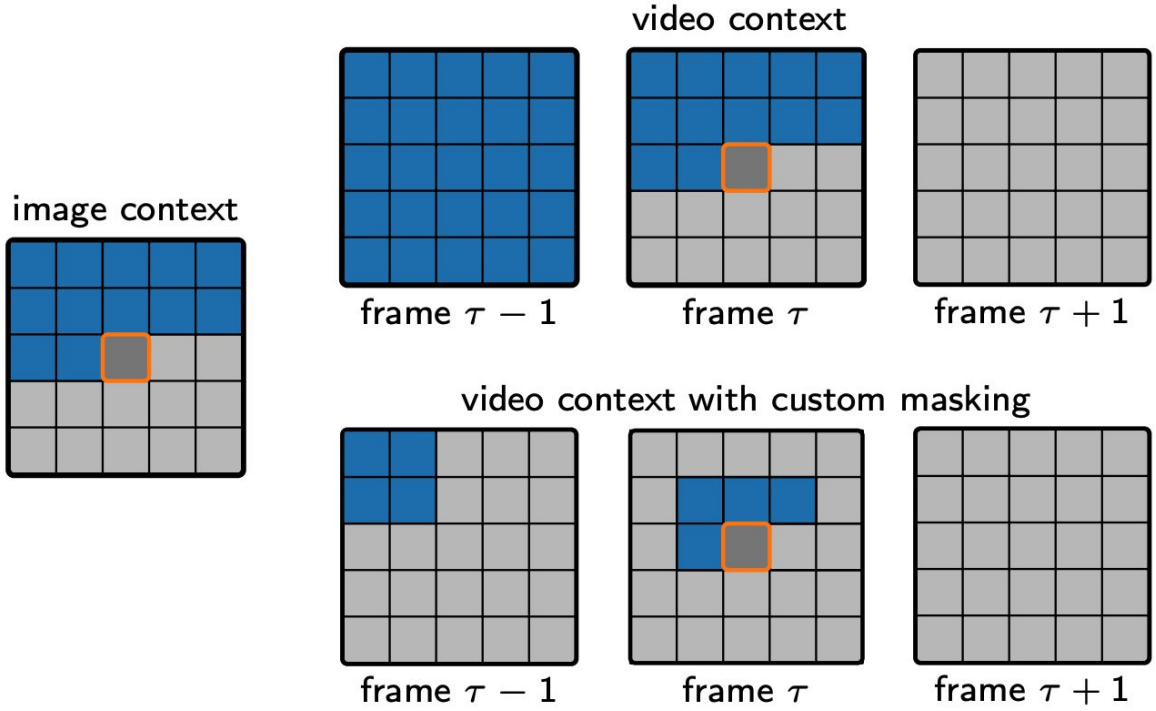


Figure 4. Visualization of entropy model's context for images and video (with and without custom masking).

Reconstructions

Figure 25. Qualitative comparison between C3 (top) and COOL-CHIC v2 (bottom). The PSNR for C3 is 30.28dB and the PSNR for COOL-CHIC v2 is 28.98dB.

Reconstructions

rate ~0.05bpp

Latent visualizations

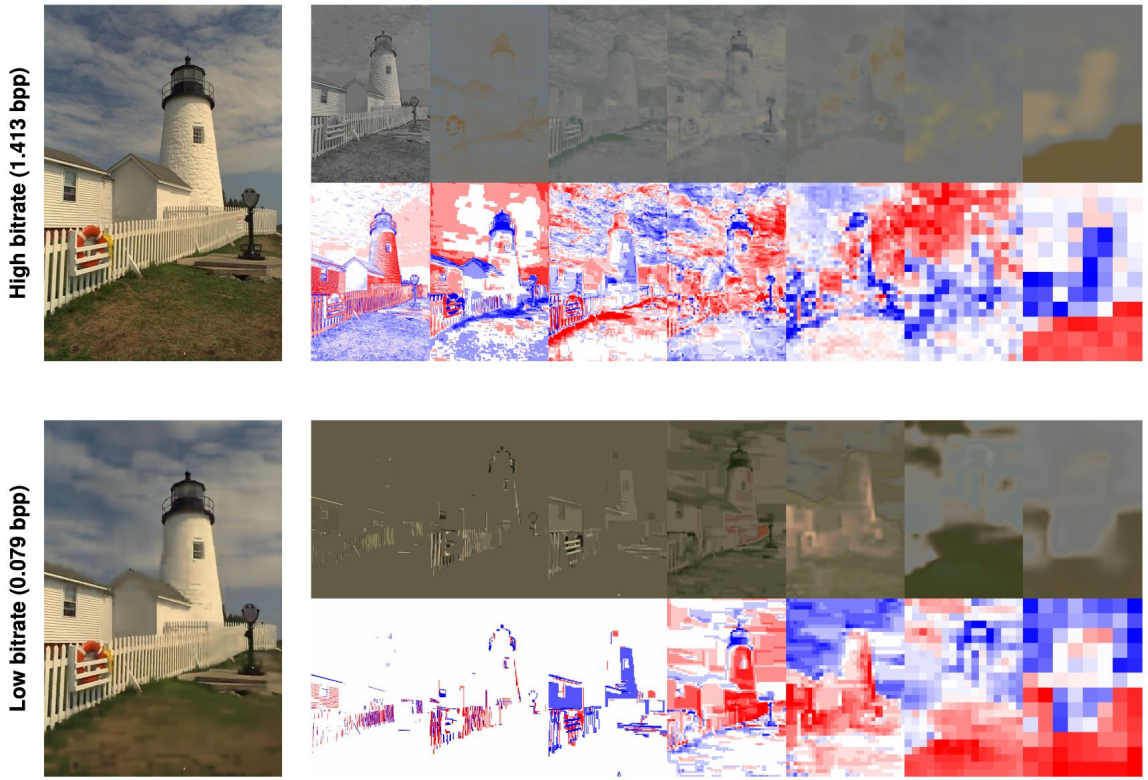
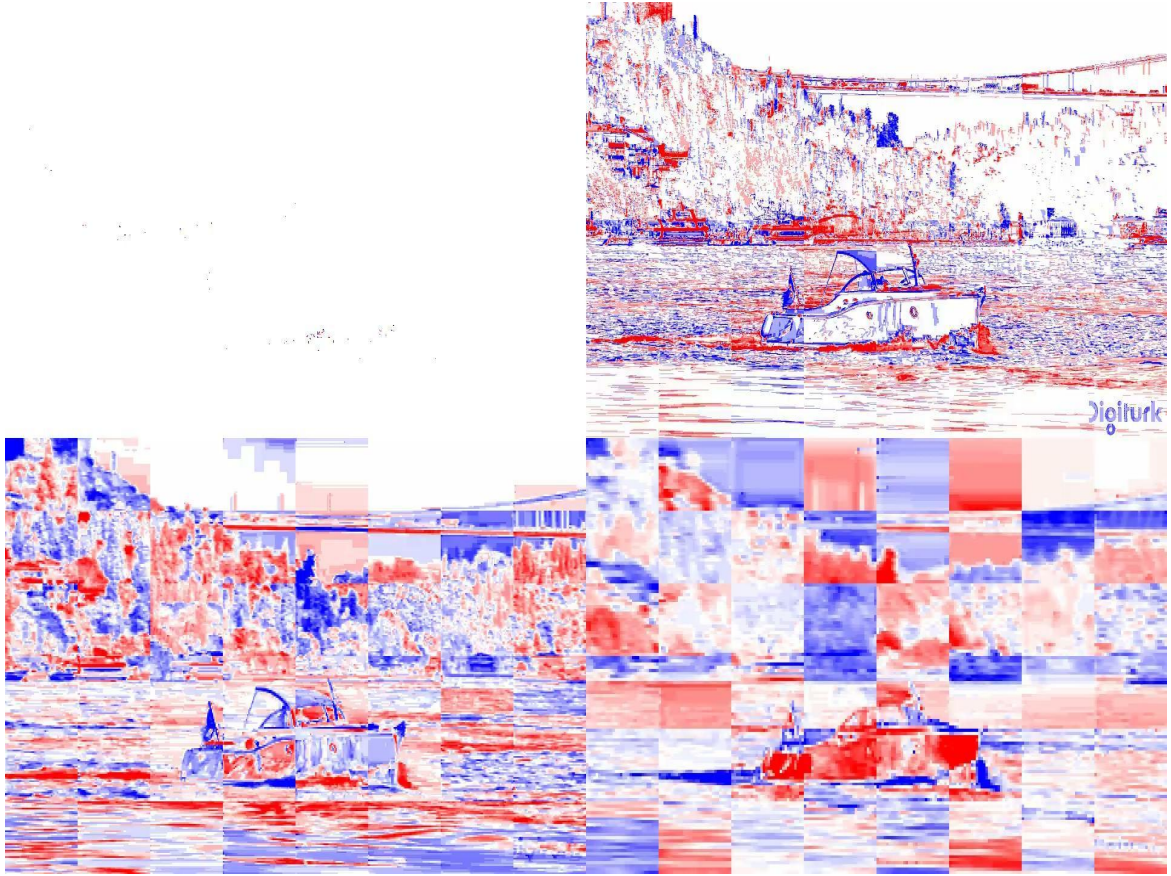


Figure 26. *Top:* Reconstruction and visualization of C3's latents for kodim19 at a high bit-rate (1.413 bpp). The first row shows reconstructions when all but one out of 7 sets of latents is set to zero. For example, the highest resolution latent grid appears to encode luminance information. The second row visualizes the raw latents, upscaled to match the resolution of the output. *Bottom:* As above but at a lower rate (0.079 bpp).

Latent visualizations



Conclusions & Future Work

We have:

- advanced **SOTA RD on single image neural compression**, reaching **VTM level RD performance** with only **3k MACs/pixel.**
- **Extended the method to video compression**, reaching **VCT level RD performance** with only **5k MACs/pixel**.

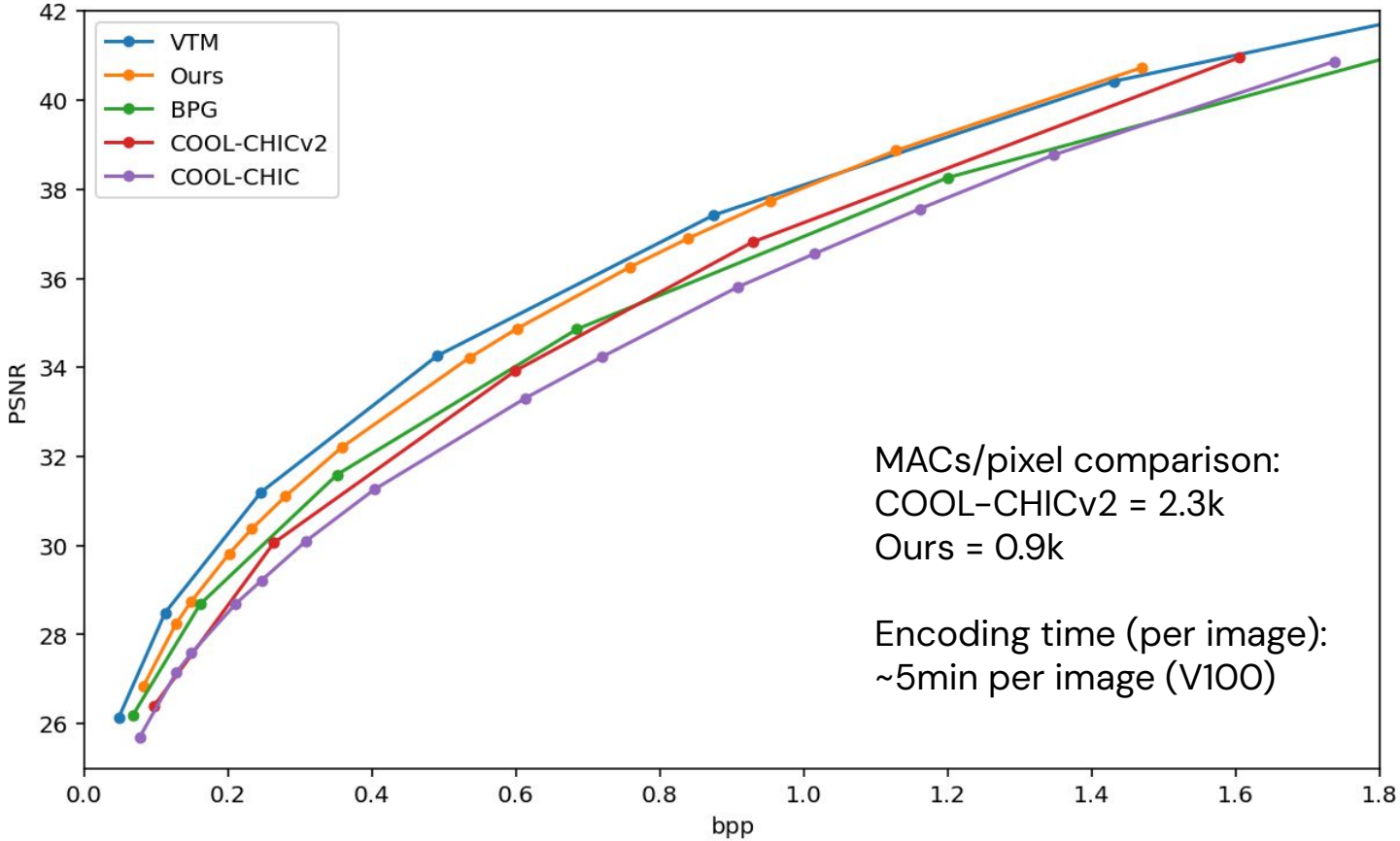
Limitations:

- **Very high encoding complexity**: minutes for images, hours for video patches.
- **Lower MACs != faster decoding**, especially with the autoregressive models we use. Don't yet have an efficient decoding algorithm implemented.

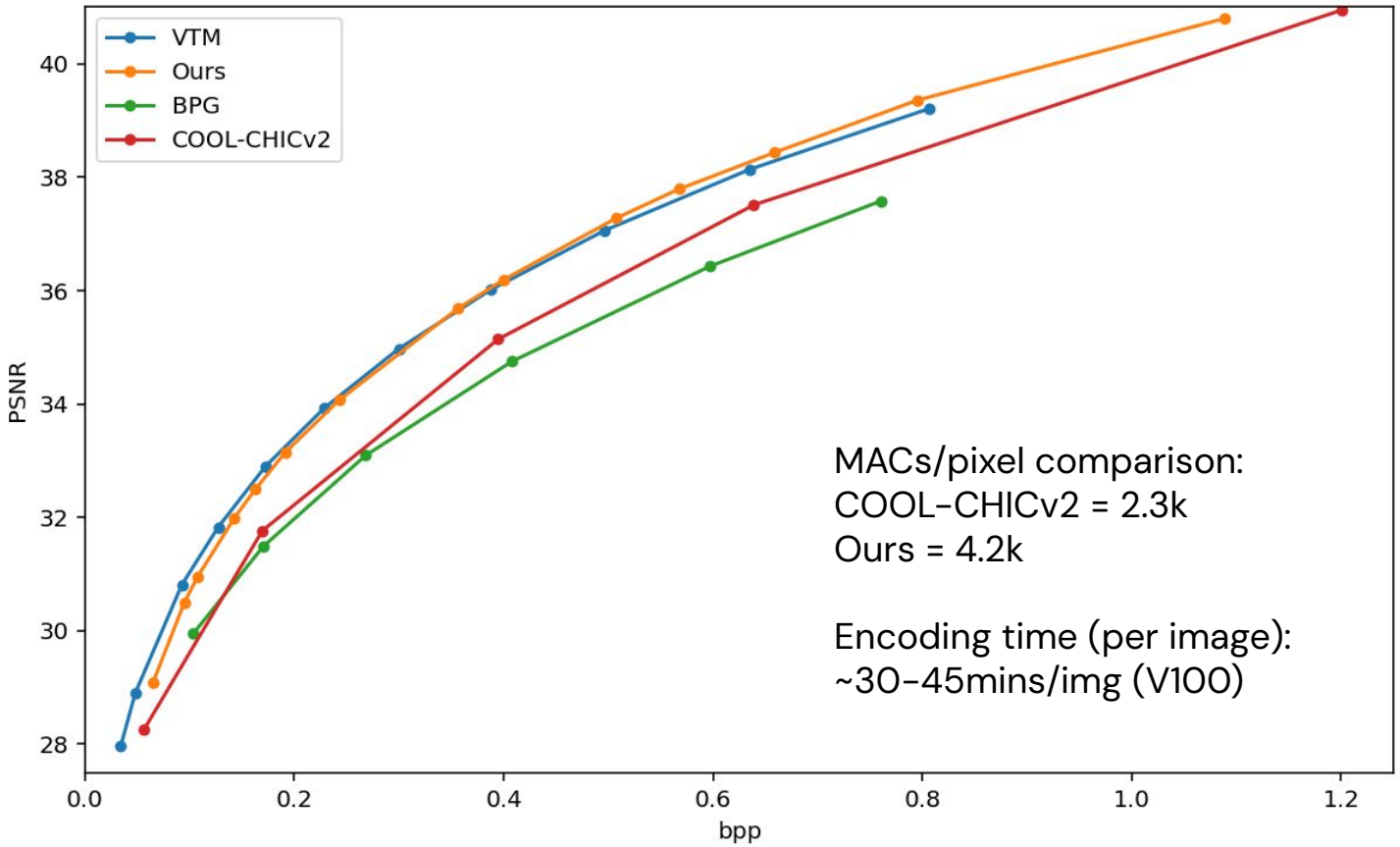
Future work:

- **Speed up encoding** to make the method practical.
- **Remove spatial patching** for videos.
- **Share model parameters** across images/videos for enhanced compression.
- **Non-autoregressive entropy models** that allow for more efficient decoding.

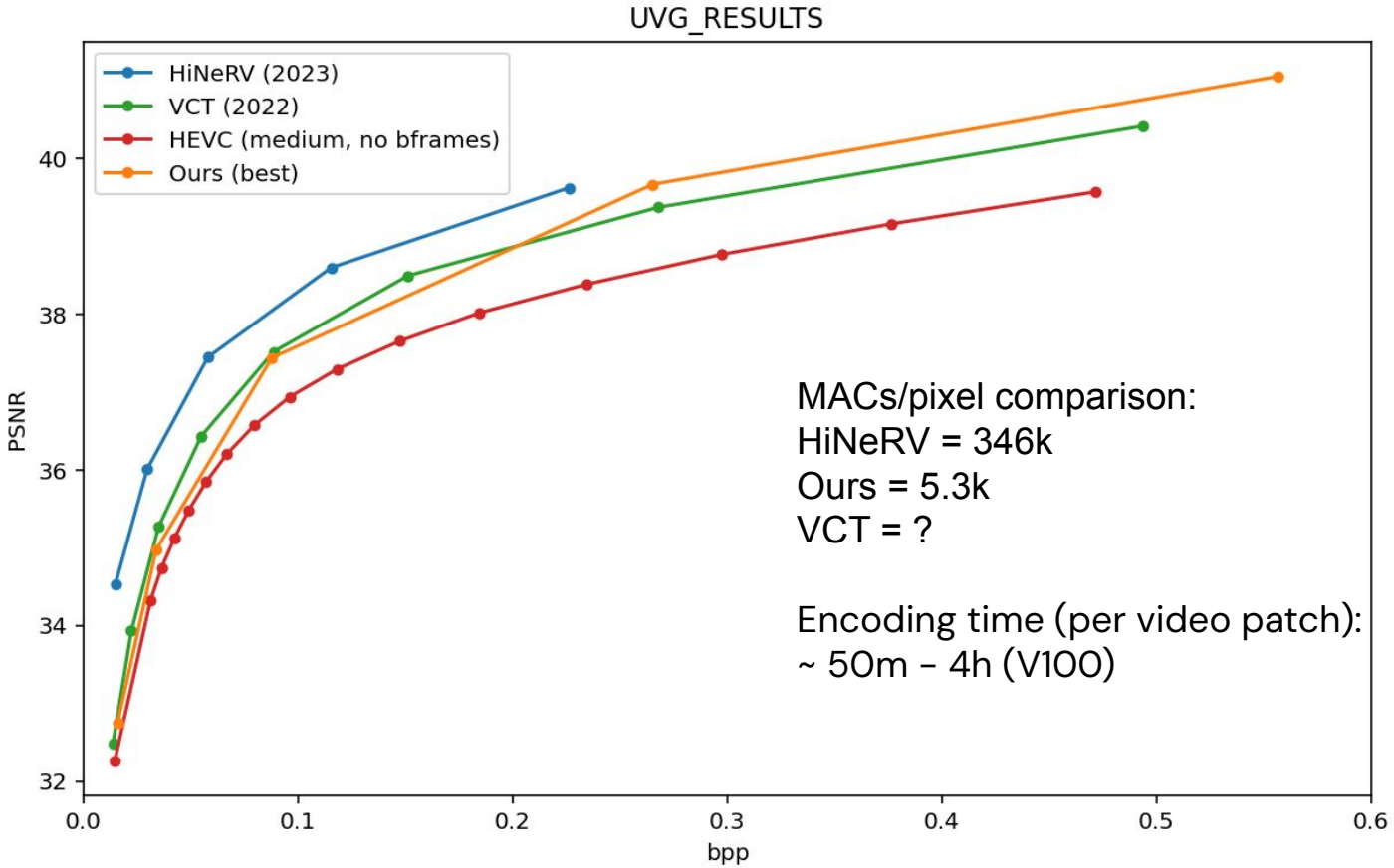
Reproducing COOL-CHIC(v2) and going beyond (KODAK)



Reproducing COOL-CHIC(v2) and going beyond (CLIC2020)

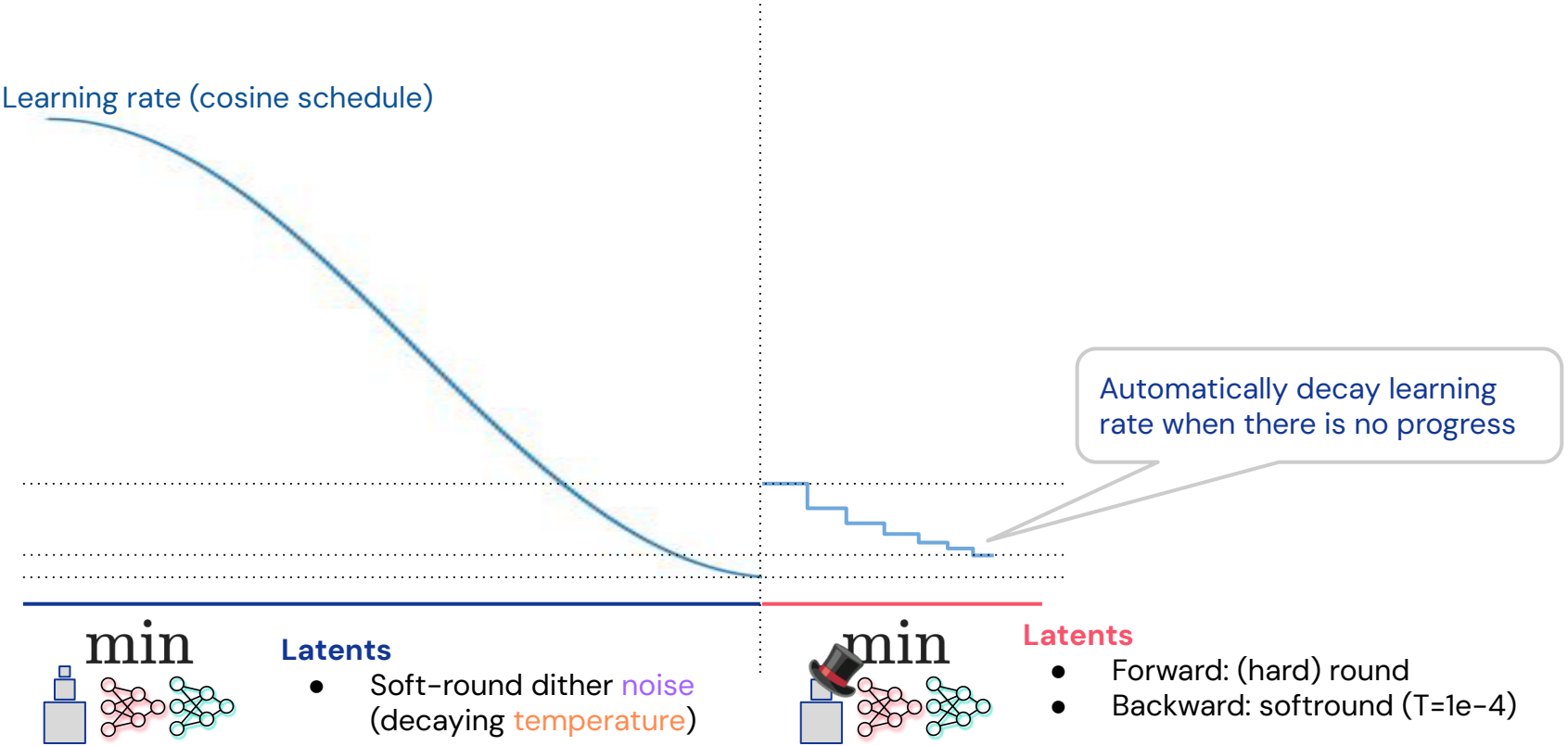


Applying the method to video (UVG)



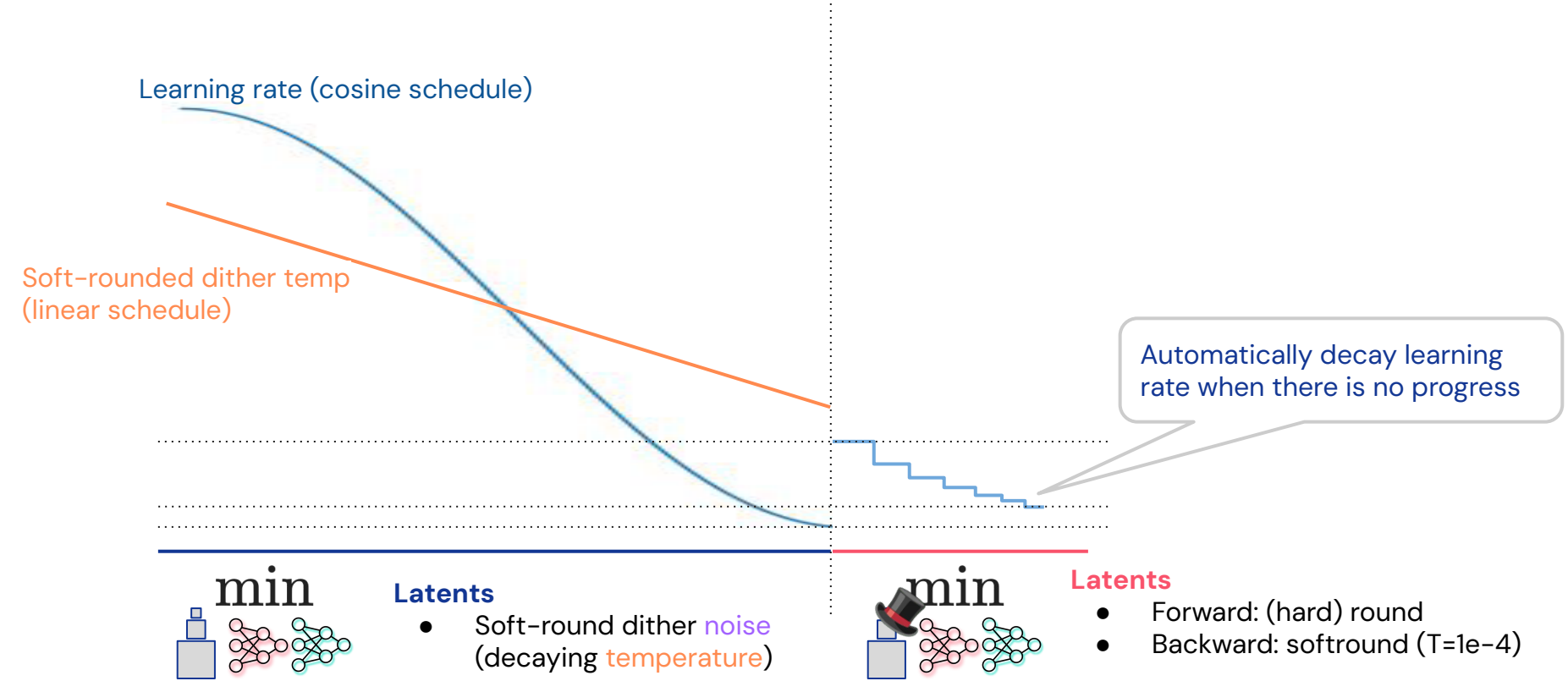
Method: training details

$$\min \left\| \text{🕸}(\text{🧱}) - \text{🖼️} \right\|^2 - \lambda \log p_{\text{🕸}}(\text{🧱})$$



Method: training details

$$\min \left\| \text{🕸}(\text{🧱}) - \text{🖼️} \right\|^2 - \lambda \log p_{\text{🕸}}(\text{🧱})$$

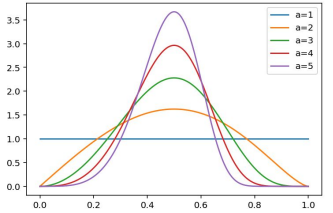


Method: training details

$$\min \left\| \text{🕸}(\text{🤖}) - \text{🖼️} \right\|^2 - \lambda \log p_{\text{🕸}}(\text{🤖})$$

Learning rate (cosine schedule)

Noise strength (linear schedule)
(think: Beta distribution that gets less peaked)



Soft-rounded dither temp
(linear schedule)

Automatically decay learning rate when there is no progress

min

Latents

- Soft-round dither noise (decaying temperature)

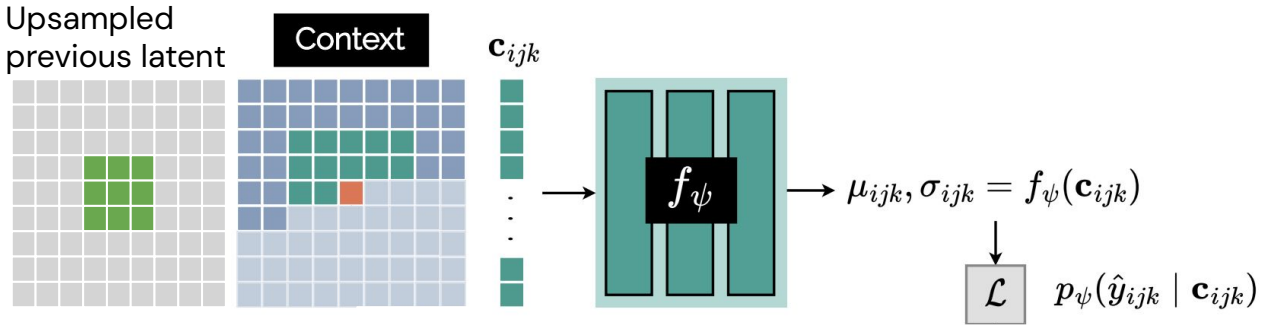
min

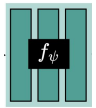
Latents

- Forward: (hard) round
- Backward: softround (T=1e-4)

Method: further architecture tweaks and improvements

- Use GeLU instead of ReLU for non-linearities
- More standard initialisation
- Make entropy-model also autogressive in grids  (look at neighboring resolution)



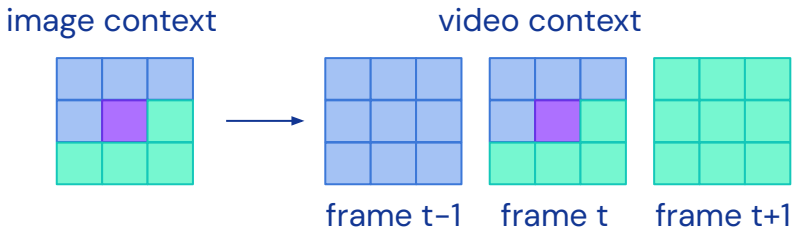
- Change mapping from  to sigma (exp -> softplus)

- More optimised architectures (depth instead of width)

Method: extending to video

2D → 3D

- latents are now 3D.
- entropy model's context is now a 3D causal neighbourhood instead of 2D.



Split video into **spatio-temporal patches**

- video pixels are usually too high dim to fit on single device.
- So fit latents/params to each spatio-temporal patch independently, then aggregate results.

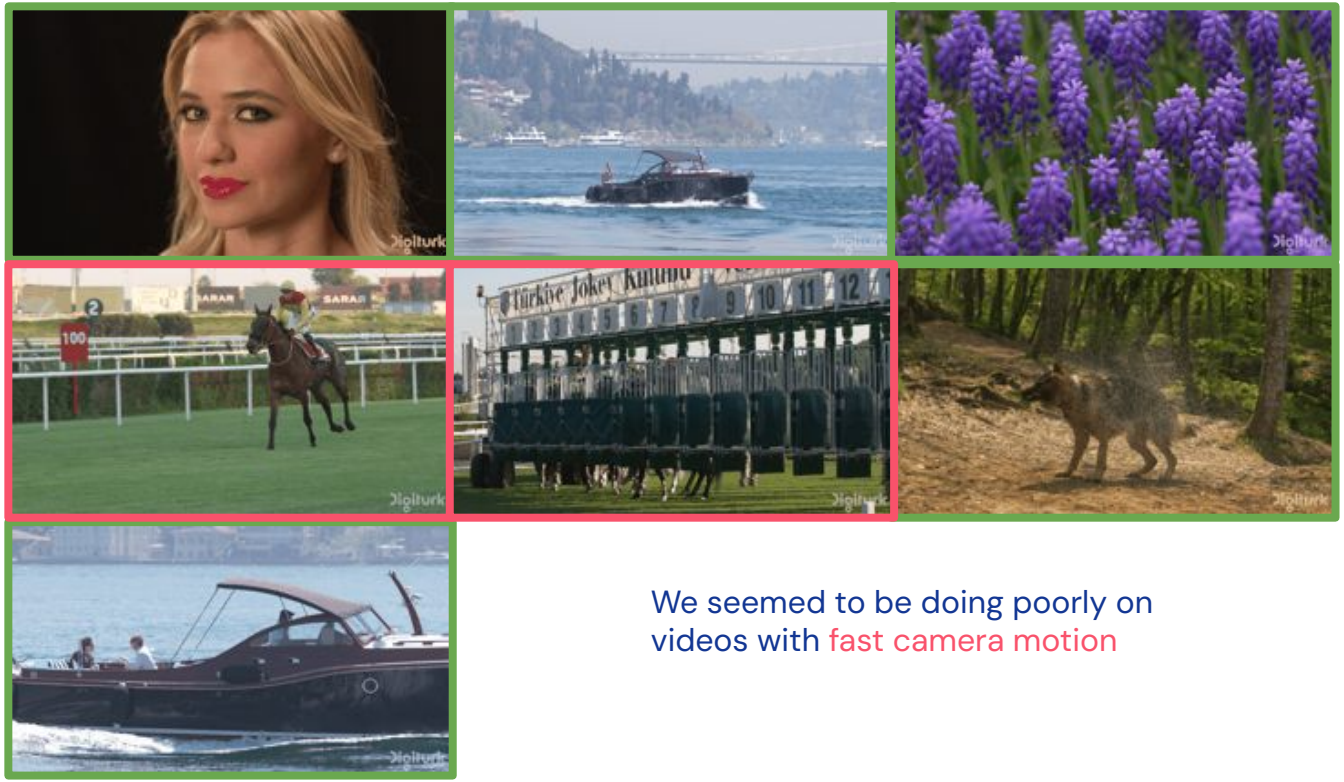
- **Pros**: can parallelize training across patches.
- **Cons**: model not shared across patches, so worse compression than using a single model for whole video.

Method: extending to video

With these changes, we were seeing good performance on some videos but not others.

GOOD

BAD

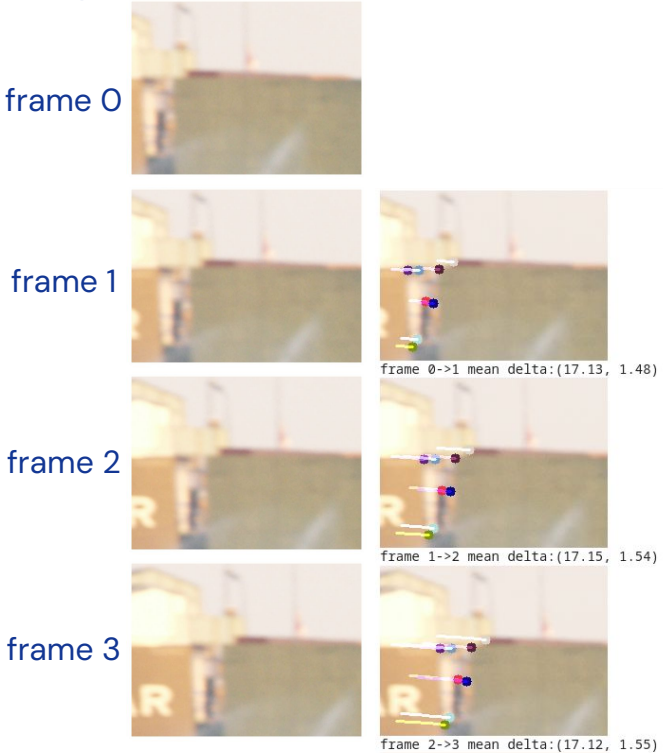


We seemed to be doing poorly on videos with fast camera motion

Consecutive frames of a video patch in a BAD video

With fast camera motion, consecutive frames shift by more than our spatial context.

For this patch the displacement is ~ 17 pixels



Consecutive frames of a video patch in a BAD video

With fast camera motion, consecutive frames shift by more than our spatial context.

For this patch the displacement is ~ 17 pixels

frame 0

frame 1

frame 2

we need this to be within context

Instead we have this, which isn't useful

frame 3

For entropy model to make predictions here,

Consecutive frames of a video patch in a BAD video

With fast camera motion, consecutive frames shift by more than our spatial context.

For this patch the displacement is ~ 17 pixels

frame 0

frame 1

frame 2

we need this to be within context

Instead we have this, which isn't useful

frame 3

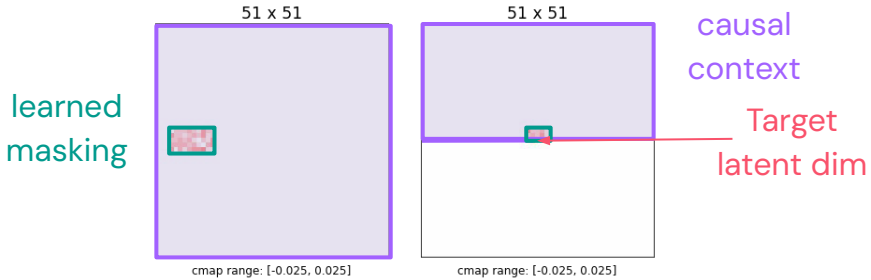
For entropy model to make predictions here,

Potential solution: use a wider spatial context.

Issue: #entropy params scales with context size.

Learned masking for context

We can preserve a small #entropy params if we mask out the context except what's useful for predictions



We learn this mask by:

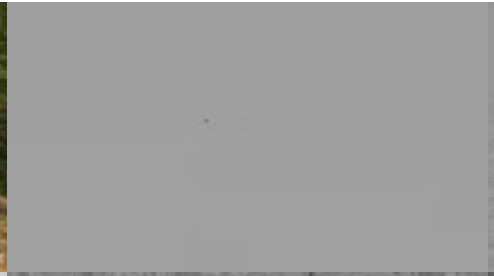
1. Train with a wide spatial context for a few iterations
2. Take the top k context dims by magnitude of entropy layer weights.
3. Choose rectangular mask location to maximize overlap with these context dims
4. Retrain from scratch with this rectangular mask.

Video latent visualizations

low bitrate:

original (T, H, W)

latents_0 (T, H, W)



latents_1 (T//2, H//2, W//2)

latents_2 (T//4, H//4, W//4)

latents_3 (T//8, H//8, W//8)

latents_4 (T//16, H//16, W//16)

Video latent visualizations

high bitrate:

original (T, H, W) latents_0 (T, H, W) latents_1 (T//2, H//2, W//2)

latents_2 (T//4, H//4, W//4) latents_3 (T//8, H//8, W//8) latents_4 (T//16, H//16, W//16)

Thank you!

Jonathan Richard Schwarz - schwarzjn@gmail.com @schwarzjn_